UNIVERSITY OF TORINO

M.Sc. in Stochastics and Data Science

Final dissertation



UNIVERSITÀ
DI TORINO

Detecting Measurable Residual Disease in B-cell Acute Lymphoblastic Leukaemia using Convolutional Neural Networks

Supervisor: Francesca Romana Crucinio
Supervisor: Yvan Saeys

Candidate: Enrico Didoli

Academic Year 2025/2026

Abstract

Measurable Residual Disease (MRD) detection in Acute Lymphoblastic Leukaemia (ALL) represents a rare-event classification challenge in multiparameter flow cytometry (MFC) data. Malignant cells may represent as few as 0.01% of total events, introducing significant difficulties for computational methods.

In this thesis, we investigated CellCNN for MRD detection. CellCNN is a neural network architecture specialized in the detection of rare cell subsets associated with disease, where single-cell measurements are represented as high-dimensional vectors of marker expressions. The original architecture generates multiple convolutional filters that are applied directly to cell expression vectors, followed by a max-pooling layer to extract the strongest filter responses. It produces a binary label at sample-level by predicting a subset of cells rather than processing the entire sample.

Methodological changes to the CellCNN model are proposed, to improve performance and prediction robustness in MRD detection. Specifically, we introduce a data augmentation strategy designed for flow cytometry distributions, which generates multiple Artificial Samples per patient to reduce the variability in the number of malignant cells across patients' samples. This enables the model to learn from synthetic samples with controlled, low percentages of malignant cells. Additionally, we develop a Prediction Approach that is particularly useful when multiple samples are processed within the same testing session, reducing limitations present in the original CellCNN framework. It generates and aggregates predictions across multiple cell subsets drawn from each sample. We also propose a threshold-tuning procedure for the final classification step to ensure reliable classification.

We evaluate the proposed methods across four experimental configurations, comparing the original CellCNN framework against the effect of Artificial Samples generation and an ensemble-based training. Using a Leave-One-Patient-Out Cross-Validation, classification is performed and results are reported in terms of F1-score, Recall, Precision and Accuracy, which are also employed during hyperparameter optimization and training monitoring.

Keywords: Measurable Residual Disease, Flow Cytometry, CellCNN, Class Imbalance

Acknowledgements

I would like to express my gratitude to my supervisors, Prof. Crucinio and Prof. Saeys, for their guidance and support throughout this project. My sincere thanks also to postdoctoral researchers Sarah and Sofie and to PhD student Robbe for their patient and continuous support. Their expertise and feedback have been fundamental to the development of this thesis.
I am also grateful to the Ghent University Hospital in Belgium for providing the clinical dataset. Working with real-world medical data enriched my research experience.
My sincere thanks go to the University of Turin and the professors of the M.Sc. in Stochastics and Data Science programme. The academic environment and the mathematical foundations I acquired here have been the solid ground that allowed the build of this work.
I would like to thank my friends Paolo and Niccolò, with whom I shared hours of study, and moments of joy during these past years. I would also like to thank my friend Alessio for his support during our evenings out.
I am grateful to my mother, my father, my brothers Michele and Giorgio and my sister Irene for their encouragement, patience and support, which allowed me to focus entirely on my university journey. In particular, thanks to my brother Michele, who hosted and put up with me during my Erasmus period in Ghent.
A special thanks to my girlfriend Alessia for consistently supporting me throughout these university years and all my related projects. Thanks to her, I believed in myself enough to begin this journey, which I would probably never have started without her support.
Finally, I would like to thank all those who, in one way or another, supported and accompanied me throughout these years of study.

Ringraziamenti
Vorrei ringraziare i miei amici Paolo e Niccolò per aver condiviso ore di studio e momenti di divertimento durante il mio percorso universitario. Grazie anche ad Alessio per avermi ascoltato e sopportato durante le nostre uscite.
Vorrei ringraziare mia madre, mio padre, i miei fratelli Michele e Giorgio e mia sorella Irene per il loro incoraggiamento, pazienza e supporto che mi hanno permesso di concentrarmi pienamente sul mio percorso universitario. In particolare, grazie a mio fratello Michele per avermi ospitato e sopportato durante il mio Erasmus a Ghent.
Infine, un ringraziamento speciale va ad Alessia, la mia fidanzata, per il suo costante supporto durante questi anni di università e di progetti. Grazie a lei ho potuto superare la paura di non poter recuperare le conoscenze perse durante i due anni di lavoro precedenti alla mia iscrizione all'università, e iniziare un percorso che altrimenti non avrei mai intrapreso.

Contents

List of Symbols **8**

1 Introduction **10**
1.1 Acute lymphoblastic leukaemia (ALL) . . . 10
1.1.1 Immunophenotyping and LAIP . . . 10
1.2 Measurable Residual Disease (MRD) . . . 10
1.3 Multiparameter Flow Cytometry and Data Representation . . . 11
1.4 Raw MFC data preprocessing . . . 11
1.5 Manual Gating . . . 12
1.6 Computational Methods in MFC: Machine Learning Approaches . . . 13
1.7 Artificial Neural Networks: Fundamentals and Architecture . . . 13
1.7.1 Feedforward Neural Networks . . . 14
1.7.2 Backpropagation . . . 16
1.7.3 Optimization: Adaptive moment estimation (ADAM) . . . 18
1.7.4 Convolutional Neural Networks . . . 21
1.8 CellCNN . . . 22
1.8.1 Convolutional Layer as a Feature Detector . . . 23
1.8.2 Pooling and Global Representation . . . 24
1.8.3 Sample Classification . . . 24

2 Objectives **26**

3 Methodology **27**
3.1 Data Introduction . . . 27
3.2 Cells Expression Features . . . 28
3.3 Patient-Level Train/Validation/Test Splits . . . 29
3.4 Data Augmentation: Artificial Samples Generation . . . 29
3.4.1 Parameter Definition . . . 29
3.4.2 Patient-Level Cell Splitting . . . 30
3.4.3 Construction of Artificial Samples (AS) . . . 30
3.4.4 Processing Unhealthy or Mixed Patients . . . 31
3.4.5 Processing Healthy Patients . . . 32
3.5 Metrics . . . 33
3.5.1 Descriptive Statistics and Distribution measures . . . 33
3.5.2 Metrics for Model Evaluation . . . 34
3.6 CellCNN Classification approach . . . 35
3.7 Sample-Level Classification Approach . . . 36
3.7.1 Dataset Resampling . . . 36
3.7.2 Subsets Predictions . . . 36
3.7.3 Positivity Score . . . 37
3.7.4 Classification Function . . . 37
3.7.5 Threshold Optimization on the Validation Set . . . 38
3.7.6 RES-tuning approach . . . 38
3.7.7 ROC-tuning approach . . . 39

3.8 Training Procedures . . . 40
3.8.1 Parameters Tuning via Bayesian Optimization . . . 40
3.8.2 Training Procedure 1: Randomized Split and Multi-Seed Initialization 42
3.8.3 Training Procedure 2: Cross-Validation Ensemble . . . 42
3.8.4 Sample Classification . . . 43
3.8.5 Leave-One-Patient-Out Cross-Validation . . . 43
3.9 Experimental Setup . . . 44
3.9.1 Implemented Parameters . . . 44
3.10 Detailed walkthrough of LOPOCV iteration 1 . . . 47
3.10.1 RES-Tuned Threshold . . . 50
3.10.2 Test prediction and Labelling: Procedure 2 . . . 52

4 Results **57**
4.1 Tuned θ^* per LOPOCV fold . . . 57
4.2 Tuned thresholds per LOPOCV fold . . . 58
4.3 Per-Experiment Results . . . 59
4.3.1 Experiment 1: Training Procedure 1 with AS . . . 59
4.3.2 Experiment 2: Training Procedure 1 without AS . . . 64
4.3.3 Experiment 3: Training Procedure 2 with AS . . . 68
4.3.4 Experiment 4: Training Procedure 2 without AS . . . 73
4.3.5 Experiments 1-4: performance metrics comparison . . . 77
4.3.6 Ensemble Misclassification Table . . . 79
4.3.7 Experiments Comparison . . . 80

5 Discussion **82**
5.1 Answers to the Research Questions . . . 82
5.2 Limitations . . . 83
5.3 Future Works . . . 83

6 Bibliography **85**

List of Figures

1 FNN Architecure . . . 16
2 CellCNN Architecture . . . 23
3 Healthy/Unhealthy cell distribution across samples . . . 28
4 Bayesian Optimization-Explored configurations . . . 48
5 RES-tuning- F_1-score vs. Thresholds . . . 52
6 Performance metrics for Patient 1 . . . 56
7 θ^* LOPOCV iteration . . . 58
8 Experiment 1: Performance metrics under $\tau = 0.5$ . . . 61
9 Experiment 1: Positivity score distribution under $\tau = 0.5$ . . . 61
10 Experiment 1: Performance metrics under $\tau_{ROC}^{*(BO)}$ . . . 62
11 Experiment 1: Positivity score distribution under $\tau_{ROC}^{*(BO)}$ . . . 62
12 Experiment 1: Performance metrics under $\tau_{RES}^{*(BO)}$ . . . 63
13 Experiment 1: Positivity score distribution under $\tau_{RES}^{*(BO)}$ . . . 63
14 Experiment 2: Performance metrics under $\tau = 0.5$ . . . 65
15 Experiment 2: Positivity score distribution under $\tau = 0.5$ . . . 65
16 Experiment 2: Performance metrics under $\tau_{ROC}^{*(BO)}$ . . . 66
17 Experiment 2: Positivity score distribution under $\tau_{ROC}^{*(BO)}$ . . . 66
18 Experiment 2: Performance metrics under $\tau_{RES}^{*(BO)}$ . . . 67
19 Experiment 2: Positivity score distribution under $\tau_{RES}^{*(BO)}$ . . . 67
20 Experiment 3: Performance metrics under $\tau = 0.5$ . . . 69
21 Experiment 3: Positivity score distribution under $\tau = 0.5$ . . . 70
22 Experiment 3: Performance metrics under τ_{ROC}^{*} . . . 70
23 Experiment 1: Positivity score distribution under τ_{ROC}^{*} . . . 71
24 Experiment 3: Performance metrics under τ_{RES}^{*} . . . 71
25 Experiment 3: Positivity score distribution under τ_{RES}^{*} . . . 72
26 Experiment 4: Performance metrics under $\tau = 0.5$ . . . 74
27 Experiment 4: Positivity score distribution under $\tau = 0.5$ . . . 74
28 Experiment 4: Performance metrics under τ_{ROC}^{*} . . . 75
29 Experiment 4: Positivity score distribution under τ_{ROC}^{*} . . . 75
30 Experiment 4: Performance metrics under τ_{RES}^{*} . . . 76
31 Experiment 1: Positivity score distribution under τ_{RES}^{*} . . . 76
32 F_1-score comparison across Experiments . . . 77
33 Recall comparison across Experiments . . . 77
34 Precision comparison across Experiments . . . 78
35 Accuracy comparison across Experiments . . . 78

List of Tables

2 Dataset . . . 27
3 Cell features . . . 28
4 Prediction Outcomes . . . 34
5 Open-source libiearies versions . . . 44
6 Tuning-Training shared hyperparameters . . . 45
7 Training-Tuning-Prediction-dependent parameter . . . 45
8 Artificial Samples generation parameters . . . 46
9 Example: Test Set . . . 47
10 Example: Patient distribution across CV folds . . . 47
11 Example: Patients per CV iteration . . . 47
12 Example: Configurations proposed during BO . . . 49
13 Example: Step-by-step θ^* selection procedure . . . 50
14 Example Subset predictions . . . 51
15 Example: Prediction collection $\hat{Y}$ . . . 51
16 Example: Classification results . . . 51
17 Example: Per-fold predictions . . . 53
18 Example: Per-fold subset predictions . . . 54
19 Example: Per-fold mean Positivity scores . . . 54
20 Example: Classification metrics per fold under CellCNN approach . . . 55
21 Example: Classification metrics per fold under Resampling approach . . . 55
22 Experiments Table . . . 57
23 Prediction Approaches table . . . 57
24 Bayesian Optimization: Configurations θ^* for each LOPOCV iteration . . . 58
25 Tuned thresholds per LOPOCV iteration . . . 59
26 Experiment 1 results across all classification approaches . . . 60
27 Experiment 2 results across all classification approaches . . . 64
28 Experiment 3 results across all classification approaches . . . 68
29 Experiment 4 results across all classification approaches . . . 73
30 Ensemble misclassification Table . . . 79

List of Algorithms

1 Stochastic Gradient Descent (SGD) . . . 18
2 Mini-Batch Stochastic Gradient Descent . . . 18
3 ADAM Optimization . . . 21
4 Artificial Samples Generation for Unhealthy or Mixed Patients . . . 32
5 Artificial Samples Generation for Healthy Patients . . . 33
6 Test Set Prediction . . . 37
7 RES Threshold Tuning . . . 39
8 ROC Threshold Tuning . . . 40
9 Optimal Configuration θ^* Selection (Tie-Breaking . . . 42

List of Symbols

Symbol	Description
Data and Cells	
x_i	Expression vector of the i-th cell, $x_i \in \mathbb{R}^{\mathcal{M}}$
$\mathcal{C}$	Number of cells in a sample
$\mathcal{M}$	Number of markers per cell
D	Data matrix, $D \in \mathbb{R}^{\mathcal{C}\times\mathcal{M}}$
f	Biological sample
n_f	Number of cells in the sample f
s	Cell subset drawn from a sample
$\mathcal{P}$	Set of patients
p_i	i-th patient
Neural Network	
$w_{j,k}^{(L)}$	Weight of neuron k in layer $L-1$ to neuron j in layer L
b_j	Bias of the neuron j
$z_j^{(L)}$	Pre-activation signal of the j-th neuron in layer L
$a_j^{(L)}$	Activated value of the j-th neuron
σ	Activation function
$\mathcal{L}$	Loss Function
$\mathcal{L}_{CE}$	Cross-Entropy Loss Function
$\mathcal{L}_{BCE}$	Binary Cross-Entropy Loss Function
W	Vector of all network parameters (weights and biases)
W_t^e	Weights at t-th iteration of the e-th epoch
$\nabla_W \mathcal{L}$	Gradient of the Loss Function computed on W
B_t	t-th mini batch
n_B	Size of a mini-batch
g_t	Gradient estimate at iteration t
g_t^e	Gradient estimate at iteration t during epoch e
E	Number of epochs
α	Number of mini-batches (iterations)
ADAM Optimization	
m_t	Gradient first moment estimate
v_t	Gradient second moment estimate
β_1, β_2	Exponential decay rates
$g_t \odot g_t$ or $(g_t)^2$	Gradient element-wise square
$\hat{m}_t$, $\hat{v}_t$	Bias-corrected m_t, v_t
ϵ	Numerical stability value
Convolutional Neural Network (CNN) and CellCNN	
κ	Kernel size
X	Input layer
$W^{(\Phi)}$	Φ-th kernel weights vector
$\mathcal{F}$	Number of filters
$Z^{(\Phi)}$	Feature map obtained using the Φ-th filter
$\mathcal{Z}$	Pre-activation output vector
Φ	Convolutional filter in CellCNN frmework
$r_{i,\Phi}$	Cell response of the filter Φ to the i-th cell
R	Cells response matrix

Symbol	Description
Optimization and Training	
n_{subs}	Number of subsets drawn per sample by CellCNN
n_{cells}	Number of cells per subset drawn by CellCNN
θ	CellCNN hyperparameter configuration (n_{subs}, n_{cells})
θ^*	Optimal θ
Θ	θ parameter space
$\Omega(\theta_i)$	Objective function of Bayesian Search
s	Subset
n_s	Number of cells in a subset
n_s^{test}	Number of cells drawn in CellCNN classification
$\mathcal{S}_{train}$	Set of samples employed for training
$\mathcal{S}_{val}$	Set of samples employed for validation
$\mathcal{S}_{test}$	Set of samples employed for test
Artificial Samples	
n_{as}	Number of Artificial Samples generated per patient
q	Number of cells per Artificial Samples
u	Vector of unhealthy cell percentages in Artificial Samples
$\mathcal{K}$	Cell phenotype
$n_f^{(\mathcal{K})}$	Number of cells of type $\mathcal{K}$ in the f-th sample
$C_{\mathcal{K}}^p$	Vector of cells of type $\mathcal{K}$ from patient p
N_i^U, N_i^H	Number of unhealthy and healthy cells for the i-th AS
$C_{rem,i}^U$ $C_{rem,i}^H$	Vectors of available unhealthy and healthy cells at the i-th AS generation
AS_k	k-th AS generated
Classification and Metrics	
β	F_β-score parameter to balance Precision and Recall
y_f	Ground Truth label of a sample f
$\hat{y}_f$	CellCNN Prediction of a sample f
$\tilde{y}_f$	Predicted Class of a sample f (using Positivity score)
$\hat{y}_{k,t,j}^{f_i^p}$	Prediction of the subset j from the sample i of the p-th patient, under model k and weights initialization j
$\hat{\mu}_{k,j}^f$	Average prediction across Seeds of the subset j drawn from the sample f
$\hat{\mu}_k^f$	Average prediction across Seeds and Subsets of the sample f
M_f	Positivity score computed from the sample f
$\mathbf{M}$	Vector of Positivity scores
K	Number of folds in the Cross-Validation Ensemble
T	Number of different weights initializations (seeds)
ζ	Threshold tuning approach
τ	Classification threshold
τ^*	Optimal classification threshold
τ_ζ^*	ROC-tuned optimal threshold
$\hat{\Psi}$	Collection of $\hat{\mu}_{k,j}^f$
$\tilde{\Psi}_\tau$	Collection predicted Classes
Ψ	Collection ground truth labels

1 Introduction

1.1 Acute lymphoblastic leukaemia (ALL)

Acute lymphoblastic leukaemia (ALL) is characterized by an uncontrolled and aggressive proliferation of malignant immature cells (lymphoblasts) in bone marrow and blood [21].
ALL is classified based on the origin of malignant cells, which can be distinguished into B-cells (B-ALL) and T-cells (T-ALL) [24].
This study focuses on the detection of residual B-ALL cells during and after treatment.

1.1.1 Immunophenotyping and LAIP

Lymphoblasts originate from stem cells that differentiate into lymphocyte precursors, which can further develop into two different cell types: T-cells and B-cells. While T-cell precursors mature in the thymus, a specialized organ, B-cell precursors mature while remaining in the bone marrow [24]. After obtaining the final and correct receptors, i.e. upon complete maturation, the B-cells are released into the blood stream [24].

B-ALL arises from genetic alterations (gene or chromosomal mutation) occurring on B-cell progenitors. Typically, healthy B-cells follow a sequential and predictable maturation process [5]. It is characterized by specific marker expression patterns that occur in the bone marrow [24]. On the other hand, leukaemic cells exhibit an aberrant or asynchronous marker expression compared to normal B-cell precursors. This aberrant expression results in an early maturation arrest, and in the loss of control over the cell cycle. Indeed, while healthy B-cells follow the correct maturation pathway, mutated B-cell progenitors do not complete the maturation process and proliferate uncontrollably [5, 21], accumulating in peripheral blood and organs [17].

To identify these pathological cells, flow cytometry (Section 1.3) is often used as a technique to collect and analyse antigen expression profiles, in order to detect deviations from the normal maturation pathway.
The unique, aberrant combination of markers that defines the cancer cells is called Leukaemia-Associated Immunophenotype (LAIP) [21].

1.2 Measurable Residual Disease (MRD)

In the last decades, the proportion of patients with ALL who relapsed has decreased among those enrolled in Children's Oncology Group trials [30]. Between 1988 and 2002, 20.5% of these patients relapsed, whereas a newer analysis of trials from 1996–2014 registered a decrease to 12.7% [30]. Nowadays, MRD detection is still one of the best ways to monitor patients after treatment and to detect early relapse during the patient's recovery [21].

MRD is a quantitative measurement that expresses the quantity of blast cells present in a blood or bone marrow sample, during or after cancer treatment [21].
Routine inspections of MRD during the patient's treatment process may be crucial to make adjustments in therapy intensity [21] and mitigate treatment-related side effects.

The primary techniques for MRD detection are Molecular Methods and Immunophenotyping Methods. Both approaches have strengths and limitations:

Molecular Methods (MM): focus on detecting specific genetic anomalies and gene rearrangements. MM may achieve higher sensitivity, reaching 10^{-6}, but they are time-consuming

and require the identification of a specific molecular marker at diagnosis [5, 21].

Immunophenotyping Methods (IM): aim to characterize the expression of cellular markers. These methods are widely applicable, rapid, and generally less expensive than molecular techniques [5, 21]. Multiparameter flow cytometry (MFC) (Section 1.3), which is the technique employed in this study, belongs to the IM category.

In general, MRD has a prognostic value at the level of 0.01% [21]. Namely, at relevant time points, patients in whom an MRD percentage $\geq 0.01\%$ is detected using flow cytometry have a significantly higher risk for disease relapse compared to those with an MRD level $< 0.01\%$ [20, 21]. Due to its clinical relevance, the 0.01% MRD level has been included into the vector of unhealthy percentages u employed in the Artificial Sample generation procedure (see Section 3.4).

1.3 Multiparameter Flow Cytometry and Data Representation

To perform MRD detection and to reach the required sensitivity, computational methods have been employed to analyse a massive number of cells simultaneously [22] and detect rare events in a multidimensional space [8]. Indeed, cells are represented as numerical vectors collected via MFC [31]. MFC has the core function of "translating" biological properties of any individual cell into a numerical format, used then to analyse the cells sample numerically. Additionally, it is fast and it has a relatively low cost per sample analysed [34].

To collect and measure cellular parameters, a stream of cells is passed through a laser beam and analysed using a system of photonic detectors [34]. The properties measured include the fluorescence intensity of fluorochrome-labeled antibodies bound to antigens present on the surface or within the cytoplasm, but also physical characteristics such as size of the cells and granularity [34].

Nowadays, although cytometry technologies can theoretically identify up to 100 features simultaneously [4], an MFC panel for B-ALL MRD detection in a clinical setting typically measures 8 to 9 features to limit technical complexity and costs [8, 22].
The result of MFC analysis is a representation of each cell as a high-dimensional feature vector $x \in \mathbb{R}^{\mathcal{M}}$, where $\mathcal{M}$ is the number of feature expressed, and each feature of $\mathcal{M}$ corresponds to a specific marker (or physical characteristic) numerical value [31].

Consequently, the entire sample of blood or bone marrow cells is structured as a data matrix $D \in \mathbb{R}^{\mathcal{C} \times \mathcal{M}}$, or as a collection of high dimensional points, consisting of $\mathcal{C}$ feature vectors (cells representations):

$$f = \{x_i\}_{i=1}^{\mathcal{C}}$$

where $x_i \in \mathbb{R}^{\mathcal{M}}$ is the vector of $\mathcal{M}$ features measured for the i-th cell, and $\mathcal{C} \in \mathbb{N}$ is the total number of feature vectors in the sample (typically from 10^5 to 10^7) [31].

1.4 Raw MFC data preprocessing

Before being analysed, the raw MFC data require a preprocessing phase. Raw MFC data contain background noise, debris, measurement artifacts and/or flow instability events that are not relevant to B-ALL MRD detection. Therefore, a preprocessing phase is mandatory to ensure data quality and the reliability of subsequent analysis [25].
The preprocessing phase is typically structured as a sequential pipeline that includes the following steps [31, 29]:

Margins Removal: some recorded cells may fall at or outside of the physical boundary of the photonic detector or of the data acquisition range (marginals). These extreme values are usually called margin events, and such events do not represent accurate cellular measurements. Thus, they must be removed to ensure an accurate representation of the sample distribution [25].

Doublets removal: doublets occur when two or more cells pass through the laser beam nearly simultaneously [25]. Due to this proximity, the system captures them as a single event showing aberrant features. Removing these data points ensures that the future analyses are strictly based on single cells.

Compensation: compensation is a critical process that aims to correct the spectral overlap occurring during the MFC data acquisition [25]. It is caused by the use of multiple fluorochromes in the same sample. If not properly corrected, the spectral overlap introduces significant inaccuracies and biases in the data.

Transformation: Following compensation, data is transformed using an appropriate scaling function. While physical features values are linearly transformed, non-linear functions (such as Biexponential, Arcsine or Logicle [26]) are employed for fluorescence intensities from the proteins. This step is necessary to properly analyse data [25].

Quality control: In this final step, data is checked to find if there are technical anomalies [25]. These control and refinement steps can be performed manually, or using specialized software tools, such as PeacoQC [9] for the identification and removal of low-quality measurements.

1.5 Manual Gating

Manual Gating is the traditional method used to identify and quantify cell populations, performed by expert immunologists. The process consists of iteratively plotting cells in two-dimensional scatter plots, and choosing, at each step, which subset of cells to explore further [31].

In the case of B-ALL cells, the manual gating strategy typically follows a hierarchical filtering process. First, data receives a preprocessing phase (see Section 1.4), where technical artifacts are removed.
Subsequently, the analysis focuses on the biological identification:

1. Isolation of B-cell precursors: cells are gated based on B-cell lineage markers and granularity (density of intracellular components) [24], independently whether they show low or intermediate granularity [33]. This feature is measured via Side Scatter (SSC), a flow cytometry parameter that reflects light refracted by internal cellular components and granules [33]. However, if cells show an increased SSC compared to normal B-cell precursors, SSC can serve as an aberrant phenotypic marker to specifically identify ALL B-cells precursor population [33].
2. Identification of Leukaemic Population: the selected subset is visually checked for aberrant antigen expression compared to normal regenerating B-cells [33].

Although this approach is widely used, it has significant limitations. The gate's position is left to the operator's expertise, leading to subjective biases, which impacts standardization and reproducibility [8]. Furthermore, single analyses are restricted to bivariate projections of the data resulting in a significant loss of high-dimensional information and patterns [31, 29].

To solve these problems and drawbacks, automatic and reproducible techniques have been studied to achieve a more stable, standardized and efficient process, developing machine learning techniques specifically for MFC data, such as the CellCNN model [3], described in Section 1.8.

1.6 Computational Methods in MFC: Machine Learning Approaches

The primary limitations of manual gating are linked to reproducibility and bi-dimensional visualization. Machine Learning (ML) algorithms are well-suited to overcome these limitations, since they may operate directly with high-dimensional datasets $D \in \mathbb{R}^{\mathcal{C} \times \mathcal{M}}$.
Depending on the presence of labels assigned to the data, ML methods can be divided into two different categories:

- Unsupervised Learning: unsupervised learning methods aim to identify structures or patterns within the data without using any label or external information about them [15]. One of these methods is clustering, which attempts to detect the distinct groups of cells corresponding to the LAIP based on their density in the feature space (e.g. $\mathbb{R}^N$).
- Supervised and Semi-Supervised Learning: While Unsupervised Learning methods do not use any additional information about the data, both Supervised Learning (SL) and Semi-Supervised Learning (SSL) use labels to train models to recognize and classify cell patterns [15]. SL requires all training samples to be labeled. Instead, SSL uses a small set of labeled data combined with a larger set of unlabeled data. The necessary ground truth labels are derived from expert analysis using manual gating (Section 1.5). This is often expensive due to the large number of cells present in both blood and bone marrow samples.

This study focuses on the CellCNN architecture, which works in a SSL environment. Additionally, the modifications proposed in this study are performed using a fully supervised framework, since the dataset used shows labels at cells level (more details in Section 3.1).

1.7 Artificial Neural Networks: Fundamentals and Architecture

An Artificial Neural Network (ANN) represents a class of models inspired by the biological structure of the human brain. An ANN is composed of elementary computational units, called artificial neurons (or nodes), organized into layers and connected by edges. An edge is represented by a parameter known as a weight $w \in \mathbb{R}$ which expresses the intensity of the transmitted signal [12].
The typical architecture is the following:

- Input Layer: receives the data.
- Hidden Layers: intermediate layers where the transformation of features occurs. A network with multiple hidden layers is referred to as a Deep Neural Network (DNN).
- Output Layer: Produces the final model predictions.

Artificial neural networks can learn complex patterns from training data and generalise to unseen samples. Leveraging these properties, ANN have been investigated for MRD detection [3].
Formally, a neural network with L layers computes:

$$\hat{y} = \gamma^{(L)} \circ \gamma^{(L-1)} \circ \cdots \circ \gamma^{(1)}(X)$$

where each $\gamma^{(\ell)}(X) = \sigma(W^{(\ell)}X + b^{(\ell)})$ is a linear transformation followed by a non-linear activation function σ, where W a weights matrix, X is the input vector and b is the bias vector (see next sections for details).
There are various types of ANN architectures, but this work focuses on Feedforward Neural Networks (FNNs), and their sub-class Convolutional Neural Networks (CNNs), as they are the foundation of the model employed in this study, the CellCNN model [3].

1.7.1 Feedforward Neural Networks

FNNs represent the fundamental architecture of deep learning models. In this structure, information flows in one direction from the input nodes, through the intermediate computations in the hidden layers, to the output nodes, without forming cycles [12].

The key components in a FNN are:

- Loss Function: the Loss Function $\mathcal{L}$ (or Cost Function) measures the error between the network's predictions $\hat{y}$ and the ground truth targets y. For regression tasks, the Mean Square Error function is often employed, while the Cross-Entropy function is commonly used for classification problems.

  Multiclass Cross-Entropy Loss:

  $$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{n=1}^{N}\sum_{c=1}^{C} y_{n,c}\log(\hat{y}_{n,c})$$

  where N is the number of samples, C is the number of classes, $y_{n,c} \in \{0,1\}$ is 1 if sample n belongs to class c and 0 otherwise, and $\hat{y}_{n,c} = \mathrm{P}(y_n = c|x_n) \in [0,1]$ is the predicted probability of sample n of belonging to class c, with $\sum_{c=1}^{C}\hat{y}_{n,c} = 1$.

  In the specific case of Binary classification (where $C = 2$), the formula simplifies to the Binary Cross Entropy (BCE):

  $$\mathcal{L}_{BCE} = -\frac{1}{N}\sum_{n=1}^{N}\left[y_n\log(\hat{y}_n) + (1-y_n)\log(1-\hat{y}_n)\right]$$

- Weights: each connection between the neuron k in layer $L-1$ and a neuron j in the layer L is associated with a weight $w_{j,k} \in \mathbb{R}$. These weights determine the strength and the influence of each input signal.
- Biases: each neuron j not in the input layer is associated with a bias b_j. It enables neuron activation even when the input signal is weak.
- Activation function: at each neuron, a non-linear function $\sigma : \mathbb{R} \to \mathbb{R}$ is applied. The non-linearity allows the neural network to capture complex patterns that cannot be represented by a pure linear model. Common choices include [28]:
  1. Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$
  2. Tanh: $\sigma(z) = \frac{\sinh(z)}{\cosh(z)} = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
  3. ReLU: $\sigma(z) = \max(0, z)$

where z is the pre-activation signal of the neuron.
In a neural network, while number of neurons per layer and number of layers is fixed beforhand, weights and biases represent the trainable parameters of the model.

Mathematical Formulation of a Neuron. Every neuron j in layer L performs two computations sequentially: an affine transformation of all signals from the connected nodes and the bias, and a non-linear transformation.
Let:

- $w_{j,k}^{(L)}$ denote the weight connecting the k-th neuron of the layer $L-1$ to the j-th neuron of the layer L.
- $a_k^{(L-1)}$ denote the signal carried by the k-th neuron in the layer $L-1$.
- $b_j^{(L)}$ denote the bias term associated with the j-th neuron in the layer L.
- $\sigma : \mathbb{R} \to \mathbb{R}$ be a fixed non-linear activation function.

The pre-activation value $z_j^{(L)}$ and the resulting activation value $a_j^{(L)}$ for the j-th neuron in the layer L are defined as:

$$z_j^{(L)} = \sum_{k=1}^{N_{L-1}} w_{j,k}^{(L)} a_k^{(L-1)} + b_j^{(L)}$$

$$a_j^{(L)} = \sigma\left(z_j^{(L)}\right)$$

where N_{L-1} represents the number of neurons in the layer $L-1$.

Consequently, each activation in layer $L \in \{1, \ldots, L_{out}\}$ is a function of the entire vector of activations from the previous layer. This fully connected architecture results in a high-dimensional parameter space, which requires significant computational resources for optimization.

An example of FNN architecture and the schema of the computation of the signal of the j-th neuron in the L-th layer is shown inis shown in Figure 1.

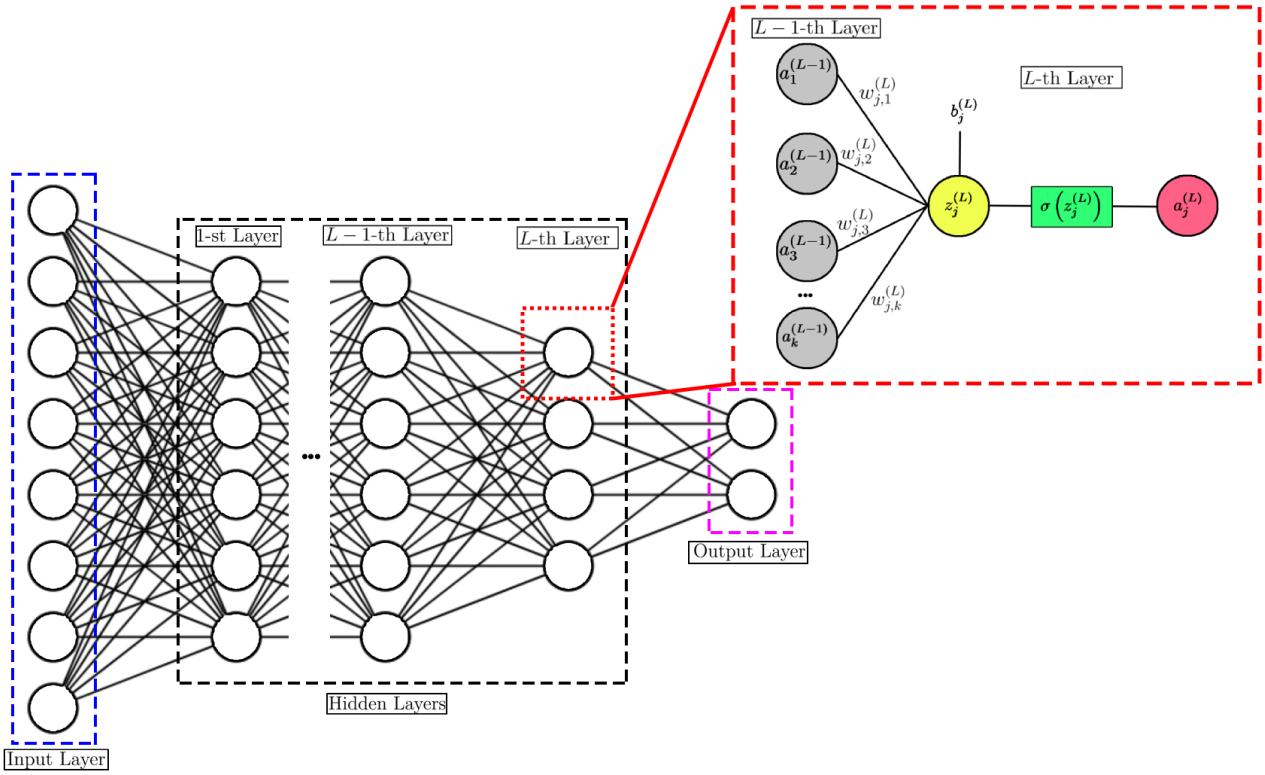


Figure 1: FNN Architecure composed by an Input layer, multiple Hidden layers and one Output layer L_{out} with two neurons. The schema of the computation of the signal of the j-th neuron in the L-th layer is shown in the red dashed square.

1.7.2 Backpropagation

In a machine learning context, the loss function $\mathcal{L}$ is typically a high-dimensional multivariate function for which a maximum or a minimum can not be analytically computed. For function maximization or minimization, numerical methods are then required. Backpropagation is the numerical algorithm used to compute the gradient of the loss function with respect to the network parameters (weights and biases).

Formally, it consists of a recursive application of the Chain Rule, propagating error signals from the output layer back toward the input layer.
The training process is divided into two distinct phases:

1. Forward Pass: the input propagates through the layers to generate a prediction $\hat{y}$ and evaluate the loss $\mathcal{L}$.
2. Backward Pass: the partial derivatives are computed via the chain rule to compute the gradient vector.

Consider the weight $w_{j,k}^{(L)}$ connecting the k-th neuron in layer $L-1$ to the j-th neuron in layer L. To compute the partial derivative of the loss L with respect to this weight, we apply the chain rule:

$$\frac{\partial \mathcal{L}}{\partial w_{j,k}^{(L)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(L)}} \cdot \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \cdot \frac{\partial z_j^{(L)}}{\partial w_{j,k}^{(L)}}$$

Analysing the three terms individually:

- $\frac{\partial \mathcal{L}}{\partial a_j^{(L)}} = \mathcal{L}'(a_j^{(L)})$ represents the sensitivity of the loss to changes in the activation of the j-th neuron.
- $\frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} = \sigma'(z_j^{(L)})$ is the derivative of the activation function evaluated at the pre-activation level.
- $\frac{\partial z_j^{(L)}}{\partial w_{j,k}^{(L)}} = a_k^{(L-1)}$ follows from the definition of affine transformation $z_j^{(L)} = \sum_{k=1}^{N_{L-1}} w_{j,k}^{(L)} a_k^{(L-1)} + b_j^{(L)}$.

Thus, the gradient for a specific weight $w_{j,k}^{(L)}$ is

$$\frac{\partial \mathcal{L}}{\partial w_{j,k}^{(L)}} = \mathcal{L}'(a_j^{(L)}) \cdot \sigma'(z_j^{(L)}) \cdot a_k^{(L-1)}$$

By computing these derivatives for all parameters in all layers, we obtain the full gradient vector $\nabla_W \mathcal{L}$ where W is the parameters vector.

Gradient Descent Optimization Once the gradient is computed, the parameter vector W is updated to minimize the loss. Given that $\mathcal{L}$ generally is a non-convex function in deep learning, iterative numerical algorithms are employed. The Gradient Descent (GD) algorithm updates parameters by moving in the direction of steepest descent (the negative gradient). Geometrically, the algorithms take steps on the loss surface in the parameter space to locate a local minimum.

Let W_t be the parameter vector at iteration t. The update rule is

$$W_{t+1} = W_t - \eta \nabla \mathcal{L}(W_t)$$

where $\eta \in \mathbb{R}^+$ is the learning rate. This hyperparameter determines the step size at each iteration. A value too small results in slow convergence and high computational costs, while a value too large may cause the updates to overshoot the minimum, leading to unstable training or divergence.

Stochastic Gradient Descent (SGD) Stochastic Gradient Descent (SGD), and its variant mini-batch SGD, is an optimization algorithm designed to address the computational complexity issues of classical Gradient Descent (GD). Indeed, classical GD is computationally expensive for large datasets because each update requires processing the entire training set $\mathcal{S}_{train}$, resulting in a complexity of $O(n)$, where n is the number of elements in the training set [12].
SGD addresses this by estimating the gradient using a single sample or a small subset (batch), reducing the per-update complexity to $O(m)$ where $m \ll n$ [12].

Let $B = \{x_1, \ldots, x_m\} \subset \mathcal{S}_{train}$ be a subset of m elements drawn randomly from the training set $\mathcal{S}_{train}$. Then the batch loss is defined as:

$$\mathcal{L}_{BCE}^B = -\frac{1}{m} \sum_{n=1}^{m} [y_n \log(\hat{y}_n) + (1 - y_n) \log(1 - \hat{y}_n)]$$

The weights W_t^B are then updated using $\mathcal{L}_{BCE}^B$'s gradient $g_t = \nabla_W \mathcal{L}_{BCE}^B(W_t^B)$:

$$W_{t+1}^B = W_t^B - \eta g_t$$

Algorithm 1 Stochastic Gradient Descent (SGD)

Require: Learning rate $\eta > 0$, initial parameter vector W_0, training set $\mathcal{S}_{train}$
for $t = 0, 1, 2, \ldots$ until convergence **do**
    Sample a batch $B \subseteq \mathcal{S}_{train}$ of size m
    Compute the batch loss: $\mathcal{L}^B(W_t)$
    Compute the gradient estimate: $g_t = \nabla_W \mathcal{L}^B(W_t)$
    Update parameters: $W_{t+1} = W_t - \eta g_t$
end for
return W_{t+1}

Mini-Batch SGD While SGD accelerates the computations, the stochastic nature of the gradient introduces variance in the minimization trajectory. Mini-batch SGD reduces SGD's drawbacks, and GD's computational cost by partitioning the dataset into batches of size m. An epoch is defined as one complete pass through the entire dataset $\mathcal{S}$, ensuring every sample has contributed to the parameter update exactly once [28].

Algorithm 2 Mini-Batch Stochastic Gradient Descent

Require: Training set $\mathcal{S}_{train}$, learning rate $\eta > 0$, batch size n_B, number of epochs E
Initialize parameter vector W_0^1
for epoch $e = 1, \ldots, E$ **do**
    Shuffle the training set $\mathcal{S}_{train}$
    Partition $\mathcal{S}_{train}$ into $\alpha = \lceil |\mathcal{S}_{train}|/n_B \rceil$ disjoint mini-batches $\{B_1, B_2, \ldots, B_\alpha\}$
    for $t = 1, \ldots, \alpha$ **do**
        $g_t^e = \nabla_W \mathcal{L}(W_t^e, B_t) = \frac{1}{n_B} \sum_{n \in B_t} \nabla_W \ell_n(W_t^e)$ ▷ where W_t^e are the updated weights
        ▷ at epoch e and at mini-batch upgrade t
        Update parameters: $W_{t+1}^e = W_t^e - \eta g_t^e$
    end for
    Set $W_0^{e+1} \leftarrow W_\alpha^e$
end for
return W_α^E

1.7.3 Optimization: Adaptive moment estimation (ADAM)

As discussed in Section 1.7.2, SGD introduces computational efficiency but suffers from limitations due to its fixed learning rate η. A fixed step size in a high-dimensional loss surface risks overshooting or stagnating in local minima. While one could employ learning rate schedulers (decreasing η over epochs), ADAM (Adaptive Moment Estimation) provides a more robust solution by computing individual adaptive learning rates for different parameters. It integrates the benefits of two other methods: Momentum, which tracks the first moment of the gradients, and Root Mean Square Propagation (RMSProp), which tracks the second moment [18].

Momentum and First Moment Estimation To stabilize the optimization trajectory, ADAM incorporates the first moment m_t of the gradients. This approach replaces the gradient of $\mathcal{L}$ with an exponential weighted moving average of all previous gradients. This mechanism reduces the stochastic noise produced by the estimate of the loss function over a random batch of sample, accelerating the convergence [18].

Let $\{B_t\}_{t \geq 1}$ be the sequences of batches of size n_B, and let g_t be the average gradient of the

loss function evaluated over the batch B_t:

$$g_t = \nabla_W \mathcal{L}(W_t, B_t) = \frac{1}{n_B} \sum_{n \in B_t} \nabla_W \ell_n(W_t)$$

where:

- $\nabla_W \mathcal{L}(W_t, B_t)$ denotes the vector of partial derivatives of $\mathcal{L}$ with respect to the vector of parameters $W \in \mathbb{R}^d$ evaluated at iteration t.
- $\ell_n(W_t)$ denotes the realization of the loss function for the n-th sample of the batch.

Then the first moment estimate m_t is defined recursively by the weighted moving average of the gradient g_t and the moment estimate at iteration $t-1$:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

where $\beta_1 \in [0, 1)$ is the exponential decay rate that controls the contribution of past gradients, and $m_0 = 0$.
Recursively expanding all moments obtained at previous iterations within the m_t equation:

$$\begin{aligned}
m_t &= \beta_1^2 m_{t-2} + \beta_1(1-\beta_1) g_{t-1} + (1-\beta_1) g_t \\
&= \beta_1^2 m_{t-2} + (1-\beta_1)(\beta_1 g_{t-1} + g_t) \\
&\dots \\
&= \beta_1^t m_0 + (1-\beta_1)(\beta_1^{t-1} g_1 + \dots + \beta_1^2 g_{t-2} + \beta_1 g_{t-1} + g_t) \\
&= \beta_1^t m_0 + (1-\beta_1) \sum_{i=1}^{t} \beta_1^{t-i} g_i \\
&\overset{m_0=0}{=} (1-\beta_1) \sum_{i=1}^{t} \beta_1^{t-i} g_i
\end{aligned}$$

Second Moment Another element required by ADAM is the second moment of the gradients. It is expressed as an exponential weighted moving average of the square of the gradient $g_t \odot g_t$ (square computed element-wise). For notational simplicity, we write $(g_t)^2$ to denote $g_t \odot g_t$. While the first moving average is the estimate of the gradient mean, the second moving average is the estimate of the uncentred variance of the gradient, and its updates are controlled by an exponential decay rate $\beta_2 \in [0, 1)$. Then, the update rule of the estimate of the second moment is

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)(g_t)^2$$

As for first moment, the update rule can be rewritten as a weighted sum:

$$\begin{aligned}
v_t &= \beta_2 v_{t-1} + (1-\beta_2)(g_t)^2 \\
&\dots \\
&\overset{v_0=0}{=} (1-\beta_2) \sum_{i=1}^{t} \beta_2^{t-i} g_i^2
\end{aligned}$$

Bias Correction Since $m_0 = 0$ and $v_0 = 0$, the estimates m_t and v_t are biased toward zero, especially during early iterations. Assume the gradient process is stationary ($\mathbb{E}[g_i] \approx \mathbb{E}[g_t]$ for $i \leq t$).
From

$$m_t = (1 - \beta_1) \sum_{i=1}^{t} \beta_1^{t-i} g_i$$

we take the expectation on left-hand and right-hand sides

$$\begin{aligned}
\mathbb{E}[m_t] &= \mathbb{E}\left[(1 - \beta_1) \sum_{i=1}^{t} \beta_1^{t-i} g_i\right] \\
&= (1 - \beta_1) \sum_{i=1}^{t} \beta_1^{t-i} \mathbb{E}[g_i] \\
&\overset{stationarity}{=} \mathbb{E}[g_t] (1 - \beta_1) \sum_{i=1}^{t} \beta_1^{t-i} \\
&= \mathbb{E}[g_t] (1 - \beta_1) \frac{(1 - \beta_1^t)}{(1 - \beta_1)} \\
&= \mathbb{E}[g_t] (1 - \beta_1^t)
\end{aligned}$$

where $\sum_{i=1}^{t} \beta_1^{t-i} = \frac{(1-\beta_1^t)}{(1-\beta_1)}$ is a finite geometric series, using $m_0 = 0$.
Finally, rearranging the terms:

$$m_t = \mathbb{E}[g_t] (1 - \beta_1^t) \iff \mathbb{E}[g_t] = \frac{m_t}{1 - \beta_1^t}$$

At each iteration, compute m_t and apply the bias correction approximating $\mathbb{E}[g_t]$ over a small batch:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} = \frac{1}{1 - \beta_1^t} (\beta_1 m_{t-1} + (1 - \beta_1) g_t)$$

Applying the same procedure using the second moment, the bias-corrected second moment approximation is

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} = \frac{\beta_2 v_{t-1} + (1 - \beta_2) (g_t)^2}{1 - \beta_2^t}$$

ADAM Update Rule The ADAM's weights W_t updating rule employs both the bias corrected first and second moments, performing a normalization of the gradient computed at iteration t accounting for past gradient expectations and fluctuations. The updating rule is then:

$$W_{t+1} = W_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

where the operations are applied element-wise, and $\epsilon > 0$ (typically 10^{-8}) ensures numerical stability [18].

Algorithm 3 ADAM Optimization

Require: Learning rate η, decay rates $\beta_1, \beta_2 \in [0, 1)$, $\epsilon > 0$, initial parameters W_0, training set $\mathcal{S}_{test}$

Initialize $m_0 \leftarrow 0$, $v_0 \leftarrow 0$, $t \leftarrow 0$
for each mini-batch $B_t \subseteq \mathcal{S}_{test}$ **do**
    $g_t \leftarrow \nabla_W \mathcal{L}(W_t, B_t)$
    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$
    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) (g_t)^2$
    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$
    $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$
    $W_{t+1} \leftarrow W_t - \eta \, \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$
    $t \leftarrow t + 1$
end for
return W_t

1.7.4 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a subclass of FNNs designed to process grid-structured data, such as vectors or matrices. These networks perform a linear operation, known as convolution, within hidden layers, called convolutional layers.
Different from fully connected layers, convolutional layers process the nodes locally, elaborating the input and its neighbours using a fixed set of weights, called kernel or filter. The number of neighbours employed at each iteration is called kernel size. The kernel slides over the inputs combining them into a weighted sum which represents the signal passed to the next layer. Depending on the kernel size, this mechanism reduces the number of parameters compared to a classical FNNs [28].

Let $v(x)$ and $g(z)$ be two real-valued continuous functions. The discrete convolution between v and g is defined as

$$h(x) = (v * g)(x) = \sum_{z=-\infty}^{\infty} v(z) g(x - z)$$

In the context of CNNs, the function $g(\cdot)$ represents the kernel. By learning the kernel weights, the model is able to detect specific patterns at various levels of abstraction.
In practical applications, such as Deep Learning, the input data is defined on a finite domain. Consequently, the summation is restricted to the inputs considered by the kernel range.

1D Convolution Example Consider a one-dimensional input where the input vector x is linearly transformed into a pre-activation vector $\mathbf{z}$ using a kernel of size κ (typically odd). Each element z_i is then the linear combination of the nodes centered at i. Let $\gamma = \lfloor \frac{\kappa}{2} \rfloor$ denote the half-width of the kernel. The convolution operation is given by:

$$z_i = \sum_{k=1}^{\kappa} w_k x_{i-\gamma+(k-1)}$$

where w_k denotes the $k - th$ weight of the filter.
Applying the activation function on each element z_i of the vector $\mathbf{z}$, we obtain a feature map, or channel.

$$a_i = \sigma(z_i) = \sigma \left(\sum_{k=1}^{\kappa} w_k \; x_{i-\gamma+(k-1)} \right)$$

Standard convolutional layers result in a reduction of dimensionality.

For example, given a finite sequence of inputs $\mathbf{x} = (x_1, \ldots, x_n)$ where $n = 5$, and a finite sequence of weights $\mathbf{w} = (w_1, \ldots, w_\kappa)$, where $\kappa = 3$ is the kernel size, then:

$$z_1 = w_1 x_1 + w_2 x_2 + w_3 x_3$$
$$z_2 = w_1 x_2 + w_2 x_3 + w_3 x_4$$
$$z_3 = w_1 x_3 + w_2 x_4 + w_3 x_5$$

This operation reduces the input for the next layer from 5 to 3.
In the context of this study, the kernel size employed in CellCNN implementation is set to the dimension of the feature expression of a cell, producing one single pre-activation value per single cell processed. Hence $\kappa = \mathcal{M}$.

Multichannel Convolution Due to the linear combination of inputs, convolution compresses information. To detect different patterns or features, CNNs employs multiple kernels in parallel (different kernel weights), generating multiple feature maps of the same size, stored in a hidden layer. These feature maps are used to compute the final output.

Consider an input layer X. By employing multiple distinct filters $W^{(\Phi)}$ where $\Phi \in \{1, \ldots, \mathcal{F}\}$ of size κ, the feature maps $Z^{(1)}, \ldots Z^{(\mathcal{F})}$ are computed in the following way:

$$Z^{(\Phi)} = X * W^{(\Phi)}$$

Pooling Layer Typically, convolutional layers are followed by pooling layers. These are specialized layers in which the activated output from convolution is replaced with a statistic of the neighboring outputs [12]. There are multiple pooling methods, but the following two are the commonly employed:

- Max pooling: max pooling replaces the output at a certain location with the maximum value observed within a neighborhood. This highlights the most prominent features in that region.
- Average Pooling (or Mean Pooling): this method replaces the output with the arithmetic mean of the values in the neighborhood. This provides a smoothing effect on the output values.

Pooling layers are performed for two primary reasons:

1. Dimensionality Reduction: by reducing the dimension of the feature maps, pooling reduces the computational cost and the number of parameters in subsequent layers.
2. Translation Invariance: pooling makes the representation approximately invariant to small translations of the input. If the input shifts slightly, the values of most pooled outputs do not change, allowing the network to detect features regardless of their exact position [12].

1.8 CellCNN

CellCNN is a specialized neural network architecture designed for the analysis of high-dimensional single-cell data [3]. While standard CNNs process spatially structured inputs such as image patches [12], CellCNN instead treats the input as an unordered multiset of single-cell measurements, where each cell is represented by a vector of feature expressions [3].

The architecture follows a Multiple Instance Learning (MIL) paradigm [3]: each input consists of a multi-cell set $f = \{x_1, \ldots, x_{n_f}\}$, where $x_i \in \mathbb{R}^{\mathcal{M}}$ is the expression vector of the i-th cell across $\mathcal{M}$ features. A single phenotype label y_f is associated with the entire set f, rather than with individual cells, reflecting the clinical setting in which a diagnosis is assigned at the patient level rather than at individual cells level.
The architecture is shown in Figure 2.

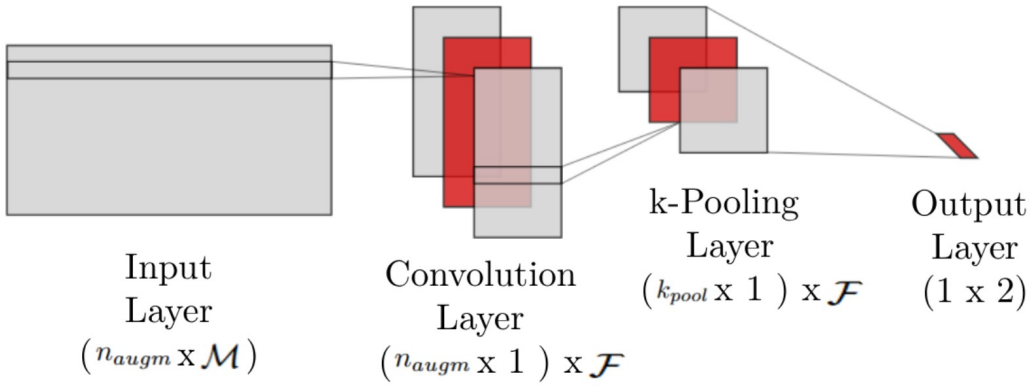


Figure 2: CellCNN Architecture after internal data augmentation. Each input of n_{augm} cells, expressed by $\mathcal{M}$ features, is processed through $\mathcal{F}$ filters, and the k_{pool} highest filter responses are retained. The resulting representation is aggregated into a two-dimensional output vector containing the predicted probabilities of belonging to the healthy and unhealthy classes.

1.8.1 Convolutional Layer as a Feature Detector

In standard CNNs, convolutional filters are applied to spatially ordered inputs such as images [12]. In CellCNN, this operation is adapted to process unordered multi-cell inputs, where individual cell measurements take the role of image patches [3]. Since cells within a sample lack any spatial ordering, the convolution reduces to a linear transformation applied independently and identically to each cell vector.
Formally, let $x_i \in \mathbb{R}^{\mathcal{M}}$ be the expression vector of the i-th cell, where $\mathcal{M}$ denotes the number of measured features. A filter $\Phi \in \mathbb{R}^{\mathcal{M}}$ computes a scalar cell-filter response r_i via the inner product:

$$r_{i,\Phi} = \langle x_i, \Phi \rangle = \sum_{j=1}^{\mathcal{M}} x_{i,j}\, \Phi_j$$

where $x_{i,j}$ is the expression level of the j-th feature in the i-th cell, and Φ_j is the j-th component of the filter vector Φ.
When $\mathcal{F}$ distinct filters $\{\Phi^{(1)}, \ldots, \Phi^{(\mathcal{F})}\}$ are applied to a sample of C cells, the output is a response matrix $R \in \mathbb{R}^{C \times \mathcal{F}}$, where entry $r_{i,\Phi}$ represents the response of the filter Φ to the i-th cell.

1.8.2 Pooling and Global Representation

To achieve permutation invariance, CellCNN employs a global pooling operation applied for each filter $\{\Phi^{(1)}, \ldots, \Phi^{(\mathcal{F})}\}$. Two pooling functions where originally proposed: max-pooling computes the maximum response over all cells for a given filter, measuring the presence of cell subsets that strongly activate that filter; mean-pooling computes the average response and serves as an approximation of the frequency of the analysed subsets.
The resulting pooling matrix $R' \in \mathbb{R}^{\mathcal{F}}$ serves as the sample-level representation and is passed to a fully connected output layer.
For classification problems, it contains one node per class, followed by a softmax activation [3]

$$\hat{y} = \sigma(W_{out}^T R' + b)$$

where $W_{out} \in \mathbb{R}^{\mathcal{F}}$ is the weight vector, $b \in \mathbb{R}$ is the bias term. The network output predicts the sample-associated phenotype, such as disease status.

Training and Model Selection The training procedure incorporates data augmentation, gradient-based optimization, and hyperparameter search:

1. Data Augmentation: since the number of biological samples is often limited, the training set $\mathcal{S}_{train}$ is augmented by drawing n_{subs} random subsets $s \subset \mathcal{S}_{train}$ with replacement, each of size n_{cells}, and each associated to the phenotype label of its parent sample.

2. Optimization: model weights are randomly initialized from a uniform distribution and optimized using mini-batch stochastic gradient descent with ADAM updates [3, 18]. The objective function is the categorical cross-entropy for classification tasks. Training runs for a maximum of 100 epochs, with early stopping if more than 5 epochs do not show improvements in validation loss, and L2 weight decay of 10^{-4}.

3. Hyperparameter Search and Model Selection: a random search is performed over the key hyperparameters (number of filters $\mathcal{F}$, learning rate η, and max-pooling drop out %) [3]. The models achieving the highest predictive accuracy on the validation set are selected for test set $\mathcal{S}_{test}$ classification [3].

1.8.3 Sample Classification

Each sample f is classified by drawing a single subset $s \subset f$ of fixed size n_s and passing it through each of the top three models selected during training. The final prediction is obtained by averaging the three individual outputs:

$$\hat{y}_f = \frac{1}{3} \sum_{i=1}^{3} \hat{y}_{f,i}$$

and the sample label is assigned according to the fixed threshold $\tau = 0.5$ through the following classification function

$$\tilde{y}_f = \begin{cases} 1 & \text{if } \hat{y}_f \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

This procedure presents several limitations. Using validation accuracy as metric for model selection is problematic in rare event classification. When the percentage of unhealthy cells is very low, a model that ignores the minority class can still achieve high accuracy, potentially leading to the selection of poorly discriminative models.

Moreover, drawing a single random subset for sample classification introduces stochasticity in the final prediction. Indeed, different subsets may yield different outputs, particularly when unhealthy cells are rare, making the classification potentially unstable across repeated runs. Finally, when a test set $\mathcal{S}_{test}$ comprises multiple samples, the subset size n_s is fixed to the size of the smallest sample and applied uniformly to all remaining samples, potentially introducing a loss of information for larger samples that remain undersampled.
These limitations have motivated the methodological extensions proposed in this work, as described in Section 3.
Note that depending on the dataset used, the CellCNN paper proposes different implementations [3]. Since the paper does not describe all implementation details, the default parameters and the baseline procedures adopted in this work are derived from the default implementation available in the official GitHub repository and from the repository code [2].

2 Objectives

The main technical challenge in MRD detection is to distinguish the rare LAIP-positive malignant cells (termed unhealthy cells for the rest of the study) from the numerous healthy B-cell precursors and other non-malignant populations found in the blood or bone marrow sample.

From a computational perspective, this task represents a rare-event classification problem characterized by extreme class imbalance. The difficulty of the task increases as the percentage of residual unhealthy cells decreases (e.g., down to 10^{-4}). Consequently, improving robust machine learning methods that are able to handle such imbalance become of crucial importance.

In this context, the main objective of this study is to improve the CellCNN model by proposing a more robust Prediction Approach that involves the aggregation of predictions from several subsets of the test set, a data augmentation strategy, called Artificial Samples generation, and two Training Procedures. These contributions are developed across four Experiments designed to address the following three research questions:

- Does CellCNN benefit from Artificial Samples generation?
- Does an ensemble approach achieve better results in terms of both predictive performance and robustness compared to single-fold and single-seed baselines?
- Do ROC-tuned or RES-tuned thresholds produce better classification performance compared to the fixed threshold $\tau = 0.5$?

3 Methodology

3.1 Data Introduction

The data analysed in this study is cell-level labelled B-ALL data from Ghent University Hospital in Belgium.
All analyses are performed on samples after the preprocessing phase explained in Section 1.4. The dataset contains 23 samples from 10 patients. From some patients, samples collected at different time points are included (primarily days 15, 29, and 71–78 following start of treatment, depending on availability).
Patients, number of samples, time points days and patients classes are summarised in the Table 2, while unhealthy cells distribution across patients' samples is shown in Figure 3.

Patient ID	**N. Samples**	**Time Points**	**Label**	**Samples Labels**
1	2	D15, D78	Mixed	1, 0
2	2	D15, D29	Unhealthy	1, 1
3	2	D15, D71	Mixed	1, 0
4	3	D15, D29, D71	Mixed	1, 1, 0
6	2	D15, D29	Unhealthy	1, 1
7	3	D15, D29, D71	Mixed	1, 1, 0
8	2	D15, D71	Unhealthy	1, 1
9	2	D15, D29	Unhealthy	1, 1
11	3	D15, D29, D71	Mixed	1, 1, 0
12	2	D15, D29	Unhealthy	1, 1

Table 2: Available Dataset. Labels are defined as follows: Unhealthy (1), Healthy (0). The notation "DX" indicates Day X.

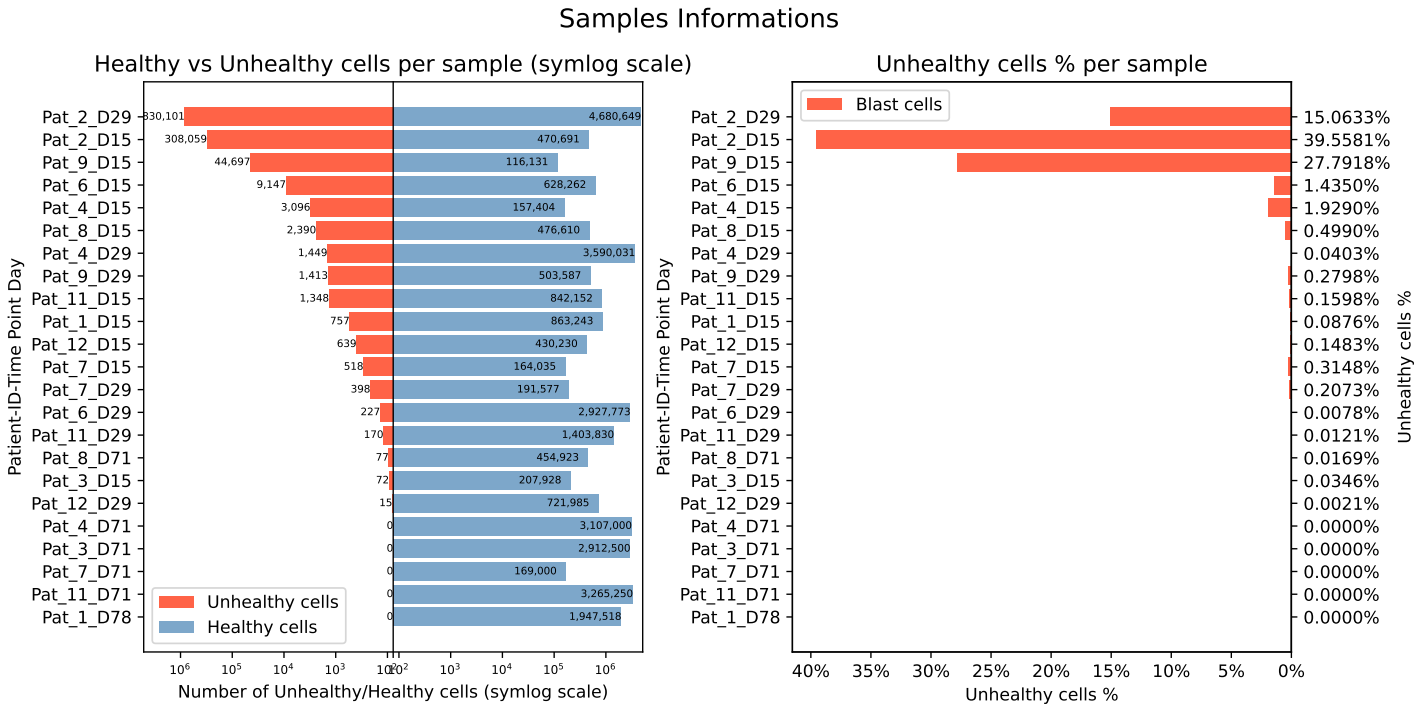


Figure 3: Healthy/Unhealthy cells distribution and Unhealthy cell % distribution. The notation "Pat_Y_DX" denotes the sample collected on Day X from Patient Y.

The highly unbalanced distribution of the number of healthy cells and unhealthy cells percentages across the samples, introduced difficulties in both model training and performance evaluation. In particular, 52% of the samples express a percentage of unhealthy cells lower than 0.1%, and more than 30% an unhealthy cell level lower than 0.01%, with the minimum achieved by the sample Pat_12_D29 showing an unhealthy percentage of 0.0021%.
This highly skewed distribution represents the challenge addressed by the proposed methodology and motivated the Artificial Samples generation. Indeed, the model must be able to detect MRD-positive samples even when unhealthy cells are a small fraction of the total cell population.

3.2 Cells Expression Features

Definition: Cell. Let $m_1, \ldots, m_{\mathcal{M}}$ be the values of $\mathcal{M}$ features, where $\mathcal{M} \in \mathbb{N}$ is the number of features measured, and $m_j \in \mathbb{R}$. Then, a cell x can be represented as

$$x = (m_1, \ldots, m_{\mathcal{M}})$$

In this study, the cells are represented by 12 features (i.e. $\mathcal{M} = 12$) and 2 additional attributes, which are a cell-level ID, valid within the same sample, and a cell-level binary label, which marks the cells as healthy (0) or unhealthy (1). These features are listed in the Table 3:

Markers (Proteins)			**Physical Features**	**Additional Info**
CD81 (FITC-A)	CD73+CD304 (PE-A)	CD34 (PerCP-Cy5-5-A)	FSC-A	Original_ID
CD19 (PE-Cy7-A)	CD10 (APC-A)	CD38 (APC-C750-A)	FSC-H	IsBlast
CD20 (PB-A)	CD45(OC515-A)		SSC-A	

Table 3: Cell features used in the dataset. Non-marker columns are excluded from model input.

3.3 Patient-Level Train/Validation/Test Splits

To avoid data leakage, all samples from a given patient are assigned together to either the training, validation, or test set, applying patient-level splits.
Depending on sample cellular composition, they are categorized as:

- Healthy sample: contains only Healthy cells.
- Unhealthy sample: contains Unhealthy and Healthy cells.

To support the proposed data augmentation strategy, patients are further categorized into three groups, based on the category of their samples (see Section 3.1 for details on the samples used in this study):

- Healthy patients: patients who provided only Healthy samples.
- Unhealthy patients: patients who provided only Unhealthy samples.
- Mixed patients: patients who provided at least one Healthy and one Unhealthy sample.

3.4 Data Augmentation: Artificial Samples Generation

This data augmentation strategy was developed to address two challenges introduced by patient-level splitting: the limited number of available training samples and the high variability in the percentage of unhealthy cells across samples. The procedure is defined at the patient level, but it can be adapted to operate at the sample level by generating Artificial Samples (AS) at sample level. It is important to notice that AS generated from a given patient must be grouped and moved all together, as if they are original patient's samples.
For both training and validation sets, AS of size q are generated and used in place of the original full samples during model training and validation.
Note that the model does not train directly on samples in the train set $\mathcal{S}_{train}$. Indeed, CellCNN internally draws n_{subs} subsets of size n_{cells} from each sample in the training set, regardless of whether samples are AS or the original full patient samples. Consequently, q controls the pool of available cells for subsampling, indirectly influencing the training of the model. By fixing the percentage of unhealthy cells in each AS to clinically relevant MRD levels, this approach reduces the variability of the training set and provides controlled cells representations of unhealthy samples.
The AS generation procedure is structured into the following steps:

- Parameter Definition
- Patient-Level Cell Splitting
- Artificial Samples Generation

3.4.1 Parameter Definition

Three parameters are required:

- Unhealthy cells percentages (u): a vector containing the target percentages of unhealthy cells to be represented in the generated AS, typically down to 0.01%.
- Number of unhealthy samples (n_{as}): the number of unhealthy AS generated per patient. For an Unhealthy or Mixed patient, an equal number of healthy AS is also generated, resulting in a total of $2 \cdot n_{as}$ AS per patient.

  The value of n_{as} is directly determind by the length of u, since we generally build one unhealthy AS per proposed percentage. It follows that $n_{as} = |u|$.
- Artificial Samples Size (q): the number of cells in each generated AS, applied uniformly to both healthy and unhealthy AS.

3.4.2 Patient-Level Cell Splitting

Each patient p is processed independently. From each sample f from the patient p, cells are partitioned into two disjoint sets according to their cell-level labels. The resulting cells are then concatenated across all samples from the same patient, separately for each cell type.

Definition: Cell Vector by Type Let p be a patient and let $\mathcal{K} \in \{\text{healthy}, \text{unhealthy}\}$ denote a cell type. Let $f_1, \ldots, f_\xi$ be the samples provided by patient p, where ξ is the total number of samples. For each sample f_i, let $n_{f_i}^{(\mathcal{K})}$ denote the number of cells of type $\mathcal{K}$, and let $x_{f_i}^{j(\mathcal{K})}$ denote the j-th cell of type $\mathcal{K}$ in sample f_i. The cell vector of type $\mathcal{K}$ for patient p is defined as:

$$C_{\mathcal{K}}^{p} = \left[x_{f_1}^{1(\mathcal{K})}, \ldots, x_{f_1}^{n_{f_1}^{(\mathcal{K})}}, \ldots, x_{f_\xi}^{1(\mathcal{K})}, \ldots, x_{f_\xi}^{n_{f_\xi}^{(\mathcal{K})}}\right].$$

For healthy patients, only $C_{healthy}^{p}$ is built. For mixed or unhealthy patients, both $C_{healthy}^{p}$ and $C_{unhealthy}^{p}$ are constructed.

3.4.3 Construction of Artificial Samples (AS)

The objective is to build n_{as} unhealthy AS and n_{as} healthy AS, each of fixed size q, such that each unhealthy AS reflects one of the target unhealthy cell percentages defined in u. This iterative construction ensures that a larger fraction of the available cell pool contributes to the generated AS (with respect to the random sampling), increasing cell utilization and reducing redundant resampling, particularly relevant for the low-frequency unhealthy cell populations.
Indeed, when $|C^p| \geq n_{as} \cdot q$, the iterative construction guarantees that exactly $n_{as} \cdot q$ distinct cells contribute to the generated AS. In contrast, random sampling with replacement draws the same total number of cells but selects each one independently. When $|C^p| \leq n_{as} \cdot q$, the iterative construction ensures that all cells in the pool are selected at least once, with only the remaining number of cells are selected by sampling with replacement.

To quantify the redundancy of random sampling, consider a pool of k cells. The probability that a given cell x_j is not selected at a single draw is $\left(1 - \frac{1}{k}\right)$. Under sampling with replacement over k independent draws, the probability that x_j is never selected is:

$$\mathrm{P}\left(x_j \text{ never selected}\right) = \left(1 - \frac{1}{k}\right) \times \cdots \times \left(1 - \frac{1}{k}\right) = \left(1 - \frac{1}{k}\right)^k$$

As $k \to \infty$, this converges to:

$$\lim_{k \to \infty} \left(1 - \frac{1}{k}\right)^k = e^{-1} \approx 0.368$$

meaning that approximately 36.8% of cells would never be selected as $k \to \infty$. The proposed iterative construction therefore guarantees a probability of 1 of observing all cells in C^p when $|C^p| \leq n_{as} \cdot q$, and a greater or equal to probability of drawing a given number of unique cells from C^p when $|C^p| \geq n_{as} \cdot q$, compared to random sampling with replacement.

Back to the procedure, both unhealthy and healthy AS contain healthy cells. Consequently, both generation phases require access to $C_{healthy}^{p}$.

3.4.4 Processing Unhealthy or Mixed Patients

The first step is to sort the target unhealthy cell percentages in ascending order. The absolute number of unhealthy cells per AS is obtained by multiplying each entry of the sorted vector u by the AS size q and the corresponding number of healthy cells is obtained by complement.

Formally, let $n_{as} \in \mathbb{N}$. Let $u = [u_1, u_2, \ldots, u_{n_{as}}]^\top \in \mathbb{R}^{n_{as}}$ where $u_i \in (0, 1) \quad \forall i$, sorted in ascending order. For each $i \in \{1, \ldots, n_{as}\}$, the number of unhealthy cells N_i^U and healthy cells N_i^H required for the i-th AS are:

$$N_i^U = \lceil q \cdot u_i \rceil \tag{1}$$

$$N_i^H = q - N_i^U \tag{2}$$

To generate the i-th AS, a dynamically sized subset is extracted from the remaining unhealthy cell pool. Let $C_{\text{rem},i}^U$ denote the sequence of unhealthy cells remaining at step i. The current subset size is determined by:

$$|subset_i| = \left\lfloor \frac{|C_{rem,i}^U|}{n_{as} - (i-1)} \right\rfloor$$

which distributes the available cells proportionally across the remaining AS to be generated. The sampling strategy for the i-th subset is then defined as follows:

1. If $|subset_i| < N_i^U$: all cells in $subset_i$ are selected, and the remaining $(N_i^U - |subset_i|)$ cells are sampled with replacement from $subset_i$.
2. If $|subset_i| \geq N_i^U$: exactly N_i^U cells are sampled from $subset_i$ without replacement.

Cells selected at least once are removed from the global pool $C_{unhealthy}^p$, while any unselected cells in the current subset are reintegrated into the pool for subsequent iterations.

The same procedure is then applied to the selection of healthy cells, with the subset size adjusted as follows:

$$|subset_i| = \left\lfloor \frac{|C_{rem,i}^H|}{2 \cdot n_{as} - (i-1)} \right\rfloor$$

The denominator $2 \cdot n_{as}$ reflects the fact that the healthy cell pool $C_{healthy}^p$ is shared between the generation of both unhealthy and healthy AS. As in the unhealthy case, the denominator decreases dynamically with each iteration to account for the remaining AS to be generated. Finally, the selected unhealthy and healthy cells are concatenated to obtain the i-th AS. This process is repeated for all n_{as} iterations until the set of n_{as} patient-specific unhealthy AS is completed. The complete procedure is summarised in the Algorithm 4.

Algorithm 4 Artificial Samples Generation for Unhealthy or Mixed Patients

Require: Patient cells vectors $C^p_{unhealthy}, C^p_{healthy}$, target proportions u, sample size q, number of AS n_{as}

for each unhealthy or mixed patient $p \in \mathcal{S}_{train} \cup \mathcal{S}_{val}$ **do**
  Phase 1: Generate Unhealthy AS
  for $k = 1$ **to** n_{as} **do**
    $N_k^U \leftarrow \lceil q \cdot u_k \rceil, \quad N_k^H \leftarrow q - N_k^U$

    // Sample unhealthy cells
    $m \leftarrow (n_{as} - k + 1)$
    $subset_size \leftarrow \lfloor |C^p_{unhealthy}|/m \rfloor$
    $B \leftarrow C^p_{unhealthy}[1 : subset_size]$
    $S_U \leftarrow Sample(B, N_k^U)$ ▷ w/ replacement if $|B| < N_k^U$, w/o otherwise
    $C^p_{unhealthy} \leftarrow C^p_{unhealthy} \setminus S_U$

    // Sample healthy cells
    $j \leftarrow 2n_{as} - k + 1$
    $subset_size \leftarrow \lfloor |C^p_{healthy}|/j \rfloor$
    $D \leftarrow C^p_{healthy}[1 : subset_size]$
    $S_H \leftarrow Sample(D,\ N_k^H)$
    ▷ w/ replacement if $|D| < N_k^H$, w/o otherwise
    $C^p_{healthy} \leftarrow C^p_{healthy} \setminus S_H$
    $AS_k \leftarrow S_U \cup S_H$
  end for

  Phase 2: Generate Healthy AS
  for $k = n_{as} + 1$ **to** $2n_{as}$ **do**
    $j \leftarrow 2n_{as} - k + 1$
    $subset_size \leftarrow \lfloor |C^p_{healthy}|/j \rfloor$
    $D \leftarrow C^p_{healthy}[1 : subset_size]$
    $S_H \leftarrow Sample(D,\ q)$ ▷ w/ replacement if $|D| < q$, w/o otherwise
    $C^p_{healthy} \leftarrow C^p_{healthy} \setminus S_H$
    $AS_k \leftarrow S_H$
  end for
end for

3.4.5 Processing Healthy Patients

Samples provided by Healthy patients contain only healthy cells. Consequently, the patient-specific cell vector $C^p_{healthy}$ is obtained by concatenating all cells across the patient's samples. Since healthy patients do not have unhealthy cells, the AS generation simplifies building only n_{as} healthy AS per patient.

Algorithm 5 Artificial Samples Generation for Healthy Patients

Require: Patient cells vector $C^p_{healthy}$, sample size q, number of AS n_{as}
for each healthy patient $p \in \mathcal{S}_{train} \cup \mathcal{S}_{val}$ **do**
  Generate Healthy AS
  for $k = 1$ **to** n_{as} **do**
    $j \leftarrow n_{as} - k + 1$
    $subset_size \leftarrow \lfloor |C^p_{healthy}|/j \rfloor$
    $D \leftarrow C^p_{healthy}[1 : subset_size]$
    $S_H \leftarrow Sample(D,\ q)$ ▷ w/ replacement if $|D| < q$, w/o otherwise
    $C^p_{healthy} \leftarrow C^p_{healthy} \setminus S_H$
    $AS_k \leftarrow S_H$
  end for
end for

3.5 Metrics

Metrics constitute the core framework to evaluate model performance. They are essential for standardizing comparison and validating results. In the context of Deep Learning, metrics play a crucial role not only in the final evaluation but also in monitoring model convergence and generalisation during the training phase.
Metrics can be categorized into two groups: descriptive statistics, used to aggregate results across multiple stochastic iterations (seeds or folds) and performance metrics, used for model evaluation.

3.5.1 Descriptive Statistics and Distribution measures

These metrics are used to describe the data distribution and to aggregate prediction and variability regarding model performance across multiple seeds, folds, or experiments.
The metrics employed are:

Mean: the expected value of a discrete set of observations [7]:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

Median: the middle value of a sorted distribution. It is a robust measure of central tendency [7]. Let $x = \{x_1, \ldots, x_n\}$ be a collection of values sorted in ascending order where $n \in \mathbb{N}$.
The median is defined as:

$$median = \begin{cases} \frac{x_{\frac{n}{2}} + x_{\frac{n}{2}+1}}{2} & \text{if n is even} \\ x_{\frac{n+1}{2}} & \text{if n is odd} \end{cases}$$

Sample Variance and Standard Deviation: they measure the dispersion of the data points from the mean of a discrete set of observations [7]:

$$\nu^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2, \quad \nu = \sqrt{\nu^2}$$

3.5.2 Metrics for Model Evaluation

These metrics quantify the predictive performance of a classification model and are derived from the Confusion Matrix (CM).
The CM is a table that summarises model predictions against ground truth labels. Although it can be used for multi-class problems, this study focuses on binary classification, where predicted labels belong to {Predicted Positive (PP), Predicted Negative (PN)} and ground truth labels belong to {Positive (P), Negative (N)}.

In the clinincal context of B-ALL MRD detection, the cost associated with Type II errors (incorrectly classifying an unhealthy sample as healthy) is significantly higher than the cost of Type I errors (incorrectly classifying a healthy sample as unhealthy).
The prediction outcomes can be distinguished in four categories:

- True Positive (TP): the model correctly predicts the positive class.
- True Negative (TN): the model correctly predicts the negative class.
- False Positive (FP): the model incorrectly predicts the positive class when the actual class is negative (Type I error).
- False Negative (FN): the model incorrectly predicts the negative class when the actual class is positive (Type II error).

These categories are summarised in the Table 4

		Predicted condition	
	Total population	Positive (PP)	Negative (PN)
Actual condition	Positive (P)	TP	FN
	Negative (N)	FP	TN

Table 4: Prediction Outcomes. TP, FN, FP, TN indicates True Positive, False Negative, False Positive and True Negative respectively.

Based on these elements, the following derived metrics are used to summarise the experimental outcomes:

Accuracy: the overall proportion of correct predictions [19, 25]. In the presence of class imbalance, it is a biased measure that tends to favor the majority class:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision: the proportion of correct positive predictions relative to all positive predictions [32, 25]:

$$Precision = \frac{TP}{TP + FP}$$

Recall: the ability of the model to correctly identify all positive instances [32, 25]. In the context of B-ALL MRD detection, high recall is critical to avoid missing residual leukaemic

cells:

$$Recall = \frac{TP}{TP + FN}$$

F_β-score: the weighted harmonic mean of Precision and Recall [19], parameterized by $\beta \geq 0$, which controls the relative weight of the two metrics:

$$F_\beta - score = (1 + \beta^2)\frac{Precision \times Recall}{\beta^2 Precision + Recall} = \frac{(1 + \beta^2)TP}{(1 + \beta^2)TP + FP + \beta^2 FN}$$

When $\beta = 1$, equal weight is given to Precision and Recall; when $\beta > 1$, Recall is weighted more heavily. Unless stated otherwise, $\beta = 1$ is used throughout this study, yielding the F_1-score.

ROC Curve: while metrics such as Precision and Recall depend on a fixed classification threshold, the Receiver Operating Characteristic (ROC) [32, 25] curve evaluates binary classifier performance across all possible thresholds by plotting the True Positive Rate (TPR) against the False Positive Rate (FPR):

$$TPR = \frac{TP}{TP + FN} = Recall, \quad FPR = \frac{FP}{FP + TN}$$

Area Under the Curve (AUC): a threshold-independent summary of classifier performance [32, 25]. It represents the probability that the model assigns a higher score to a randomly chosen positive sample than to a randomly chosen negative one. An AUC of 1.0 indicates a perfect classifier, while an AUC of 0.5 corresponds to random guessing.

3.6 CellCNN Classification approach

The classification approach originally proposed in the CellCNN framework [3] predicts the class label of a sample f based on a single subset s of fixed size n_s. If no size is specified, n_s is automatically set to the minimum cardinality observed across all samples in the test set. Given the test set $\mathcal{S}_{test}$ the subset size is set to the minimum sample cardinality observed:

$$n_s = \min_{f \in \mathcal{S}_{test}} |f|.$$

Let $f \in \mathcal{S}_{test}$ be a sample in the test set with ground truth label y_f, and let $s \subset f$ be a randomly drawn subset of fixed size n_s. The model processes s and outputs $\hat{y}_f \in [0, 1]$, representing the probability of f belonging to the unhealthy class. The sample is then classified using the following indicator function:

$$\tilde{y}_f = \mathbb{1}_{[\tau,\,1]}(\hat{y}_f) = \begin{cases} 1 & \text{if } \hat{y}_f \geq \tau \\ 0 & \text{otherwise} \end{cases}$$

where $\tau = 0.5$ is the default classification threshold.

While computationally efficient, this approach presents potential limitations in the context of MRD detection. Although for sufficiently large samples the single-subset strategy may capture the true cellular distribution, the approach relies on a single stochastic draw, which introduces variability in the final prediction across repeated runs. To address these limitation and improve the robustness and reproducibility of the classification, we propose a multiple-sampling aggregation strategy, described in the following section.

3.7 Sample-Level Classification Approach

Instead of processing a single subset, we generate multiple independent subsets from each sample $f \in \mathcal{S}_{test}$. Each subset is individually classified, and the final sample label is determined by aggregating predictions across subsets and applying a tuned threshold τ_ζ^*, where ζ represents the tuning strategy:

- $\zeta = RES$: threshold tuned using the F_1-score, to ensure a robust balance between precision and recall.
- $\zeta = ROC$: threshold tuned using the ROC curve, ensuring more weight on Recall.

This classification approach consists of four steps:

- Dataset Resampling: for each sample $f \in \mathcal{S}_{test}$, generate n_{res} independently drawn random subsets of a fixed size n_s.
- Subsets Predictions: predict each generated subset using the trained model(s).
- Positivity Score: for each sample f, aggregate the subset-level predictions into a single score $M_f \in [0, 1]$, referred to as the Positivity Score.
- Classification: classify each sample f based on M_f and the tuned threshold τ_ζ^*.

3.7.1 Dataset Resampling

Let $f \in \mathcal{S}_{test}$ be a sample in the test set.
From f, we generate n_{res} subsets $\{s_1, \ldots, s_{n_{res}}\}$, each of fixed size n_s, drawn independently and uniformly at random from the cells of f with replacement. Consequently, different subsets may share individual cells, while each subset independently captures a random subset of the sample's cellular distribution.
The ground truth label of each subset s_j drawn from sample f is defined as:

$$y_j^f = \begin{cases} 1 & \text{if } s_j \text{ contains at least one unhealthy cell} \\ 0 & \text{otherwise} \end{cases}$$

3.7.2 Subsets Predictions

During Training Procedure 2 (Section 3.8.3), a K-Fold cross-validation (CV) is performed. In each fold $k \in \{1, \ldots, K\}$, the model is trained and validated on different patient combinations. Within each fold, T models are trained using different random weight initializations (seeds), generating a total of $K \times T$ trained models.
For each subset s_j drawn from sample f, each of the $K \times T$ models produces a prediction consisting of the probability of belonging to the unhealthy class. Let $\hat{y}_{k,t,j}^f \in [0, 1]$ denote the predicted probability of belonging to the unhealthy class of the j-th subset (s_j) , produced by the model trained in fold k with seed t. The full collection of predictions for sample f is organized into a tensor $\hat{\mathbf{Y}}^f \in [0, 1]^{K \times T \times n_{res}}$, where element $\hat{\mathbf{Y}}_{k,t,j}^f = \hat{y}_{k,t,j}^f$.

3.7.3 Positivity Score

Let $\hat{y}^f_{k,t,j} \in [0,1]$ denote the predicted probability that subset s_j, drawn from sample f, belongs to the unhealthy class, produced by model t trained in the k-th fold.
We define $\hat{\mu}^f_{k,j}$ as the average prediction across all T seeds for subset s_j in fold k:

$$\hat{\mu}^f_{k,j} = \frac{1}{T} \sum_{t=1}^{T} \hat{y}^f_{k,t,j}$$

We then define $\hat{\mu}^f_j$ as the average across all K folds:

$$\hat{\mu}^f_j = \frac{1}{K} \sum_{k=1}^{K} \hat{\mu}^f_{k,j}$$

The Positivity Score $M_f \in [0,1]$ is then obtained by averaging $\hat{\mu}^f_j$ across all n_{res} subsets, representing the expected likelihood of sample f containing unhealthy cells:

$$M_f = \frac{1}{n_{res}} \sum_{j=1}^{n_{res}} \hat{\mu}^f_j = \frac{1}{n_{res} \cdot K \cdot T} \sum_{j=1}^{n_{res}} \sum_{k=1}^{K} \sum_{t=1}^{T} \hat{y}^f_{k,t,j}$$

Applying the above procedure to all $f \in \mathcal{S}_{test}$, we obtain the vector of Positivity Scores:

$$\mathbf{M} = \left[M_1, \ldots, M_{|\mathcal{S}_{test}|} \right]$$

where each entry corresponds to one sample in the test set.

3.7.4 Classification Function

Each sample $f \in \mathcal{S}_{test}$ is classified based on its Positivity Score M_f using the following rule:

$$\tilde{y}_f = \mathbb{1}_{[\tau^*_\zeta, 1]}(M_f) = \begin{cases} 1 & \text{if } M_f \geq \tau^*_\zeta \\ 0 & \text{otherwise} \end{cases}$$

where $\tau^*_\zeta \in [0,1]$ is the tuned threshold and $\zeta \in \{ROC, RES\}$ is the tuning strategy (see Section 3.7.5).
The full procedure is summarised in Algorithm 6.

Algorithm 6 Test Set Prediction

Require: Test set $\mathcal{S}_{test}$, trained models $\{m_{k,t}\}_{k=1,\,t=1}^{K,\,T}$, subset size n_s, number of subsets n_{res}, threshold τ^*_ζ

for each sample $f \in \mathcal{S}_{test}$ **do**
  Draw n_{res} subsets $\{s_1, \ldots, s_{n_{res}}\}$ of size n_s from f, independently and with replacement
  for $j = 1$ **to** n_{res} **do**
    $\hat{\mu}^f_j \leftarrow \frac{1}{K \cdot T} \sum_{k=1}^{K} \sum_{t=1}^{T} \hat{y}^f_{k,t,j}$ ▷ Average probability across folds and seeds
  end for
  $M_f \leftarrow \frac{1}{n_{res}} \sum_{j=1}^{n_{res}} \hat{\mu}^f_j$ ▷ Positivity Score
  $\tilde{y}_f \leftarrow \mathbb{1}_{[\tau^*_\zeta, 1]}(M_f)$ ▷ Classify sample
end for

3.7.5 Threshold Optimization on the Validation Set

Threshold tuning aims to find a value τ_ζ^* that best discriminates the healthy and unhealthy classes. For both RES and ROC approaches, this tuning phase may be performed on the validation set and follows the same resampling structure described in Section 3.7.
Depending on the Training Procedure we are employing, this tuning phase may be performed either after Bayesian Optimization or after the final training phase. Since both cases share the same structure, only one is covered here.
Let $\mathcal{S}_{val}^k$ denote the validation set of the k-th CV fold. For each sample $f \in \mathcal{S}_{val}^k$, n_{res} subsets of fixed size n_s are independently drawn with replacement. Since each sample appears in the validation set of exactly one fold, no averaging across folds is performed. The Positivity Score for validation sample f in fold k is computed as:

$$M_f^k = \frac{1}{n_{res}} \sum_{j=1}^{n_{res}} \hat{\mu}_{k,j}^f = \frac{1}{n_{res} \cdot T} \sum_{j=1}^{n_{res}} \sum_{t=1}^{T} \hat{y}_{k,t,j}^f$$

where k denotes the fold in which f appears as a validation sample, and $\hat{\mu}_{k,j}^f = \frac{1}{T} \sum_{t=1}^{T} \hat{y}_{k,t,j}^f$ is the average predictions across seeds for subset s_j.

3.7.6 RES-tuning approach

For each fold $k \in \{1, \ldots, K\}$ and each sample $f \in \mathcal{S}_{val}^k$, n_{res} subsets of size n_s are independently drawn with replacement. Each subset s_j is predicted by the T models of fold k, and the average predictions across seeds prediction is computed as:

$$\hat{\mu}_{k,j}^f = \frac{1}{T} \sum_{t=1}^{T} \hat{y}_{k,t,j}^f$$

Computing $\hat{\mu}_{k,j}^f$ $\forall j \in \{1, \ldots, n_{res}\}$, $\forall f \in \mathcal{S}_{val}^k$, and $\forall k \in \{1, \ldots, K\}$, we obtain the collection of average predictions across seeds predictions:

$$\hat{\Psi} = \left\{ \hat{\mu}_{k,j}^f : \ k \in \{1, \ldots, K\}, \ f \in \mathcal{S}_{val}^k, \ j \in \{1, \ldots, n_{res}\} \right\}$$

of cardinality $|\hat{\Psi}| = n_{res} \times |\mathcal{S} \setminus \mathcal{S}_{test}|$, since each sample appears in the validation set of exactly one fold.
For each $\tau \in \{0.00, 0.01, \ldots, 1.00\}$, each element of $\hat{\Psi}$ is classified as:

$$\tilde{y}_{k,j}^f(\tau) = \mathbb{1}_{[\tau,1]} \left(\hat{\mu}_{k,j}^f \right) = \begin{cases} 1 & \text{if } \hat{\mu}_{k,j}^f \geq \tau \\ 0 & \text{otherwise} \end{cases}$$

obtaining the collection of predicted binary labels:

$$\tilde{\Psi}(\tau) = \left\{ \tilde{y}_{k,j}^f(\tau) : \ k \in \{1, \ldots, K\}, \ f \in \mathcal{S}_{val}^k, \ j \in \{1, \ldots, n_{res}\} \right\}$$

The F_1-score is then computed by comparing $\tilde{\Psi}(\tau)$ against the ground truth collection:

$$\Psi = \left\{ y_{k,j}^f : \ k \in \{1, \ldots, K\}, \ f \in \mathcal{S}_{val}^k, \ j \in \{1, \ldots, n_{res}\} \right\}$$

The optimal threshold is selected as:

$$\tau_{RES}^* = \arg\min_{\tau \in \mathcal{T}} \ |\tau - 0.5|$$

where $\mathcal{T} = \left\{\tau \in [0,1] : F_1(\tau) = \max_{\tau' \in [0,1]} F_1(\tau')\right\}$ is the set of thresholds achieving the maximum F_1-score. If the $\tau = \arg\max_{\tau' \in [0,1]} F_1(\tau')$ is unique, $\mathcal{T}$ is a singleton and $\tau^*_{RES} = \arg\max_{\tau \in [0,1]} F_1(\tau)$.

Algorithm 7 RES Threshold Tuning

Require: Validation sets $\{\mathcal{S}^k_{val}\}_{k=1}^{K}$, trained models $\{m_{k,t}\}_{k=1,t=1}^{K,T}$, subset size n_s, number of subsets n_{res}

for $k = 1$ **to** K **do**
  for each sample $f \in \mathcal{S}^k_{val}$ **do**
    Draw n_{res} subsets $\{s_1, \ldots, s_{n_{res}}\}$ of size n_s from f, independently with replacement
    for $j = 1$ **to** n_{res} **do**
      for $t = 1$ **to** T **do**
        Compute $\hat{y}^f_{k,t,j}$
      end for
      $\hat{\mu}^f_{k,j} \leftarrow \dfrac{1}{T}\sum_{t=1}^{T} \hat{y}^f_{k,t,j}$ ▷ average predictions across seeds prediction
    end for
  end for
end for
Collect all $\hat{\mu}^f_{k,j}$ into $\hat{\Psi}$
Collect all $y^f_{k,j}$ into Ψ
for $\tau \in \{0.00, 0.01, \ldots, 1.00\}$ **do**
  Compute $\hat{\Psi}(\tau)$ and $F_1(\tau)$ against Ψ
end for
$\mathcal{T} \leftarrow \{\tau : F_1(\tau) = \max_{\tau'} F_1(\tau')\}$
$\tau^*_{RES} \leftarrow \arg\min_{\tau \in \mathcal{T}} |\tau - 0.5|$

3.7.7 ROC-tuning approach

While the RES-tuned approach works at the subset level, the ROC-tuning approach aggregates predictions at the sample level. For each sample $f \in \mathcal{S}^k_{val}$, predictions are averaged across both seeds and subsets, obtaining a single Positivity Score $M^k_f \in [0,1]$:

$$M^k_f = \frac{1}{n_{res} \cdot T} \sum_{j=1}^{n_{res}} \sum_{t=1}^{T} \hat{y}^f_{k,t,j}$$

The sample-level Positivity Scores are collected in the vector:

$$\mathbf{M}^{val} = \left[M^k_f : \ k \in \{1, \ldots, K\}, \ f \in \mathcal{S}^k_{val}\right]$$

of length $|\mathcal{S} \setminus \mathcal{S}_{test}|$, since each sample appears in the validation set of exactly one fold. The threshold τ^*_{ROC} is selected as the smallest τ such that $TPR(\tau) \geq 0.95$, i.e.

$$\tau^*_{ROC} = \min\{\tau : TPR(\tau) \geq 0.95\}$$

where $TPR(\tau)$ and $FPR(\tau)$ are computed by classifying each sample $f \in \mathcal{S} \setminus \mathcal{S}_{test}$ as $\mathbb{1}_{[\tau,1]}(M^k_f)$ and comparing against the sample-level ground truth label y_f, collected in $\mathbf{y}^{val}$. The value $TPR = 0.95$ was chosen empirically after observing that the Youden index [35] optimized as

$$\tau^*_J = \arg\max_{\tau \in [0,1]} (J(\tau)) \quad \text{where } J(\tau) = TPR(\tau) - FPR(\tau)$$

produced suboptimal results on the validation set. On the other hand, $TPR = 1.0$ generally returned extremely low thresholds. The value $TPR = 0.95$ represents a compromise between prioritizing the prediction sensitivity and specificity.

Algorithm 8 ROC Threshold Tuning

Require: Validation sets $\{\mathcal{S}_{val}^k\}_{k=1}^K$, trained models $\{m_{k,t}\}_{k=1,t=1}^{K,T}$, subset size n_s, number of subsets n_{res}

for $k = 1$ **to** K **do**
  for each sample $f \in \mathcal{S}_{val}^k$ **do**
    Draw n_{res} subsets $\{s_1, \ldots, s_{n_{res}}\}$ of size n_s from f, independently with replacement
    for $j = 1$ **to** n_{res} **do**
      for $t = 1$ **to** T **do**
        Compute $\hat{y}_{k,t,j}^f$
      end for
      $\hat{\mu}_{k,j}^f \leftarrow \frac{1}{T} \sum_{t=1}^{T} \hat{y}_{k,t,j}^f$ ▷ Seed-averaged prediction
    end for
    $M_f^k \leftarrow \frac{1}{n_{res}} \sum_{j=1}^{n_{res}} \hat{\mu}_{k,j}^f$ ▷ Positivity Score
  end for
end for
Collect M_f^k into $\mathbf{M}^{val}$ and ground truth labels y_f into $\mathbf{y}^{val}$
Compute ROC curve from $\mathbf{M}^{val}$ against $\mathbf{y}^{val}$
$\tau_{ROC}^* \leftarrow \min\{\tau : TPR(\tau) \geq 0.95\}$

3.8 Training Procedures

This section describes two distinct Training Procedures that share the parameter tuning phase, but differ in how the training is structured.
The classification and threshold tuning procedures described in Section 3.7 and Section 3.7.6 are components of both Training Procedures. Specifically, the ensemble prediction methodology performed for test prediction and the resampling-based classification (Section 3.7) are applied during the testing phase, while threshold optimization (Sections 3.7.7–3.7.6) is performed on the validation set after the tuning phase (see Section 3.8.1 for Procedure 1 or after the final training phase for procedure 2). The present section focuses on the training structure, referring back to those methods where needed.

3.8.1 Parameters Tuning via Bayesian Optimization

As described in Section 1.8, the internal data augmentation of CellCNN is driven by two parameters: the number of subsets drawn from each sample (n_{subs}) and the number of cells per subset (n_{cells}). The objective of this phase is to identify the optimal configuration $\theta^* = (n_{subs}, n_{cells})$ within a predefined search space Θ.
Bayesian Optimization (BO) [10] is a sequential algorithm for hyperparameter tuning that aims to find the optimal hyperparameter configuration $\theta^* \in \Theta$, by building a probabilistic surrogate model of the true unknown objective function $\psi : \Theta \to \mathbb{R}$
The surrogate model computes a posterior distribution $P(\psi \mid \mathcal{D})$ over the objective function, updated at each step via Bayes' Theorem:

$$P(\psi \mid \mathcal{D}) = \frac{P(\mathcal{D} \mid \psi), P(\psi)}{P(\mathcal{D})}$$

where $\mathrm{P}(\psi)$ is the prior distribution, representing the initial beliefs about the objective function before any evaluation, $\mathrm{P}(\mathcal{D}|\psi)$ is the likelihood of the observations given ψ, $\mathrm{P}(\psi|\mathcal{D})$ is

the posterior distribution, which provides an estimate of the objective function given the explored configuration evaluations, and

$$P(\mathcal{D}) = \int P(\mathcal{D} \mid \psi), P(\psi), d\psi$$

is the normalization constant. Finally, $\mathcal{D} = \{(\theta_i, y_{\theta_i})\}_{i=1}^{n_\theta}$ denotes the collection of the observed configurations, θ_i is the i-th explored configuration, y_{θ_i} is the corresponding score, and n_θ is the number of explored configurations.
At each iteration, the next configuration to evaluate is selected by maximizing an acquisition function $\lambda : \Theta \to \mathbb{R}$.

$$\theta_{next} = \arg\max_{\theta \in \Theta} \lambda \left(P(\psi \mid \mathcal{D}), \mathcal{D}\right)$$

The acquisition function is defined from the posterior distribution to assign more weight on the unexplored regions or regions that are expected to perform well. After θ_{next} evaluation, $\mathcal{D}$ is updated and the posterior is recomputed, until n_θ configurations are explored.

In this study, the evaluation of θ_{next} is achieved through F_1-score. Additionally, due to computational constraints, the hyperparameter space Θ and n_θ are very limited.

To ensure the robustness of the tuning process, each proposed configuration θ_i is evaluated using K-Fold Cross-Validation (with $K = 5$) [15]. For each fold:

1. The model is trained on the training partition using configuration θ_i.
2. A validation set is used for Early Stopping to prevent overfitting and for threshold optimization.

The mean F_1-score across folds is used as the objective function:

$$\Omega(\theta_i) = \frac{1}{K} \sum_{k=1}^{K} F_1^{(k)}(\theta_i)$$

The probability distribution over the parameter space is updated based on $\Omega(\theta_i)$ at each iteration.

Computational Efficiency and Pruning Given the high computational cost of training multiple CNNs per iteration, we implement a pruning strategy. If the intermediate performance of a configuration after 3 out of 5 folds falls below the median performance of all previously evaluated configurations, its evaluation is terminated early. This allows a larger number of configurations θ_i to be explored within the available computational budget.

Choice of θ^* When multiple θ_i are associated to the highest scored F_1-score value, a tie-breaking procedure is applied. We observed in preliminary experiments that configurations with larger values of n_{cells} and n_{subs} tend to achieve better generalisation performance compared to other top-scoring configurations. To incorporate this observation without selecting extreme parameter values arbitrarily, we adopt the following 5-step procedure:

1. Select the set of configurations achieving the maximum F_1-score:

$$\Theta' = \left\{\theta_i \in \Theta : \Omega(\theta_i) = \max_{\theta \in \Theta} \Omega(\theta)\right\}$$

2. Compute the empirical sum $n_{cells} + n_{subs}$ for each $\theta_i \in \Theta'$.
3. Sort the configurations in ascending order based on their empirical sum.

4. Discard the $\left\lceil \frac{|\Theta'|}{2} \right\rceil$ configurations with the smallest empirical sum, obtaining Θ'', such that $|\Theta''| = \left\lfloor \frac{|\Theta'|}{2} \right\rfloor$.

5. Select the upper median of Θ'': if $|\Theta''|$ is odd, select the middle element; if even, select the element at position $\left\lceil \frac{|\Theta''|}{2} \right\rceil$.

This criterion favors configurations with higher sample coverage, while avoiding the systematic selection of extreme parameter values. An example is presented in Section 3.10.
The complete procedure for selecting the optimal configuration is summarized in Algorithm 9.

Algorithm 9 Optimal Configuration θ^* Selection (Tie-Breaking)

Require: Set of evaluated configurations $\mathcal{D} = \{(\theta_i, \Omega(\theta_i)\}_{i=1}^{n_\theta}$ where $\theta_i = (n_{cells}^{\theta_i}, n_{subs}^{\theta_i})$
  $\Omega_{max} \leftarrow \max_{\theta \in D} \Omega(\theta)$
  $\Theta' \leftarrow \{\theta \in D : \Omega(\theta) = \Omega_{max}\}$
  if $|\Theta'| == 1$ **then return** $\theta \in \Theta'$
  else
    // Compute empirical sums
    for each $\theta \in \Theta'$ **do**
      $ES(\theta) \leftarrow n_{cells}^{\theta} + n_{subs}^{\theta}$
    end for
    // Sort configurations
    Sort Θ' in ascending order based on $ES(\theta)$
    // Discard lower half
    $discard_count \leftarrow \lceil |\Theta'|/2 \rceil$
    $\Theta'' \leftarrow \Theta'[discard_count + 1 \dots |\Theta'|]$ ▷ Resulting size is $\lfloor |\Theta'|/2 \rfloor$
    // Select upper median
    $idx \leftarrow \lceil |\Theta''|/2 \rceil$
    $\theta^* \leftarrow \Theta''[idx]$ **return** θ^*
  end if

3.8.2 Training Procedure 1: Randomized Split and Multi-Seed Initialization

The model is trained on $\mathcal{S}_{train}$ using the optimal configuration θ^*, with a higher number of epochs than those employed during BO, while $\mathcal{S}_{val}$ is used for early stopping to prevent overfitting. The first procedure follows the original CellCNN training structure. The set of patients, except for the left-out patient (see Section 3.8.5), is randomly divided using an 80-20% split into a training set $\mathcal{S}_{train}$ and a validation set $\mathcal{S}_{val}$, respectively. To mitigate the sensitivity of gradient descent to initial conditions, we perform T independent training runs using different random weight initializations. This results in T trained models. Each model predicts the samples in the test set $\mathcal{S}_{test}$, and the final prediction is obtained by aggregating their outputs, reducing the variance associated with any single weight initialization.
The classification threshold τ_ζ^* (to which we refer as $\tau_\zeta^{*(BO)}$ for procedure 1) is determined when BO is concluded. Specifically, the $K \times T$ models trained during the BO phase are used to predict their respective validation sets $\mathcal{S}_{val}^k$, and the optimal thresholds are computed following the procedures described in Sections 3.7.7 and 3.7.6, respectively.

3.8.3 Training Procedure 2: Cross-Validation Ensemble

The second procedure adopts an ensemble learning approach inspired by Bagging [36], combined with a K-Fold Cross-Validation [15]. The same K partitions generated during the BO phase are reused, since we tuned θ^* on these specific partitions.

Classical Bagging trains each model on a bootstrap sample, which in our case would be a set of patients, $\mathcal{S}_{train}^{boot}$, obtained by sampling with replacement $|S_{train}|$ patients from S_{train} and averaging the resulting model predictions [36].
In this study, instead of performing bootstrap sampling, models are trained on partitions generated by a K-Fold CV (details in Section 3.7.3 and 3.10) and predictions are averaged across CV iterations. This results in a Cross-Validation Ensemble, which obtains the variance-reduction benefits of Bagging [36] while bootstrap sampling redundancy is not introduced.
As in procedure 1, the number of epochs is increased with respect to those used during BO to allow better convergence.

For each fold $k \in \{1, \ldots, K\}$, T models are trained on $\mathcal{S}_{train}^{k}$ and validated on $\mathcal{S}_{val}^{k}$. Differently from Procedure 1, where the threshold τ_{ζ}^{*} is determined using the BO validation predictions, Procedure 2 performs threshold optimization after the final training phase, using the validation predictions produced by the trained ensemble. The optimal thresholds τ_{ROC}^{*} and τ_{RES}^{*} are computed following Sections 3.7.7 and 3.7.6, respectively.

3.8.4 Sample Classification

Both Training Procedures generate a collection of trained models, which are used to predict test samples using the Positivity Score M_f and the classification rule defined in Section 3.7.4. Three threshold configurations are built:

- CellCNN approach ($\tau = 0.5$): the default fixed threshold is applied directly to M_f, as in the original CellCNN framework.
- ROC-tuned threshold (τ_{ROC}^{*}): the smallest threshold such that $TPR(\tau) \geq 0.95$ on the validation set (Section 3.7.7).
- RES-tuned threshold (τ_{RES}^{*}): the threshold maximizing the F_1-score on the validation set (Section 3.7.6).

Note that τ_{ROC}^{*} and τ_{RES}^{*} are employed for Procedure 2, while $\tau_{ROC}^{*(BO)}$ and $\tau_{RES}^{*(BO)}$ are employed for Procedure 1.

3.8.5 Leave-One-Patient-Out Cross-Validation

To evaluate the robustness of the tuning and Training Procedures, we employ a Leave-One-Patient-Out Cross-Validation (LOPOCV). This is a specialized variant of Leave-One-Out Cross-Validation (LOOCV) [11] designed to maintain the independence of the training and test sets, since standard cross-validation may assign different samples from the same patient to both sets simultaneously, leading to data leakage and over-optimistic performance estimates. LOPOCV prevents this by ensuring that all samples from a given patient are assigned to the same set at each iteration.

Let $\mathcal{P} = \{p_1, \ldots, p_N\}$ be the set of N patients. At each iteration $i \in \{1, \ldots, N\}$, the model is tuned and trained on $\mathcal{P} \setminus \{p_i\}$ and evaluated on the held-out patient's samples:

$$\mathcal{S}_{test}^{(i)} := \{f : f \text{ belongs to patient } p_i\}$$

Depending on the procedure employed, samples $f \in \mathcal{S}_{test}$ are predicted by T models (Procedure 1, where $K = 1$) or K models (Procedure 2, where $T = 1$), resulting in a total of $T \times K$ predictions per sample. These are aggregated into a Positivity Score M_f for each sample f, which is then converted in a binary label $\tilde{y}_f \in \{0, 1\}$ via the classification rule defined in Section 3.7.4, 3.7.6, 3.7.7.

At each LOPOCV iteration, model performance on the held-out patient is evaluated in the following three ways:

1. Seed/Fold Sensitivity Evaluation: each of the $T \times K$ individual predictions are evaluated independently to retrieve the variability of the model with respect to weight initialization (Procedure 1) or training-validation splits (Procedure 2). For consistency of notation, we refer to $T \times K$ predictions for both procedures, knowing that $K = 1$ in Procedure 1 and $T = 1$ in Procedure 2. The F_1-score, Recall, Precision, and Accuracy are computed for each of the $T \times K$ predictions.
2. Ensemble Prediction: the $T \times K$ predictions are aggregated by averaging across seeds and folds, computing the Positivity Score M_f for each sample f, as defined in Section 3.7. The F_1-score, Recall, Precision, and Accuracy are then computed from the resulting ensemble predictions.
3. LOPOCV Inference: F_1-score, Recall, Precision and Accuracy are computed across LOPOCV iterations, producing a final estimate of the Procedures' performance.

3.9 Experimental Setup

All analyses were performed using the following open-source libraries: TensorFlow [1], Scikit-learn [27], NumPy [14] and Pandas [23].
Libraries versions are reported in Table 5

Library	TensorFlow	Scikit-learn	Numpy	Pandas
Version	2.19.0	1.6.1	2.0.2	2.2.2

Table 5: Versions of the open-source libiearies employed in this study.

Due to hardware and package version constraints and the available resource budget, the final training and hyperparameter tuning experiments were conducted on Google Colab [13], using the CPU-based runtime. The code repository and package requirements are available on GitHub (`https://github.com/enricodidoli-source/master-thesis`).

3.9.1 Implemented Parameters

Tuning and training parameters were selected to balance model capacity and computational feasibility.
The Tables 6, 7 and 8 report these parameters, distinguishing between shared parameters, phase-dependent parameters, and AS generation parameters respectively.

Parameter	Notation	Value(s)	Reason
Convolutional Filters	$\mathcal{F}$	$[5, 7, 9]$	Subset of the original collection proposed by CellCNN. Chosen to balance computational cost and model capabilities
Top % max-Pooling	/	$[0.01, 1, 10, 100]$	Subset of the original collection originally proposed by CellCNN
Learning Rates	η	$[0.001, 0.005]$	Subset of the original collection originally proposed by CellCNN
Patience	/	5	Empirical value originally proposed by CellCNN

Table 6: Tuning-Training shared hyperparameters. $\mathcal{F}$ represents the number of convolutional filters and η represents the learning rate. Patience is a key parameter for early stopping which defines the number of consecutive iterations without improvement in the validation loss before training is stops.

Parameter	Notation	Tuning Value(s)	Training Value(s)	Prediction Value(s)	Reason
Hyperparameters search	/	Random	Grid	/	Random search during hyperparameters tuning to reduce cost; full grid search during final training (over $\mathcal{F}$, max-pooling, and η)
N. Explored Hyper. Comb.	/	10	24	/	24 is the algebraic product of $\mathcal{F} \times$ max-pooling $\times \eta$
N. Epochs	E	35	100	/	Tuning value has been chosen to reduce costs; training value has been originally proposed by CellCNN
Max n_{cells}, n_{subs}	θ^{max}	$(700, 700)$	/	/	Upper bound of BO search space, selected under hardware constraints
N. n_{cells}, n_{subs} Comb.	n_θ	15	/	/	Different combinations of θ explored during BO
N. subsets drawn from a sample f	n_{res}	/	/	50	Computational contraints

Table 7: Training-Tuning-dependent parameters. $\mathcal{F}$ represents the number of convolutional filters, η represents the learning rate and BO stands for Bayesian Optimization.

Parameter	**Notation**	**Value(s)**	**Reason**
Number of AS	n_{as}	3	Determined by the length of u; one unhealthy AS is generated per target MRD level.
Number of cells per AS	q	10^5	Selected under memory constraints, while ensuring enough cells for unhealthy cells distributions
% of Unhealthy cells per AS	u	$[0.1\%, 0.05\%, 0.01\%]$	Covers clinically relevant MRD ranges

Table 8: Artificial Samples (AS) generation parameters. MRD is the abbreviation of Measurable Residual Disease.

3.10 Detailed walkthrough of LOPOCV iteration 1

Before presenting aggregated results, we report the BO, the RES-tuning threshold procedure and final classification procedure for the first LOPOCV iteration, where patient 1 is held out as test patient, to illustrate concretely how predictions are generated.
The aggregated results across all LOPOCV iterations are shown in the following subsection. The test patient for this iteration is Patient 1, whose two samples are described in Table 9. The first sample (D15) is MRD-positive with 0.0876% unhealthy cells, while the second (D78) is MRD-negative.

Pat. ID	**Time Point**	**TOT cells**	**Healthy c.**	**Unhealthy c.**	**%Unhealthy c.**	**Label($y_{f_j^p}$)**
1	D15	864 000	863243	757	0.0876%	1
1	D78	1 947 518	1947518	0	0%	0

Table 9: Samples of the test patient (Patient 1). A label of 1 indicates an MRD-positive sample; 0 indicates MRD-negative.

The remaining patients are distributed across $K = 5$ folds for the CV procedure, as reported in Table 10.

Fold ID	**Patients ID**	**N.Samples per Patient**	**%Unhealthy c.**
1	11, 4	3, 3	[0.160%, 0.012%, 0%], [1.929%, 0.040%, 0%]
2	6	2	[1.435%, 0.008%]
3	9, 7	2, 3	[27.792%, 0.280%], [0.315%, 0.207%, 0%]
4	3, 8	2, 2	[0.035%, 0%], [0.5%, 0.017%]
5	12, 2	2,2	[0.148%, 0.002%], [39.558%, 15.063%]

Table 10: Patient distribution across the 5 CV folds. Percentages of unhealthy cells are listed in time-point order for each patient.

At each CV iteration k, four patients folds are grouped in the training set $\mathcal{S}^k_{train}$, while the remaining one is set as $\mathcal{S}^k_{val}$. Table 11 summarises the patients within the train and the validation sets at each of the 5 CV iterations.

CV iteration	Training set $\mathcal{S}_{train}$	Validation set $\mathcal{S}_{val}$	Test set $\mathcal{S}_{test}$
1	6, 9, 7, 3, 8, 12, 2	11, 4	1
2	11, 4, 9, 7, 3, 8, 12, 2	6	1
3	11, 4, 6, 3, 8, 12, 2	9, 7	1
4	11, 4, 6, 9, 7, 12, 2	3, 8	1
5	11, 4, 6, 9, 7, 3, 8	12, 2	1

Table 11: Patients per CV iteration (first LOPOCV iteration).

Bayesian Tuning Results During BO, each configuration θ was evaluated across all $K = 5$ CV folds, resulting in one F_1-score per fold. The average F_1-score associated with each θ is then computed across the 5 folds.
A unique θ^* is selected according to the procedure described in Section 3.8.1. The result of each step is reported in Table 13, while the set BO-proposed configurations and their associated F1-scores is shown in Figure 4.

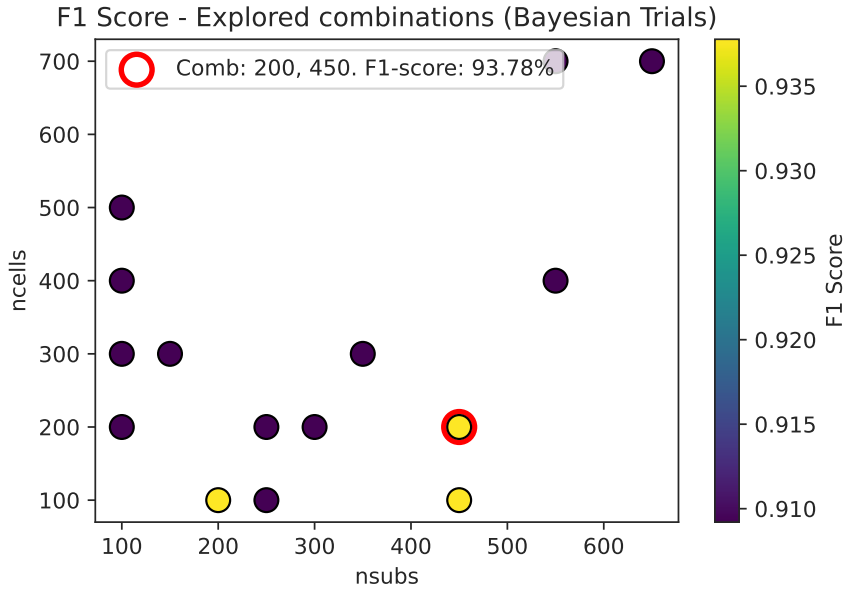


Figure 4: BO-explored configurations $\theta = (n_{cells}, n_{subs})$ and their associated F_1-scores. The selected configuration $\theta^* = (200, 450)$ is highlighted with a red circle.

BO i-th proposal	θ (n_{cells}, n_{subs})	F_1**-score**
1	(100, 450)	0.9378
2	(400, 100)	0.9092
3	(200, 450)	0.9378
4	(400, 550)	0.9092
5	(100, 250)	0.9092
6	(700, 650)	0.9092
7	(300, 150)	0.9092
8	(300, 350)	0.9092
9	(500, 100)	0.9092
10	(700, 550)	0.9092
11	(100, 200)	0.9378
12	(200, 250)	0.9092
13	(200, 300)	0.9092
14	(200, 100)	0.9092
15	(300, 100)	0.9092

Table 12: Configurations proposed during Bayesian Optimization(BO) with associated mean F_1-scores across $K = 5$ folds.

As shown in Table 12, only three configurations, (100, 450), (200, 450), and (100, 200), achieved the highest F_1-score of 0.9378, while the remaining twelve configurations obtained a lower score of 0.9092.
Additionally, none of the configurations proposed by the BO performed below the median performance of previously evaluated configurations; therefore, no pruning occurred. The step-by-step θ^* selecting procedure is shown in Table 13.

Step 1: select the θ associated to the highest F_1-score. If not unique, collect in Θ'	F_1**-score**	Associated parameters (Θ')		
	0.9378	(100, 450)	(200, 450)	(100, 200)

Step 2: compute the empirical sum $n_{subs} + n_{cells}$ for each $\theta \in \Theta'$		Θ'		
	θ	(100, 450)	(200, 450)	(100, 200)
	$n_{subs} + n_{cells}$	550	650	300

Step 3: sort Θ' by empirical sum (ascending)		Θ'		
	θ	(100, 200)	(100, 450)	(200, 450)
	$n_{subs} + n_{cells}$	300	550	650

Step 4: discard the $\lceil \lvert\Theta'\rvert/2 \rceil = 2$ configurations with smallest empirical sum obtaining Θ''		Θ''
	θ	(200, 450)

Step 5: $\theta^* =$ median(Θ'')		Θ''
	θ^*	(200, 450)

Table 13: Step-by-step θ^* selection procedure.

The selected optimal configuration is therefore $\theta^* = (200, 450)$, corresponding to $n_{cells} = 200$ cells and $n_{subs} = 450$ subsets per subset, which is then used in both Training Procedure 1 and Procedure 2.

3.10.1 RES-Tuned Threshold

The threshold τ^*_{RES} is tuned following the procedure presented in Section 3.7.6.
Since only $T = 1$ model is trained per fold in both post-training and post-BO threshold optimization, the seed index is dropped throughout this section.
Each fold's validation set consists of samples from one or more patients. From each sample f, $n_{res} = 50$ subsets are drawn.
To present the results for LOPOCV iteration 1, we slightly modify the sample notation to include the patient index, denoting each subset prediction as $\hat{y}^{f_i^p}_{k,j}$, where $k \in \{1, \ldots, 5\}$ is the

fold index, $j \in \{1, \ldots, 50\}$ is the subset index, and f_i^p is the i-th sample of patient p. Additionally, Table 14 shows the subset predictions for the validation set of the last CV fold ($k = 5$, $p \in \{12, 2\}$).

Subset / Sample f_i^p	s_1	s_2	$\ldots$	s_{49}	s_{50}	**True Label (y_f)**
f_1^{12}	$\hat{y}_{5,1}^{f_1^{12}} = 0.9836$	0.9837	$\ldots$	0.9838	0.9835	1
f_2^{12}	0.9535	0.9545	$\ldots$	0.9545	0.9550	1
f_1^2	0.8324	0.8309	$\ldots$	0.8322	0.8265	1
f_2^2	0.9200	0.9200	$\ldots$	0.9227	0.9247	1

Table 14: Subset predictions for patients 12 and 2, belonging to the validation set of the fifth CV fold ($k = 5$). f_i^p represents the i-th sample from patient p, while s_j represents the j-th subsets drawn from a sample.

For notational simplicity, let $\mathbb{F} = \{f_i\}_{i=1}^{f_{TOT}}$ denote the collection of all samples from non-left-out patients, where $f_{TOT} = 21$ for this LOPOCV iteration (see column "N. Samples" in Table 2). All subset predictions are then concatenated across folds, patients, and samples into:

$$\hat{Y} = \{\hat{y}_j\}_{j=1}^{n_{res} \times f_{TOT}}, \qquad Y = \{y_j\}_{j=1}^{n_{res} \times f_{TOT}}$$

for a total of $n_{res} \times f_{TOT} = 50 \times 21 = 1050$ elements in, as illustrated in Table 15.

f_i^p	f_1^{11}	$\ldots$	f_1^{12}			$\ldots$	f_2^4		
j-th subset	1	$\ldots$	851	$\ldots$	900	$\ldots$	1001	$\ldots$	1050
$\hat{y}_j$	0.9617	$\ldots$	0.9836	$\ldots$	0.9535	$\ldots$	0.92	$\ldots$	0.9247
y_j	1	$\ldots$	1	$\ldots$	1	$\ldots$	1	$\ldots$	1

Table 15: Prediction collection $\hat{Y}$ and ground-truth collection Y, obtained by concatenating subset predictions across all folds, patients, and samples. f_i^p represents the i-th sample from patient p.

For each $\tau \in \{0.00, 0.01, \ldots, 1.00\}$, each element of $\hat{Y}$ is classified and the F_1-score is computed against Y. Table 16 summarises the results for selected threshold values.

			Threshold τ					
Subset	$\hat{y}_j$	y_j	0.00	0.01	$\ldots$	0.78	$\ldots$	1.00
1	0.9617	1	1	1	$\ldots$	1	$\ldots$	0
			$\ldots$					
1050	0.9836	1	1	1	$\ldots$	1	$\ldots$	0
F_1-score			0.8924	0.8924	$\ldots$	0.9387	$\ldots$	0

Table 16: Classification results and F_1-scores under selected threshold values $\tau \in [0, 1]$. Each column under the "Threshold τ" group reports the binary label assigned to each subset by using the correspoding threshold value τ. The last row reports the F_1-score computed over all subsets for each threshold value.

The highest F_1-score is achieved by multiple values of τ. Following Section 3.7.6, we collect them in:

$$\mathcal{T} = \{\tau \in [0,1] : F_1(\tau) = \max_{\tau' \in [0,1]} F_1(\tau')\}$$

and select the element closest to 0.5, yielding $\tau^*_{RES} = 0.78$ (see Figure 5).

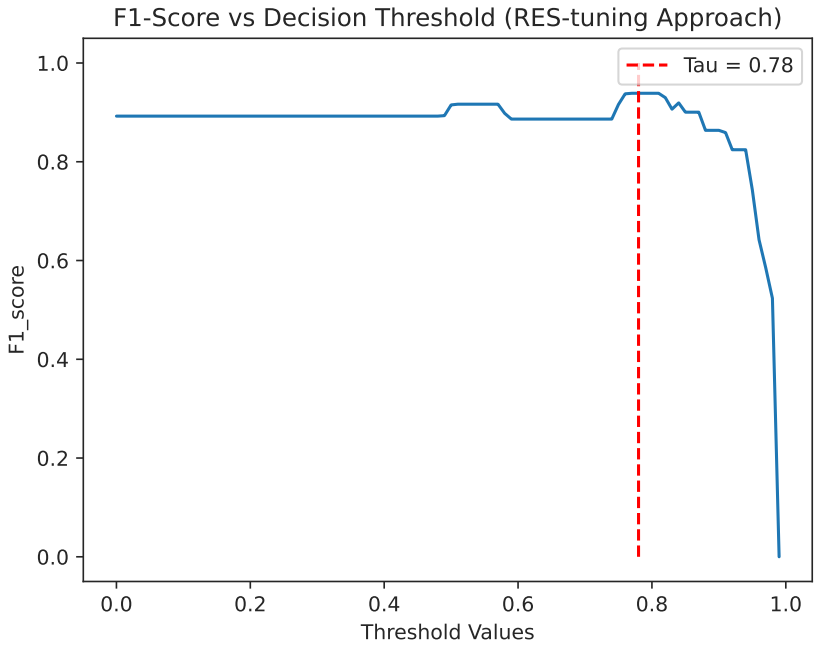


Figure 5: F_1-score as a function of the classification threshold τ (expressed as a percentage). The optimal threshold $\tau^*_{RES} = 0.78$ is highlighted in red.

In summary, for this LOPOCV iteration the optimized RES-threshold is $\tau^*_{RES} = 0.78$. It exceeds the default threshold of 0.5, indicating a less conservative decision boundary that increass the risk of misclassifying unhealthy samples as healthy.

Note that this example is derived from the post-training threshold optimization within Training Procedure 2 (Cross-Validation Ensemble) without AS. The same procedure applies analogously to the thresholds computed after BO, used in Training Procedure 1 or for the case in which AS are employed.

3.10.2 Test prediction and Labelling: Procedure 2

In this section we distinguish the prediction results according to the classification approach employed. The CellCNN approach classifies test samples using the fixed threshold $\tau = 0.5$, which serves as the baseline for threshold comparison. The Resampling approach evaluates predictions under both τ^*_{ROC} and τ^*_{RES}.

CellCNN prediction approach The subset size for test prediction is determined by the smallest sample in the test set. From Table 9, this is f_1^1 with 864 000 cells, so $n_s^{test} = 864\,000$. Since Procedure 2 uses $T = 1$ and CellCNN takes one subset only ($n_{res} = 1$), the Positivity Score reduces to:

$$
\begin{aligned}
M_{f_i^p} &= \frac{1}{n_{res} \cdot K \cdot T} \sum_{j=1}^{n_{res}} \sum_{k=1}^{K} \sum_{t=1}^{T} \hat{y}_{k,j,t}^{f_i^p} \\
&\overset{T=1}{=} \frac{1}{n_{res} \cdot K} \sum_{j=1}^{n_{res}} \sum_{k=1}^{K} \hat{\mu}_{k,j}^{f_i^p} \\
&\overset{n_{res}=1}{=} \frac{1}{K} \sum_{k=1}^{K} \hat{\mu}_{k}^{f_i^p}
\end{aligned}
$$

Since $n_{res} = T = 1$, each Fold-Level prediction $\hat{\mu}_k^{f_i^p}$ coincides with the single raw prediction $\hat{y}_{k,1,1}^{f_i^p}$, denoted by $\hat{y}_{f_i^p}^k$ following the notation for original CellCNN predictions introduced in Section 3.6). The fold-predictions of f_1^1 and f_2^1 are reported in Table 17.

	$y_{f_1^1} = 1$		$y_{f_2^1} = 0$	
Fold k	$\hat{y}_{f_1^1}^k$	$\tilde{y}_{f_1^1}^k$	$\hat{y}_{f_2^1}^k$	$\tilde{y}_{f_2^1}^k$
Fold 1	0.975943	1	0.762282	1
Fold 2	0.983196	1	0.751268	1
Fold 3	0.998809	1	0.737908	1
Fold 4	0.999972	1	0.390214	0
Fold 5	0.999673	1	0.359993	0

$M_{f_1^1}$	$\tilde{y}_{f_1^1}$	$M_{f_2^1}$	$\tilde{y}_{f_2^1}$
0.991519	1	0.600333	1

Table 17: Per-fold predictions and labels for the two test samples using the CellCNN approach ($\tau = 0.5$). The bottom row reports the ensemble mean and the corresponding label.

Resampling Prediction Approach Following the resampling procedure in Section 3.7, $n_{res} > 1$ subsets of fixed size are drawn from each test sample and predicted by each of the K fold models. The Positivity Score is computed as:

$$
\begin{aligned}
M_{f_i^p} &= \frac{1}{n_{res} \cdot K \cdot T} \sum_{j=1}^{n_{res}} \sum_{k=1}^{K} \sum_{t=1}^{T} \hat{y}_{k,t,j}^{f_i^p} \\
&\overset{T=1}{=} \frac{1}{n_{res} \cdot K} \sum_{j=1}^{n_{res}} \sum_{k=1}^{K} \hat{\mu}_{k,j}^{f_i^p} \\
&= \frac{1}{K} \sum_{k=1}^{K} \left(\frac{1}{n_{res}} \sum_{j=1}^{n_{res}} \hat{\mu}_{k,j}^{f_i^p} \right)
\end{aligned}
$$

Finally, the sample is then classified using τ^*_ζ with $\zeta \in \{ROC, RES\}$, depending on the chosen approach.
In table 18 we show the per-fold subset predictions for sample f_1^1.

Fold \ Subset	s_1	s_2	...	s_{49}	s_{50}
1	$\hat{\mu}^{f_1^1}_{1,1} = 0.975938$	0.975964	...	0.975933	0.975936
2	0.98319	0.983222	...	0.98319	0.983188
3	0.9298	0.9302	...	0.9302	0.9302
4	0.998738	0.998738	...	0.99883	0.998836
5	0.999972	0.999972	...	0.999973	0.999973

Table 18: Per-fold subset predictions $\hat{\mu}^{f_1^1}_{k,j}$ for sample f_1^1 ($y_{f_1^1} = 1$) using the Resampling approach.

We obtain the Positivity score $M_{f_1^1}$ by taking the mean across folds and subsets.
Note that averaging across subsets only would produce a score equivalent to a single model trained on a fixed split of patient. Our ensemble approach provides a more robust estimate by incorporating the variability introduced by different training configurations.
The per-subset prediction table for f_2^1 is omitted but we report the means across folds for both samples in Table 19.

	$y_{f_1^1} = 1$			$y_{f_2^1} = 0$		
Fold	$\hat{\mu}^{f_1^1}_k$	$\tilde{y}^k_{f_1^1}(\tau^*_{ROC})$	$\tilde{y}^k_{f_1^1}(\tau^*_{RES})$	$\hat{\mu}^{f_2^1}_k$	$\tilde{y}^k_{f_2^1}(\tau^*_{ROC})$	$\tilde{y}^k_{f_2^1}(\tau^*_{RES})$
1	$\hat{\mu}^{f_1^1}_1 = 0.975945$	$\tilde{y}^1_{f_1^1}(\tau^*_{ROC}) = 1$	1	0.762052	1	0
2	0.983187	1	1	0.751032	1	0
3	0.99881	1	1	0.731957	1	0
4	0.999972	1	1	0.389754	0	0
5	0.999674	1	1	0.360621	0	0

Fold	$M_{f_1^1}$	$\tilde{y}_{f_1^1}(\tau^*_{ROC}))$	$\tilde{y}_{f_1^1}(\tau^*_{RES}))$	$M_{f_2^1}$	$\tilde{y}_{f_2^1}(\tau^*_{ROC}))$	$\tilde{y}_{f_2^1}(\tau^*_{RES}))$
Ensemble	0.991518	1	1	0.599083	1	0

Table 19: Per-fold mean Positivity scores $\hat{\mu}^{f_i^1}_k$ and assigned labels for both test samples under τ^*_{ROC} and τ^*_{RES}. Thresholds are reported in Figure 6 and in Table 25. The bottom row reports the ensemble aggregation.

Metrics computation We compute the classification metrics for both Prediction Approaches and all threshold configurations. In particular, Tables 20 and 21 show the Cell-CNN and Resampling approaches respectively. Fold-Level metrics quantify the variability of the model across different training and validation set configurations, while the ensemble metrics represent the final prediction for this LOPOCV iteration. These metrics are shown in Figure 6 divided by Prediction Approach.

Original Approach	**Threshold** $\tau = 0.5$			
Framework \ Metric	F_1-score	Recall	Precision	Accuracy
Fold 1	0.6667	1	0.5	0.5
Fold 2	0.6667	1	0.5	0.5
Fold 3	0.6667	1	0.5	0.5
Fold 4	1	1	1	1
Fold 5	1	1	1	1
Ensemble	0.6667	1	0.5	0.5

Table 20: Classification metrics per fold, mean across folds, and ensemble for the CellCNN approach ($\tau = 0.5$) on the test patient of LOPOCV iteration 1.

Resampling Approach	**Threshold** $\tau^*_{ROC} = 0.5895$				**Threshold** $\tau^*_{RES} = 0.78$			
Framework \ Metric	F_1-score	Recall	Prec.	Acc.	F_1-score	Recall	Prec.	Acc.
Fold 1	0.6667	1	0.5	0.5	1	1	1	1
Fold 2	0.6667	1	0.5	0.5	1	1	1	1
Fold 3	0.6667	1	0.5	0.5	1	1	1	1
Fold 4	1	1	1	1	1	1	1	1
Fold 5	1	1	1	1	1	1	1	1
Ensemble	0.6667	1	0.5	0.5	1	1	1	1

Table 21: Classification metrics per fold, mean across folds, and ensemble for the Resampling approach under $\tau^*_{ROC} = 0.5895$ and $\tau^*_{RES} = 0.78$ on the test patient of LOPOCV iteration 1.

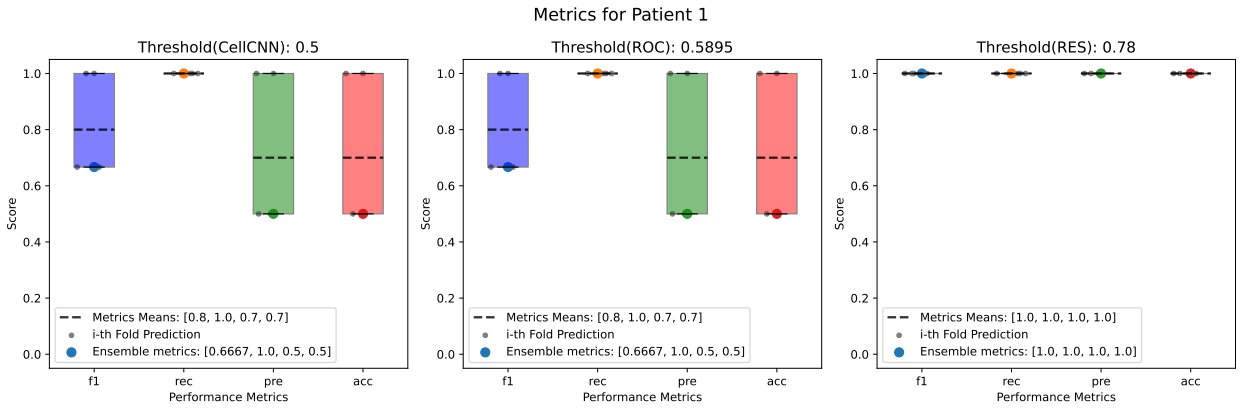


Figure 6: Classification metrics for Patient 1 across all three threshold configurations. Each boxplot shows the distribution of per-fold predictions (grey dots), the mean across folds (dashed line), and the ensemble metric (colored dot). The RES-tuned threshold ($\tau^*_{RES} = 0.78$) achieves perfect scores across all metrics, while CellCNN and ROC-tuned thresholds obtained a lower ensemble F_1-score of 0.6667 despite a higher per-fold mean of 0.8, due to the misclassification of the healthy sample f_2^1 at the ensemble level. "f1", "rec", "pre" and "acc" stand for F_1-score, Recall, Precision and Accuracy, respectively.

4 Results

The four experiments shown in Table 22 share a common Bayesian Optimization tuning phase, and they are designed to explore the effect of Artificial Samples generation on classification performance and the robustness of predictions across different training configurations.

Training Procedure 1 (Section 3.8.2) isolates the effect of weight initialization variability by performing T independent training runs on a fixed $80-20\%$ patient-level split, following the original CellCNN implementation. Training Procedure 2 (Section 3.8.3) instead evaluates the variability introduced by different patient combinations in training, by adopting a K-Fold CV ensemble. Both procedures are tested with and without AS generation, for a total of four Experiments, summarised in Table 22.

Two baselines are defined. The first consists of the Seed-Level and Fold-Level predictions, representing the originally proposed CellCNN classification approach, against which the multi-initialization and ensemble procedures are compared. The second is the fixed threshold $\tau = 0.5$, against which the ROC-tuned and RES-tuned threshold approaches are compared. The Prediction Approaches are summarized in Table 23.

		Training Dataset	
		Artificial Samples	Patient samples
Procedure	Procedure 1	Experiment 1	Experiment 2
	Procedure 2	Experiment 3	Experiment 4

Table 22: Summary of the four Experiments, defined by the combination of Training Procedure and training dataset.

	Prediction Approach		
	CellCNN	**Resampling**	
Experiments 1-2	$\tau = 0.5$	$\tau^{*(BO)}_{ROC}$	$\tau^{*(BO)}_{RES}$
Experiment 3-4	$\tau = 0.5$	$\tau = \tau^{*}_{ROC}$	$\tau = \tau^{*}_{RES}$

Table 23: Prediction Approaches applied in each Experiment.

4.1 Tuned θ^* per LOPOCV fold

In this section, the results about BO hyperparameter search are presented. Each LOPOCV iteration produces a θ^* which is employed in the internal CellCNN data augmentation. At each LOPOCV iteration, the configuration θ^* is selected as the one returning the highest F_1-score, according to the algorithm presented in Section 3.8.1. Table 24 and Figure 7 report the optimal θ^* and associated F_1-scores for each LOPOCV iteration.
Regarding Figure 7, the highest F_1-scores tend to cluster toward larger values of both n_{cells} and n_{subs}, suggesting that an extended range of configuration values might achieve further improvements.

	θ^*		
LOPOCV Fold	n_{cells}	n_{subs}	F_1-score
1	200	450	0.9378
2	700	600	0.96
3	700	550	0.9278
4	600	450	0.9278
5	500	300	0.9378
6	300	450	0.91
7	500	600	0.96
8	600	700	0.8778
9	600	600	0.9111
10	400	400	0.96

Table 24: Configurations θ^* for each LOPOCV iteration. The F_1-score column reports the F_1-scores obtained by the corresponding θ^* during BO.

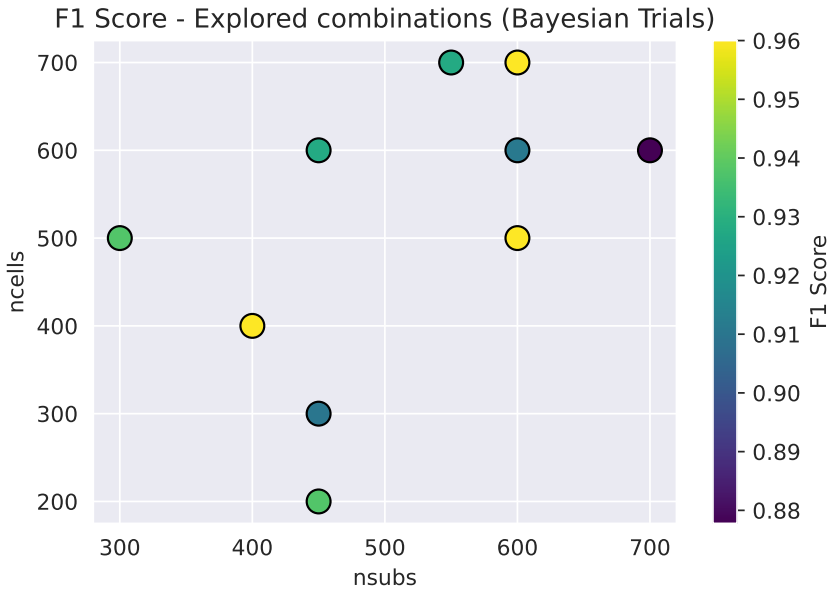


Figure 7: F_1-score achieved by each of the N_{LOPOCV} selected configuration θ^*, colored by F_1-score value.

4.2 Tuned thresholds per LOPOCV fold

The tuned thresholds are reported in Table 25, distinguishing across Training Procedure, presence of AS and threshold tuning procedures. Note that Experiments 1 and 2 are reported together since they share the threshold tuning procedure performed after BO.

Experiment	1-2		3		4	
Training Procedure	1 (AS and NO-AS)		2 (AS)		2 (NO-AS)	
Threshold / LOPO Fold	τ_{ROC}^{*BO}	τ_{RES}^{*BO}	τ_{ROC}^{*}	τ_{RES}^{*}	τ_{ROC}^{*}	τ_{RES}^{*}
1	0.580919	0.50	0.589529	0.78	0.470267	0.46
2	0.649454	0.50	0.635230	0.58	0.468126	0.50
3	0.744908	0.74	0.731488	0.77	0.421762	0.44
4	0.644309	0.65	0.739984	0.74	0.428423	0.41
5	0.681786	0.50	0.732295	0.67	0.445361	0.44
6	0.672843	0.73	0.667170	0.57	0.466269	0.46
7	0.787222	0.77	0.921140	0.91	0.402345	0.39
8	0.636352	0.61	0.682140	0.65	0.411828	0.40
9	0.674158	0.66	0.716857	0.68	0.455933	0.43
10	0.640859	0.50	0.653649	0.50	0.444638	0.44

Table 25: Tuned thresholds per LOPOCV iteration, divided via Training Procedure. $\tau_{\zeta}^{*(BO)}$ refers to threshold tuned according to resampling approach in post-hoc BO tuning phase, while τ_{ζ}^{*} refers to post-training threshold tuning phase. Note that only Training Procedure 1 employs $\tau_{\zeta}^{*(BO)}$ since they do not perform ensemble training. AS stands for Artificial Samples, while NO-AS stands for without Artificial Samples.

4.3 Per-Experiment Results

4.3.1 Experiment 1: Training Procedure 1 with AS

This experiment is characterized by the generation of AS according to the procedure shown in Section 3.4. The generated AS are then used as training set. Additionally, Training Procedure 1 (Section 3.8.2) is employed and the test samples are classified using the default threshold $\tau = 0.5$, and the two thresholds $\tau_{ROC}^{*(BO)}$ and $\tau_{RES}^{*(BO)}$, which are tuned after BO.
The mean and standard deviation of F_1-score, Recall, Precision and Accuracy are computed across LOPOCV iterations and divided by Seed-Level and Training Procedure, as shown in the Table 26.

Experiment 1 — Training Procedure 1 generating Artificial Samples (Mean ± Std)

	CellCNN Approach		**Resampling Approach**			
	$\tau = 0.5$		$\tau^{*(BO)}_{ROC}$		$\tau^{*(BO)}_{RES}$	
Metric	Seed-Level	Single-Split	Seed-Level	Single-Split	Seed-Level	Single-Split
F_1	**0.7607** $\pm$ 0.2963	0.76 $\pm$ 0.305	0.4067 $\pm$ 0.4469	0.4333 $\pm$ 0.4727	0.58 $\pm$ 0.4595	0.6167 $\pm$ 0.4585
Recall	0.79 $\pm$ 0.3364	**0.8** $\pm$ 0.3496	0.38 $\pm$ 0.4352	0.4 $\pm$ 0.4595	0.56 $\pm$ 0.4589	0.6 $\pm$ 0.4595
Precision	**0.79** $\pm$ 0.3209	0.7833 $\pm$ 0.3338	0.46 $\pm$ 0.4932	0.5 $\pm$ 0.527	0.62 $\pm$ 0.4798	0.65 $\pm$ 0.4743
Accuracy	**0.75** $\pm$ 0.2265	0.75 $\pm$ 0.2257	0.4833 $\pm$ 0.3932	0.5 $\pm$ 0.4157	0.6533 $\pm$ 0.3822	0.6833 $\pm$ 0.3723

Table 26: Experiment 1 results across all classification approaches. *Seed-Level* summarises the $T \times N_{LOPOCV} = 5 \times 10 = 50$ individual Seed-Level predictions. *Single-Split* summarises the $N_{LOPOCV} = 10$ predictions, each obtained by averaging across $T = 5$ seeds. The highest score for each metric is highlighted in bold. "Std" represents the Standard Deviation.

Figures 8, 10, and 12 show the performance distributions for the CellCNN approach ($\tau = 0.5$), Resampling ROC (τ^*_{ROC}), and Resampling RES (τ^*_{RES}), respectively. In all three conditions, both Seed-Level and Single-Split methods show high variability, with Positivity score spanning the full range $[0, 1]$ across LOPOCV iterations. Note that several LOPOCV iterations produced performance pretrics scores of 0.0, corresponding to folds in which the model failed to correctly classify any positive sample.

The CellCNN approach ($\tau = 0.5$) achieves the highest mean performance across all metrics, with $F_1 = 0.76$ and Recall = 0.79-0.80. The $\tau^{*(BO)}_{RES}$ approach shows intermediate performance ($F_1 = 0.58$-0.62), while $\tau^{*(BO)}_{ROC}$ obtained the lowest means across all metrics ($F_1 \approx 0.41$-0.43). Seed-Level and Single-Split performance metrics scores can be compared across all three approaches, showing differences in mean below 4%. The limited variability across samples predictions is further visible in Figures 9, 11 and 13, which show the Positivity score achieved by each sample across different weights initializations.

Additionally, these figures show the $\tau^{*(BO)}_{ROC}$ and $\tau^{*(BO)}_{RES}$ thresholds respectively at each LOPOCV iteration, providing insight into the variability of the computed metrics. Indeed, although the variability is relatively contained for some samples, the model fails to consistently separate the unhealthy samples from healthy samples, showing Positivity score of healthy samples higher than those of unhealthy. This inability to discriminate is further reflected in the overall performance metrics.

Note that Figures 9, 11 and 13 show the same distribution of Positivity scores for each sample, but the tuned thresholds differ, which are $\tau^{*(BO)}_{ROC}$ and $\tau^{*(BO)}_{RES}$ respectively. The two figures are maintained separated to better distinguish the thresholds and their classification across LOPOCV iterations.

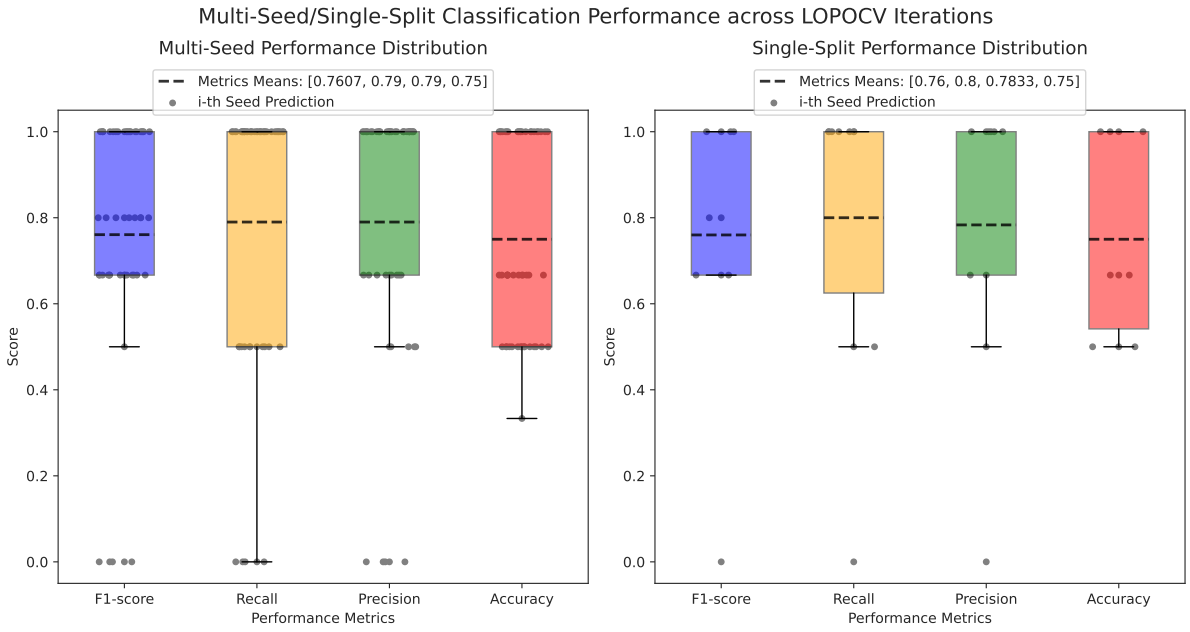


Figure 8: Performance distribution of the CellCNN approach with $\tau = 0.5$ across LOPOCV iterations in AS setting. *Left*: performance distribution of the Seed-Level predictions. *Right*: performance distribution of the Single-Split procedure, which averages predictions across $T = 5$ seeds. The single grey points in the *Left* plots represent the $T \times N_{LOPOCV} = 50$ Seed-Level predictions, while in the *Right* plots, each point represents one of the $N_{LOPOCV} = 10$ seed-averaged predictions. The black dashed lines indicate the mean across all iterations.

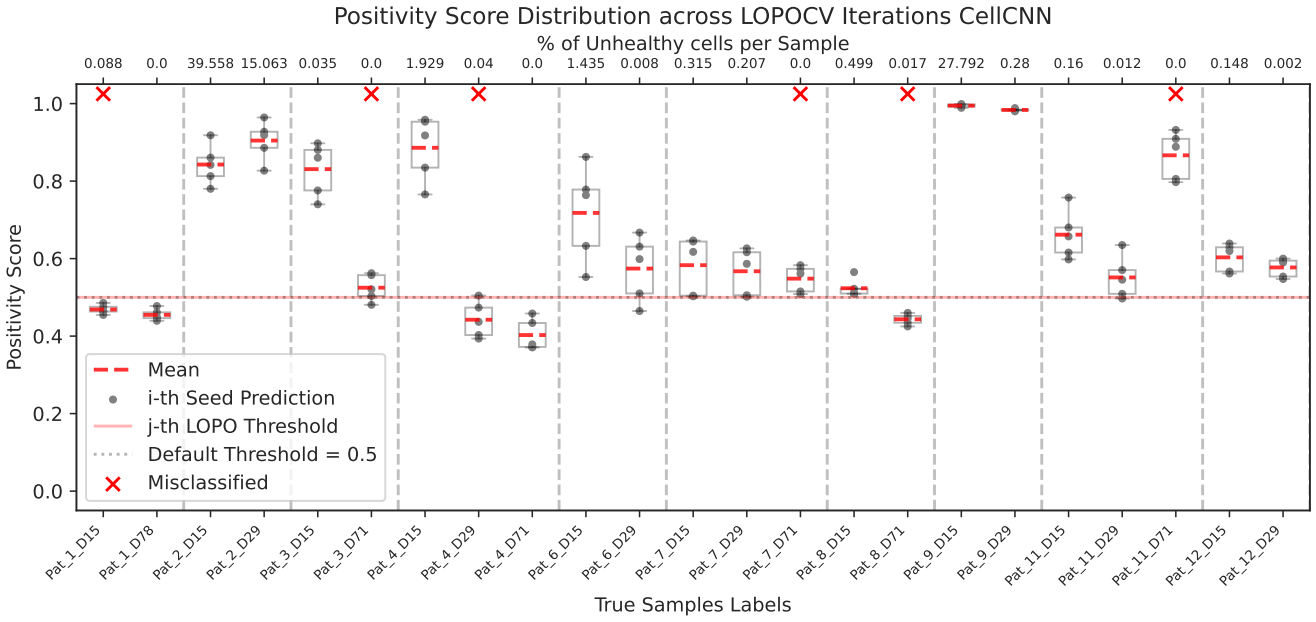


Figure 9: Positivity score distribution per sample across LOPOCV iterations under $\tau = 0.5$ (Experiment 1). Each box summarises the $T = 5$ Seed-Level predictions for that sample. The orange horizontal lines indicate the tuned threshold $\tau = 0.5$ at each LOPOCV iteration. The percentages of unhealthy cells are reported on the top x-axis, and the dotted horizontal line marks the default threshold $\tau = 0.5$. The bottom x-axis shows the samples ID. Misclassified samples are indicated by a red **x** marker at the top of the plot. Finally, the vertical dashed lines separates the LOPOCV iterations. LOPOCV is the abbreviation of Leave-One-Patient-Out Cross Validation.

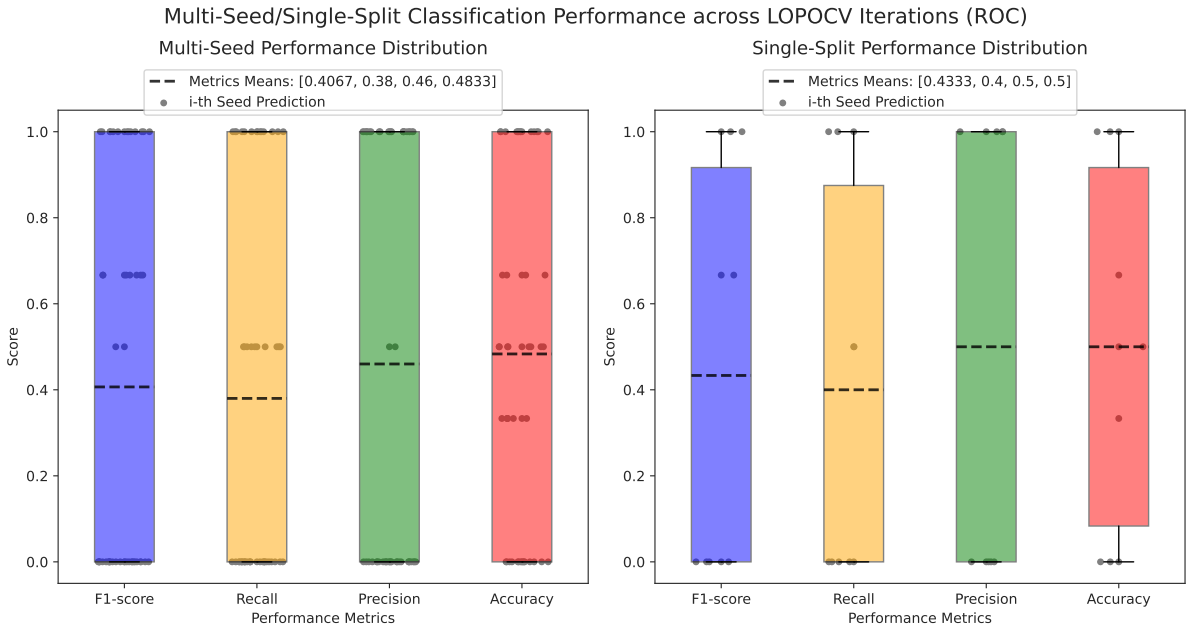


Figure 10: Performance distribution of the Resampling approach with $\tau_{ROC}^{*(BO)}$ across LOPOCV iterations in AS setting. *Left*: performance distribution of the Seed-Level predictions. *Right*: performance distribution of the Single-Split procedure, which averages predictions across $T = 5$ seeds. The single grey points in the *Left* plots represent the $T \times N_{LOPOCV} = 50$ Seed-Level predictions, while in the *Right* plots, each point represents one of the $N_{LOPOCV} = 10$ seed-averaged predictions. The black dashed lines indicate the mean across all iterations.

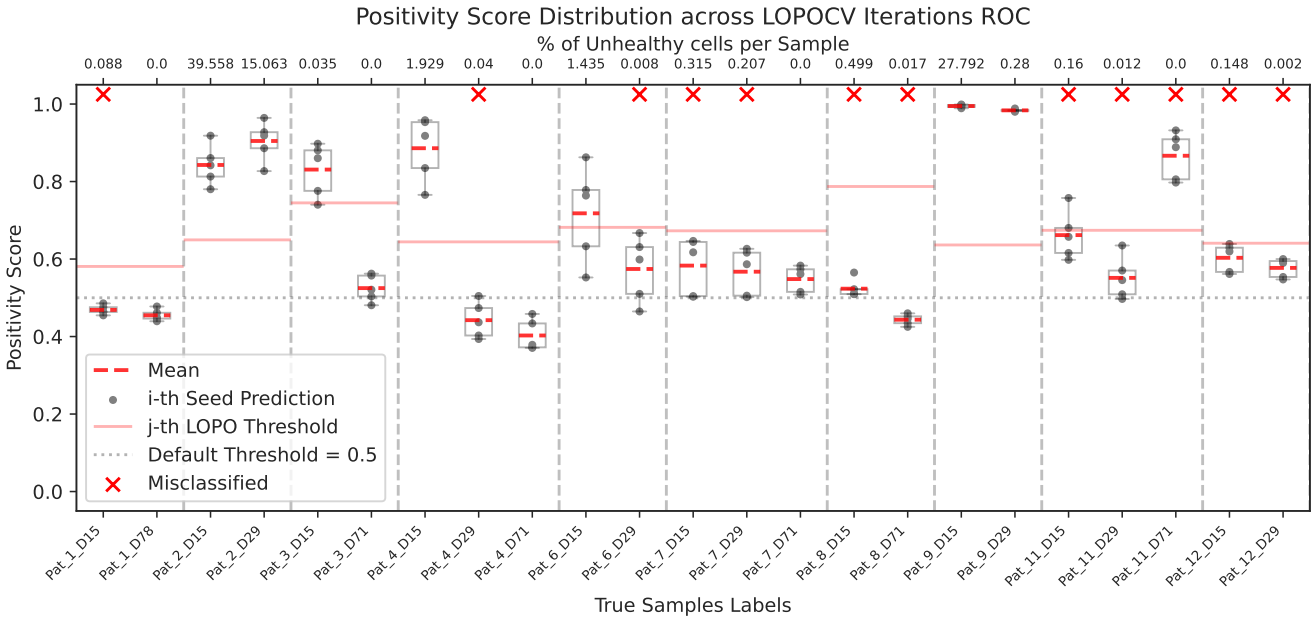


Figure 11: Positivity score distribution per sample across LOPOCV iterations under $\tau_{ROC}^{*(BO)}$ (Experiment 1). The orange horizontal solid lines indicate the tuned threshold $\tau_{ROC}^{*(BO)}$ at each LOPOCV iteration. Each box summarises the $T = 5$ Seed-Level predictions for that sample. The percentages of unhealthy cells are reported on the top x-axis, and the dotted horizontal line marks the default threshold $\tau = 0.5$. The bottom x-axis shows the samples ID. Misclassified samples are indicated by a red **x** marker at the top of the plot. Finally, the vertical dashed lines separates the LOPOCV iterations. LOPOCV is the abbreviation of Leave-One-Patient-Out Cross Validation.

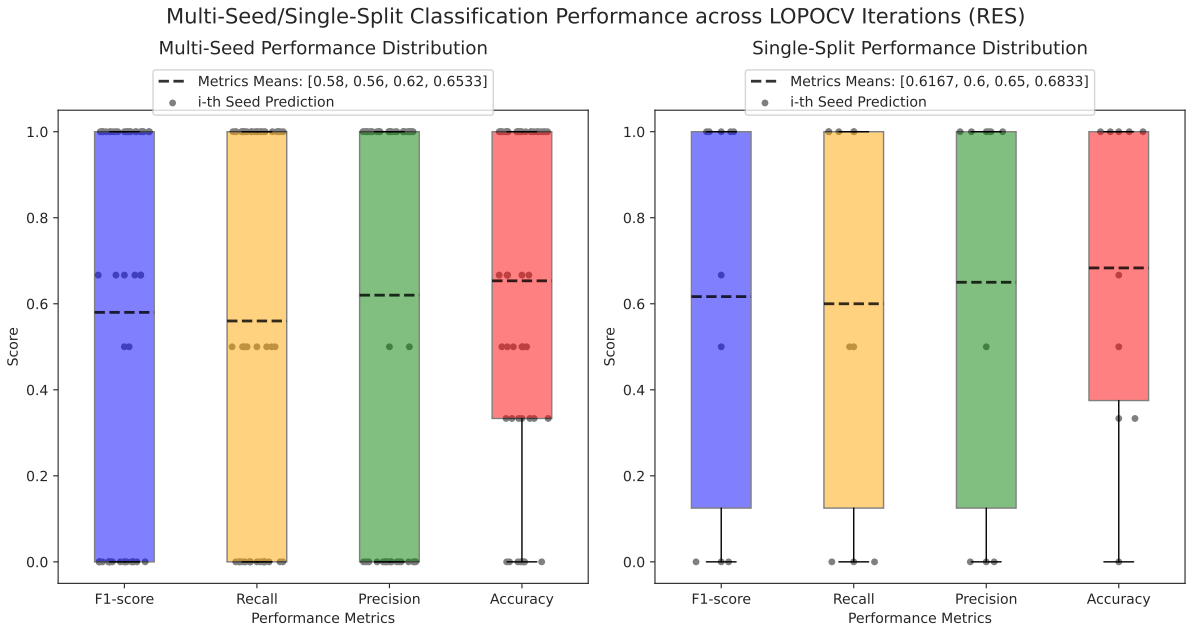


Figure 12: Performance distribution of the Resampling approach with $\tau_{RES}^{*(BO)}$ across LOPOCV iterations in AS setting. *Left*: performance distribution of the Seed-Level predictions. *Right*: performance distribution of the Single-Split procedure, which averages predictions across $T = 5$ seeds. The single grey points in the *Left* plots represent the $T \times N_{LOPOCV} = 50$ Seed-Level predictions, while in the *Right* plots, each point represents one of the $N_{LOPOCV} = 10$ seed-averaged predictions. The black dashed lines indicate the mean across all iterations.

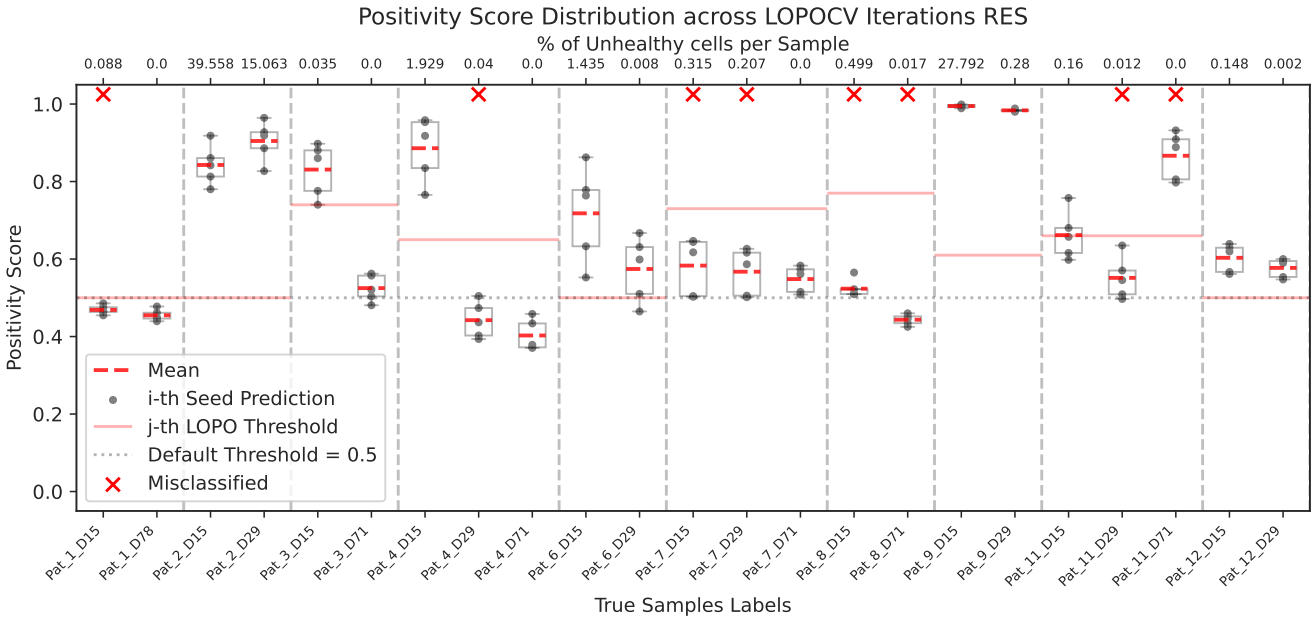


Figure 13: Positivity Score distribution per sample across LOPOCV iterations under $\tau_{RES}^{*(BO)}$ (Experiment 1). The orange horizontal solid lines indicate the tuned threshold $\tau_{RES}^{*(BO)}$ at each LOPOCV iteration. Each box summarises the $T = 5$ Seed-Level predictions for that sample. The percentages of unhealthy cells are reported on the top x-axis, and the dotted horizontal line marks the default threshold $\tau = 0.5$. The bottom x-axis shows the samples ID. Misclassified samples are indicated by a red **x** marker at the top of the plot. Finally, the vertical dashed lines separates the LOPOCV iterations. LOPOCV is the abbreviation of Leave-One-Patient-Out Cross Validation.

4.3.2 Experiment 2: Training Procedure 1 without AS

This experiment uses the patient samples as training set. Additionally, Training Trocedure 1 3.8.2 is employed and the test samples are classified using the default threshold $\tau = 0.5$, and the two thresholds $\tau_{ROC}^{*(BO)}$ and $\tau_{RES}^{*(BO)}$, which are tuned after BO.
The mean and standard deviation of F_1-score, Recall, Precision and Accuracy are computed across LOPOCV iterations and divided by Seed-Level and Training Trocedure, as shown in the Table 27.

Experiment 2 — Training Procedure 1 without Artificial Samples (Mean ± Std)

	CellCNN Approach:		**Resampling Approach:**			
	$\tau = 0.5$		$\tau_{ROC}^{*(BO)}$		$\tau_{RES}^{*(BO)}$	
Metric	Seed-Level	Single-Split	Seed-Level	Single-Split	Seed-Level	Single-Split
F_1	0.8813 ± 0.1375	0.8733 ± 0.1421	$\mathbf{0.9267} \pm 0.1184$	$\mathbf{0.9267} \pm 0.1235$	$\mathbf{0.9267} \pm 0.1184$	$\mathbf{0.9267} \pm 0.1235$
Recall	$\mathbf{0.95} \pm 0.1515$	$\mathbf{0.95} \pm 0.1581$	$\mathbf{0.95} \pm 0.1515$	$\mathbf{0.95} \pm 0.1581$	$\mathbf{0.95} \pm 0.1515$	$\mathbf{0.95} \pm 0.1581$
Precision	0.8633 ± 0.1896	0.85 ± 0.1995	$\mathbf{0.9333} \pm 0.1347$	$\mathbf{0.9333} \pm 0.1405$	$\mathbf{0.9333} \pm 0.1347$	$\mathbf{0.9333} \pm 0.1405$
Accuracy	0.8133 ± 0.212	0.8 ± 0.2194	$\mathbf{0.8833} \pm 0.1852$	$\mathbf{0.8833} \pm 0.1933$	$\mathbf{0.8833} \pm 0.1852$	$\mathbf{0.8833} \pm 0.1933$

Table 27: Experiment 2 results across all classification approaches. *Seed-Level* summarises the $T \times N_{LOPOCV} = 5 \times 10 = 50$ individual Seed-Level predictions. *Single-Split* summarises the $N_{LOPOCV} = 10$ ensemble predictions, each obtained by averaging across $T = 5$ seeds. The highest Positivity score for each metric is highlighted in bold. "Std" represents the Standard Deviation.

Figures 14, 16, and 18 show the performance distributions for the CellCNN approach ($\tau = 0.5$), Resampling ROC ($\tau_{ROC}^{*(BO)}$), and Resampling RES ($\tau_{RES}^{*(BO)}$), respectively, trained and evaluated on patient samples.
Overall, Experiment 2 shows higher mean performance compared to Experiment 1 across all metrics and approaches and variability across performance metrics and Positivity score is reduced compared to Experiment 1.
The CellCNN approach achieves $F_1 = 0.88$ and Recall $= 0.95$, while both $\tau_{ROC}^{*(BO)}$ and $\tau_{RES}^{*(BO)}$ reach $F_1 = 0.9267$ and Recall $= 0.95$, with identical results across both threshold strategies. Indeed, in this experiment the Positivity score computed per sample do not show high variability across the fold predictions, as it can be seen in Figures 15, 17 and 19. Additionally, the tuned thresholds $\tau_{ROC}^{*(BO)}$ and $\tau_{RES}^{*(BO)}$ show both similar values, well separating the unhealthy and healthy classes, except for the samples f_3^7 and f_2^8 which are misclassified by both. As a result, the ROC-tuned and RES-tuned approaches achieved identical classifications for each sample, producing equal performance metrics for both approaches.

Finally, Seed-Level and Single-Split Recall scores are equivalent across all three approaches, and the other metrics scored similar std values, further confirming the limited impact of weight initialization on model performance.

Note that Figures 15, 17 and 19 show the same distribution of Positivity score for each sample, but the tuned thresholds differ, which are $\tau_{ROC}^{*(BO)}$ and $\tau_{RES}^{*(BO)}$ respectively. The two figures are maintained separated to better distinguish the thresholds and their classification across LOPOCV iterations.

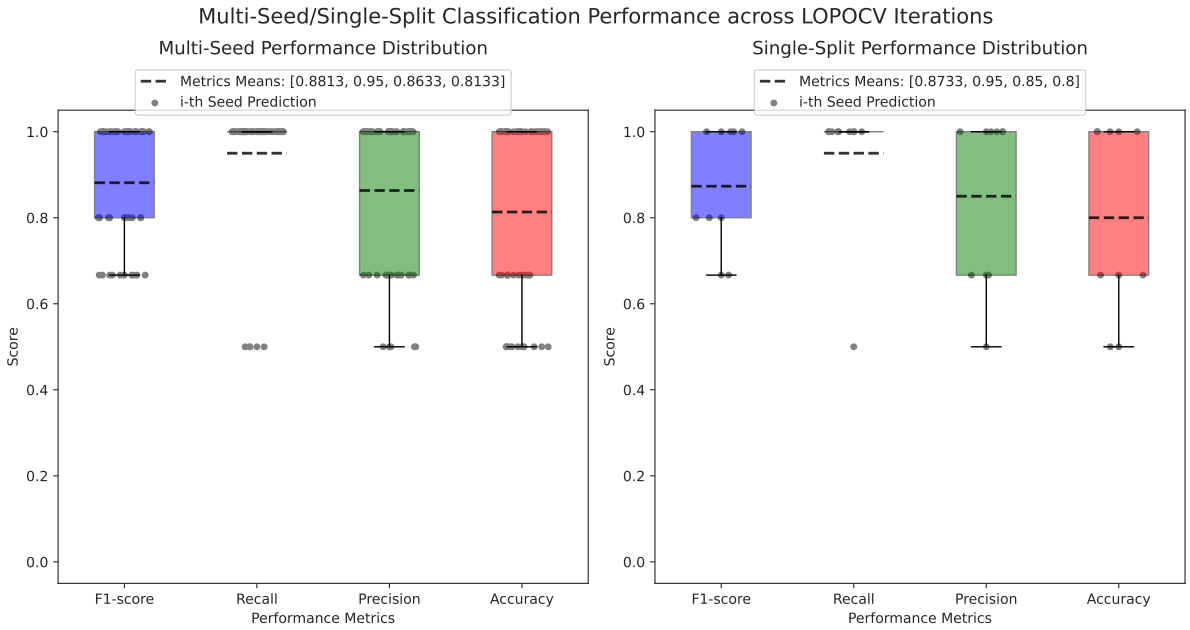


Figure 14: Performance distribution of the CellCNN approach with $\tau = 0.5$ across LOPOCV iterations in setting without AS. *Left*: performance distribution of the Seed-Level predictions. *Right*: performance distribution of the Single-Split procedure, which averages predictions across $T = 5$ seeds. The single grey points in the *Left* plots represent the $T \times N_{LOPOCV} = 50$ Seed-Level predictions, while in the *Right* plots, each point represents one of the $N_{LOPOCV} = 10$ seed-averaged predictions. The black dashed lines indicate the mean across all iterations.

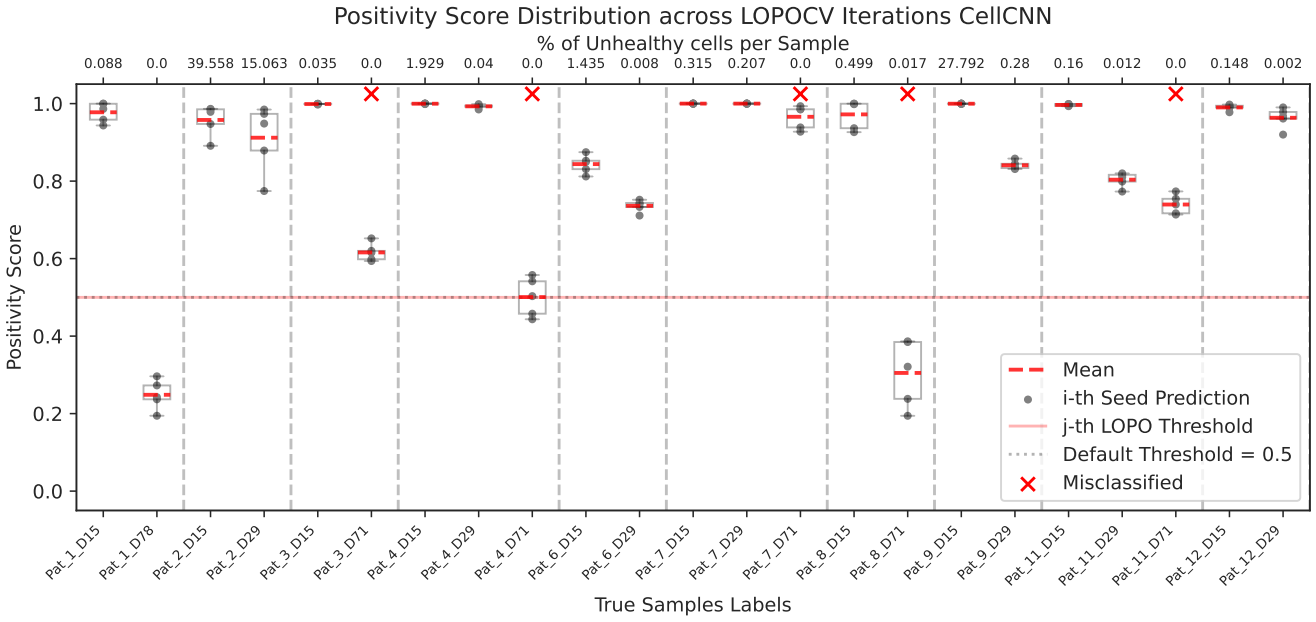


Figure 15: Positivity Score distribution per sample across LOPOCV iterations under $\tau = 0.5$ (Experiment 2). Each box summarises the $T = 5$ Seed-Level predictions for that sample. The orange horizontal lines indicate the tuned threshold $\tau = 0.5$ at each LOPOCV iteration. The percentages of unhealthy cells are reported on the top x-axis, and the dotted horizontal line marks the default threshold $\tau = 0.5$. The bottom x-axis shows the samples ID. Misclassified samples are indicated by a red **x** marker at the top of the plot. Finally, the vertical dashed lines separates the LOPOCV iterations. LOPOCV is the abbreviation of Leave-One-Patient-Out Cross Validation.

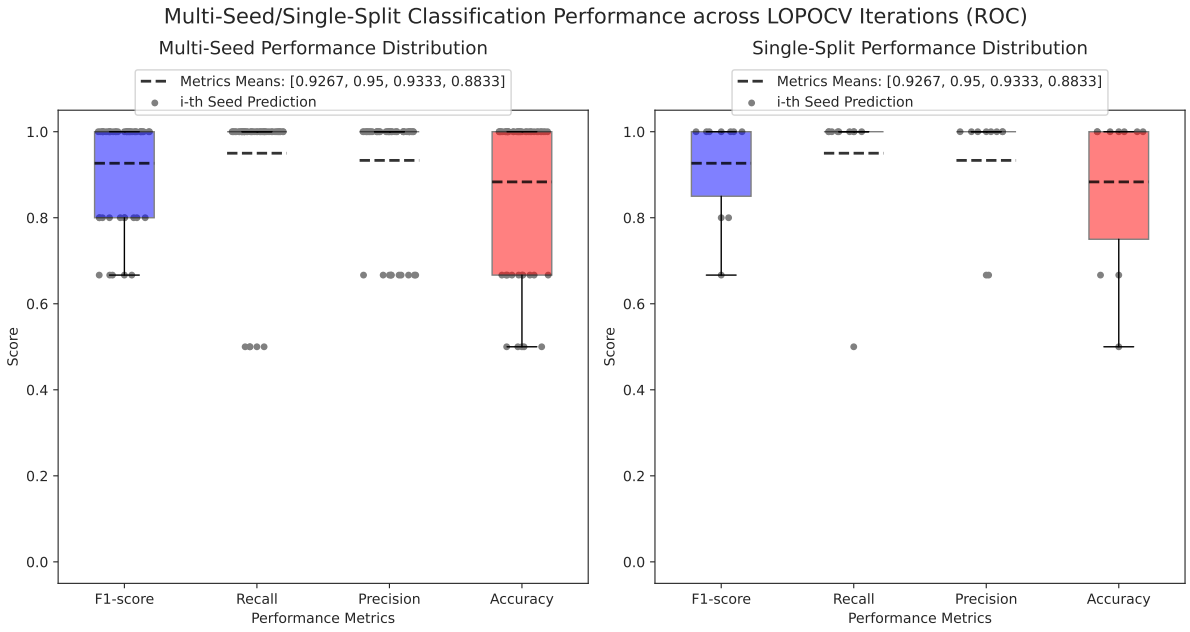


Figure 16: Performance distribution of the Resampling approach with $\tau^{*(BO)}_{ROC}$ across LOPOCV iterations in setting without AS. *Left*: performance distribution of the Seed-Level predictions. *Right*: performance distribution of the Single-Split procedure, which averages predictions across $T = 5$ seeds. The single grey points in the *Left* plots represent the $T \times N_{LOPOCV} = 50$ Seed-Level predictions, while in the *Right* plots, each point represents one of the $N_{LOPOCV} = 10$ seed-averaged predictions. The black dashed lines indicate the mean across all iterations.

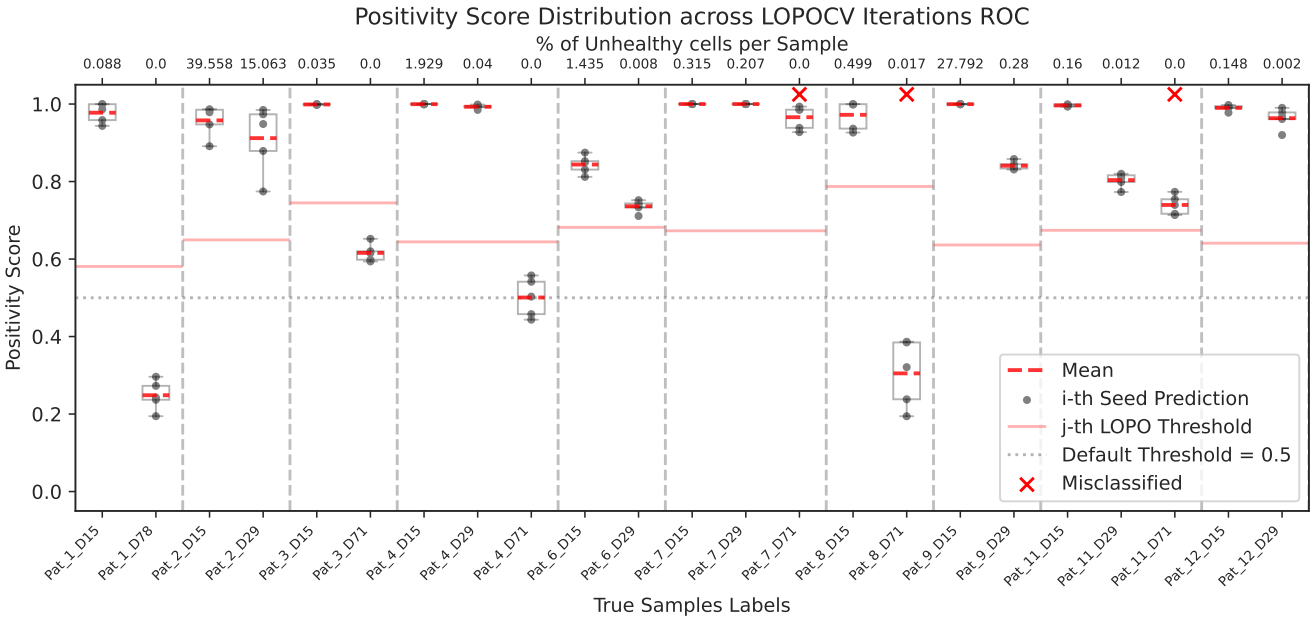


Figure 17: Positivity score distribution per sample across LOPOCV iterations under $\tau^{*(BO)}_{ROC}$ (Experiment 2). The orange horizontal solid lines indicate the tuned threshold $\tau^{*(BO)}_{RES}$ at each LOPOCV iteration. Each box summarises the $T = 5$ Seed-Level predictions for that sample. The percentages of unhealthy cells are reported on the top x-axis, and the dotted horizontal line marks the default threshold $\tau = 0.5$. The bottom x-axis shows the samples ID. Misclassified samples are indicated by a red **x** marker at the top of the plot. Finally, the vertical dashed lines separates the LOPOCV iterations. LOPOCV is the abbreviation of Leave-One-Patient-Out Cross Validation.

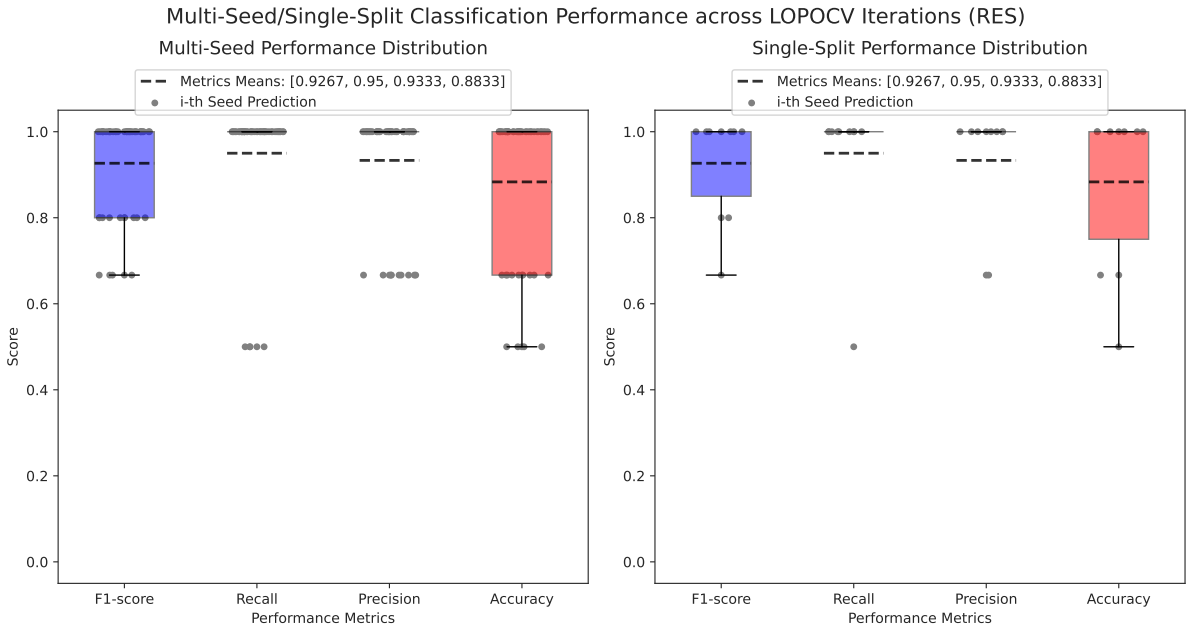


Figure 18: Performance distribution of the Resampling approach with $\tau_{RES}^{*(BO)}$ across LOPOCV iterations in setting without AS. *Left*: performance distribution of the Seed-Level predictions. *Right*: performance distribution of the Single-Split procedure, which averages predictions across $T = 5$ seeds. The single grey points in the *Left* plots represent the $T \times N_{LOPOCV} = 50$ Seed-Level predictions, while in the *Right* plots, each point represents one of the $N_{LOPOCV} = 10$ seed-averaged predictions. The black dashed lines indicate the mean across all iterations.

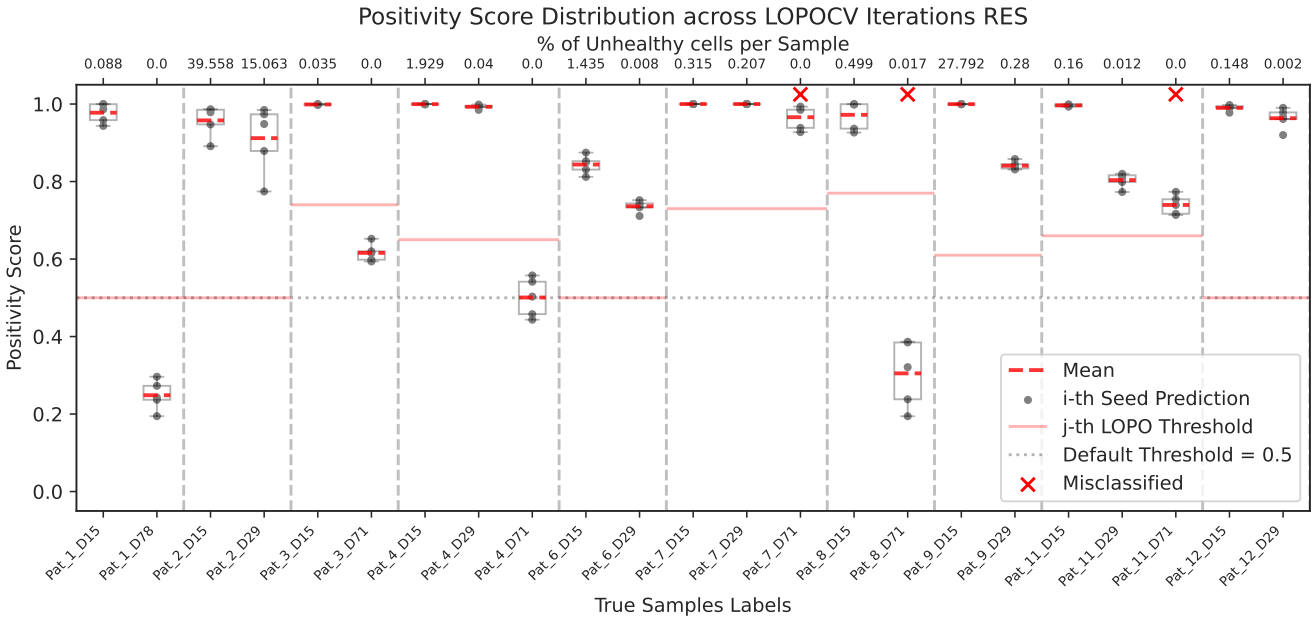


Figure 19: Positivity score distribution per sample across LOPOCV iterations under $\tau_{RES}^{*(BO)}$ (Experiment 2). The orange horizontal solid lines indicate the tuned threshold $\tau_{RES}^{*(BO)}$ at each LOPOCV iteration. Each box summarises the $T = 5$ Seed-Level predictions for that sample. The percentages of unhealthy cells are reported on the top x-axis, and the dotted horizontal line marks the default threshold $\tau = 0.5$. The bottom x-axis shows the samples ID. Misclassified samples are indicated by a red **x** marker at the top of the plot. Finally, the vertical dashed lines separates the LOPOCV iterations. LOPOCV is the abbreviation of Leave-One-Patient-Out Cross Validation.

4.3.3 Experiment 3: Training Procedure 2 with AS

In this experiment the training set is composed by the generated AS and the models are trained using Training Procedure 2 (Section 3.8.3). The test samples are classified using the default threshold $\tau = 0.5$, and the two thresholds τ^*_{ROC} and τ^*_{RES}, tuned after the training. The mean and standard deviation of F_1-score, Recall, Precision and Accuracy are computed across LOPOCV iterations and divided by Fold-Level and Training Procedure, as shown in the Table 28.

Experiment 3 — Training Procedure 2 with Artificial Samples (Mean ± Std)

	CellCNN Approach		**Resampling Approach**			
	$\tau = 0.5$		τ^*_{ROC}		τ^*_{RES}	
Metric	Fold-Level	Ensemble	Fold-Level	Ensemble	Fold-Level	Ensemble
F_1	0.7 ± 0.3651	0.8933 ± 0.1447	0.818 ± 0.2574	$\mathbf{0.8933} \pm 0.1447$	0.8833 ± 0.1485	0.8733 ± 0.1421
Recall	0.73 ± 0.394	$\mathbf{1.0} \pm 0.0$	0.88 ± 0.277	$\mathbf{1.0} \pm 0.0$	0.96 ± 0.137	$\mathbf{1.0} \pm 0.0$
Precision	0.7167 ± 0.3858	0.8333 ± 0.2222	0.8067 ± 0.2864	0.8333 ± 0.2222	$\mathbf{0.8533} \pm 0.2038$	0.8 ± 0.2194
Accuracy	0.7133 ± 0.2878	$\mathbf{0.8333} \pm 0.2222$	0.78 ± 0.253	$\mathbf{0.8333} \pm 0.2222$	0.82 ± 0.2229	0.8 ± 0.2194

Table 28: Experiment 3 results across all classification approaches. *Fold-Level* summarises the $K \times N_{LOPOCV} = 5 \times 10 = 50$ individual Fold-Level predictions. *Ensemble* summarises the $N_{LOPOCV} = 10$ ensemble predictions, each obtained by averaging across $K = 5$ folds. The highest score for each metric is highlighted in bold. "Std" represents the Standard Deviation.

Figures 20, 22, and 24 show the performance distributions for the CellCNN approach ($\tau = 0.5$), Resampling ROC (τ^*_{ROC}), and Resampling RES (τ^*_{RES}), respectively, trained on AS using Training Procedure 2.
The most notable result of Experiment 3 is the improvement introduced by the Fold-Ensemble strategy compared to the Fold-Level. The CellCNN approach achieves a mean $F_1 = 0.70$ at the Fold-Level, which increases to $F_1 = 0.8933$ after ensemble aggregation, corresponding to an improvement of $\approx 19\%$. A similar pattern is observed for τ^*_{ROC} ($F_1 : 0.818 \rightarrow 0.8933$), with a more modest improvement. For τ^*_{RES}, a slight decrease in mean F_1, Precision, and Accuracy is observed after aggregation, linked to a general reduction in metric standard deviation (see Figure 24). This suggests that fold averaging introduces more stability in the predictions, but also additional false positives in this setting. In all three classification approaches, Recall reaches 1.0 after ensemble aggregation, indicating that all positive samples are correctly identified. However, looking at the other performance metrics, it is evident that the perfect Recall has been achieved because of the low thresholds values, clsssifying almost all samples as unhealthy.

The Fold-Level distributions remain highly variable, with Positivity score spanning $[0, 1]$ and several iterations producing performance metrics scores of 0.0, consistent with the behaviour observed in Experiments 1 and 2. In contrast, the Ensemble distributions for the CellCNN and ROC-resampling approaches show a marked reduction in performance metrics variability, with Ensemble scores concentrated in the upper range $[0.8, 1.0]$, as shown in Figure 20 and Figure 22.

The variability of Positivity score per samples remains generally high as it can be seen in Figures 21, 23 and 25, while the tuned thresholds τ^*_{ROc} and τ^*_{RES} tend to shift closer to the 0.5 value, failing in clearly separating the unhealthy and healthy classes, as shown in Figure 23 and Figure 25 respectively. This reflects the improvements in Recall = 1 and decrease in variability of performance metrics, but also suggests the inability of the model to recognize

the healthy class.
Note that Figures 23 and 25 show the same distribution of Positivity score for each sample, but the tuned thresholds differ, which are τ^*_{ROC} and τ^*_{RES} respectively. The two figures are maintained separated to better distinguish the thresholds and their classification across LOPOCV iterations.

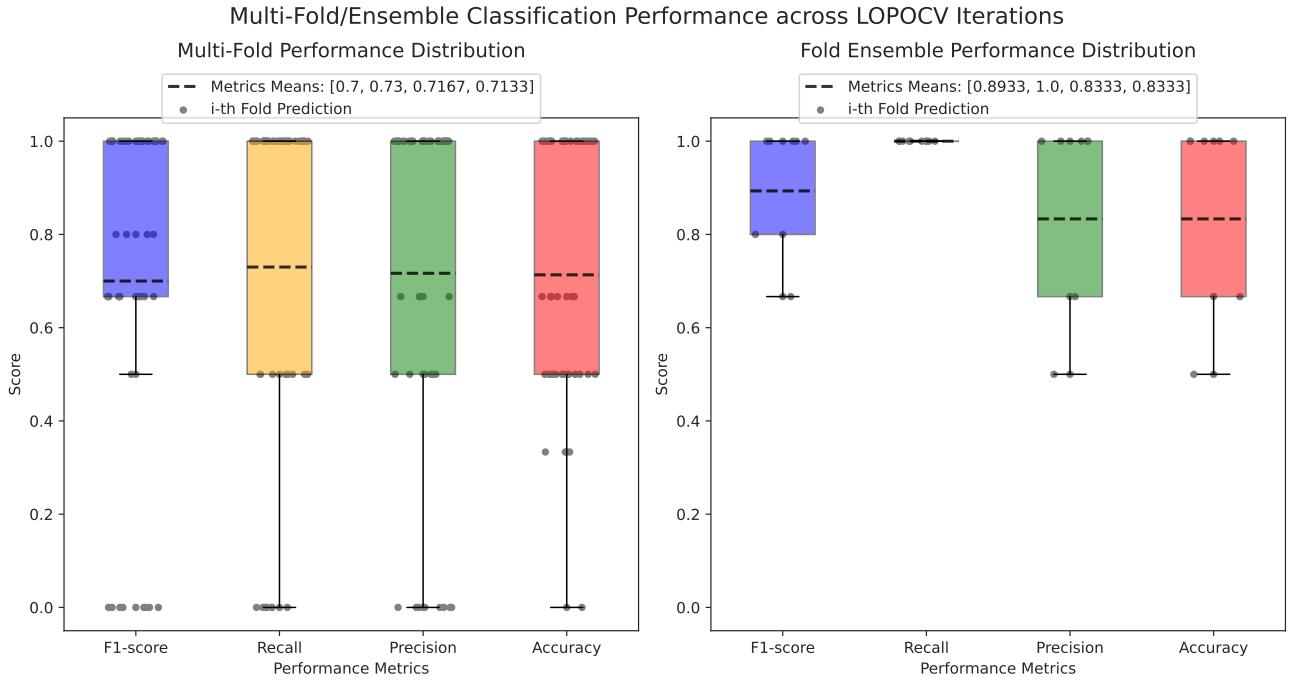


Figure 20: Performance distribution of the CellCNN approach ($\tau = 0.5$) across LOPOCV iterations (Experiment 3). *Left*: performance distribution of the Fold-Level predictions. *Right*: performance distribution of the Ensemble procedure, which averages predictions across $K = 5$ folds. The single grey points in the *Left* plots represent the $K \times N_{LOPOCV} = 50$ Fold-Level predictions, while in the *Right* plots, each point represents one of the $N_{LOPOCV} = 10$ fold-averaged predictions. The black dashed lines indicate the mean across all iterations.

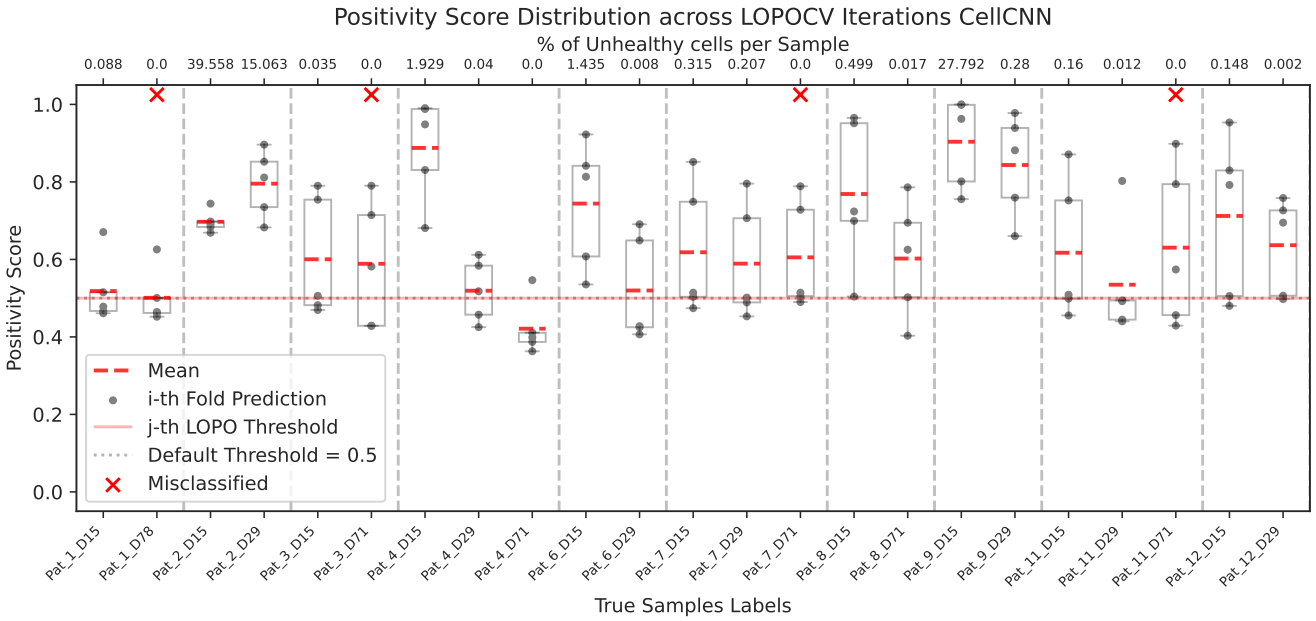


Figure 21: Positivity score distribution per sample across LOPOCV iterations under $\tau = 0.5$ (Experiment 3). Each box summarises the $K = 5$ Fold-Level predictions for that sample. The orange horizontal lines indicate the tuned threshold $\tau = 0.5$ at each LOPOCV iteration. Each box summarises the $K = 5$ Fold-Level predictions for that sample. The percentages of unhealthy cells are reported on the top x-axis, and the dotted horizontal line marks the default threshold $\tau = 0.5$. The bottom x-axis shows the samples ID. Misclassified samples are indicated by a red **x** marker at the top of the plot. Finally, the vertical dashed lines separates the LOPOCV iterations. LOPOCV is the abbreviation of Leave-One-Patient-Out Cross Validation.

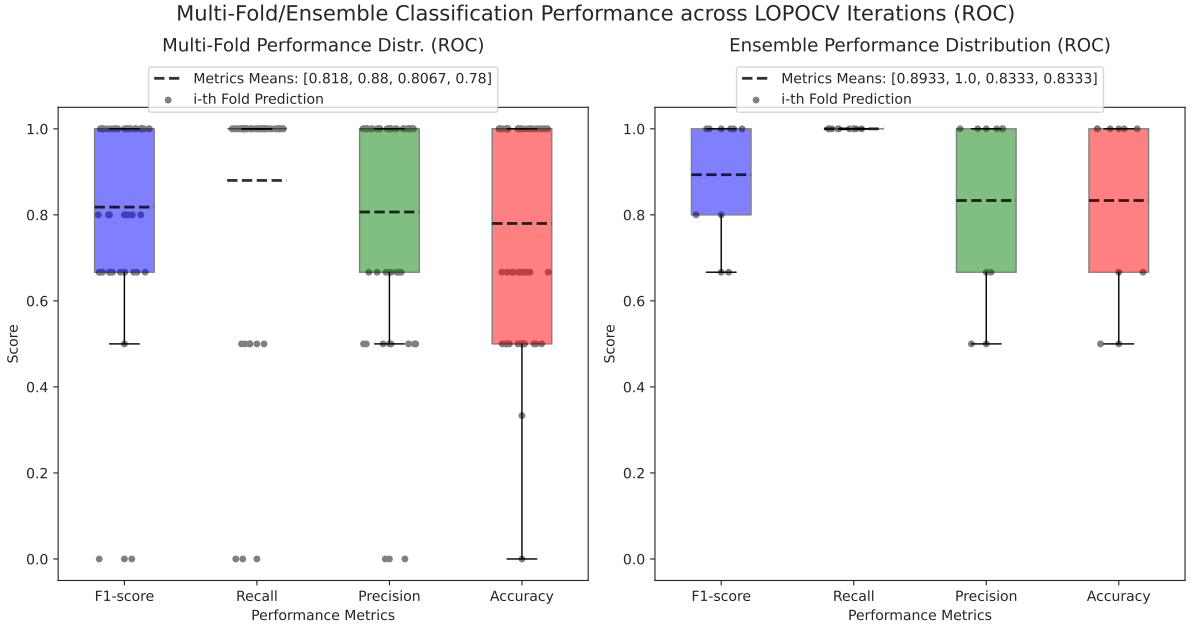


Figure 22: Performance distribution of the Resampling approach with τ^*_{ROC} across LOPOCV iterations (Experiment 3). *Left*: performance distribution of the Fold-Level predictions. *Right*: performance distribution of the Ensemble procedure, which averages predictions across $K = 5$ folds. The single grey points in the *Left* plots represent the $K \times N_{LOPOCV} = 50$ Fold-Level predictions, while in the *Right* plots, each point represents one of the $N_{LOPOCV} = 10$ fold-averaged predictions. The black dashed lines indicate the mean across all iterations.

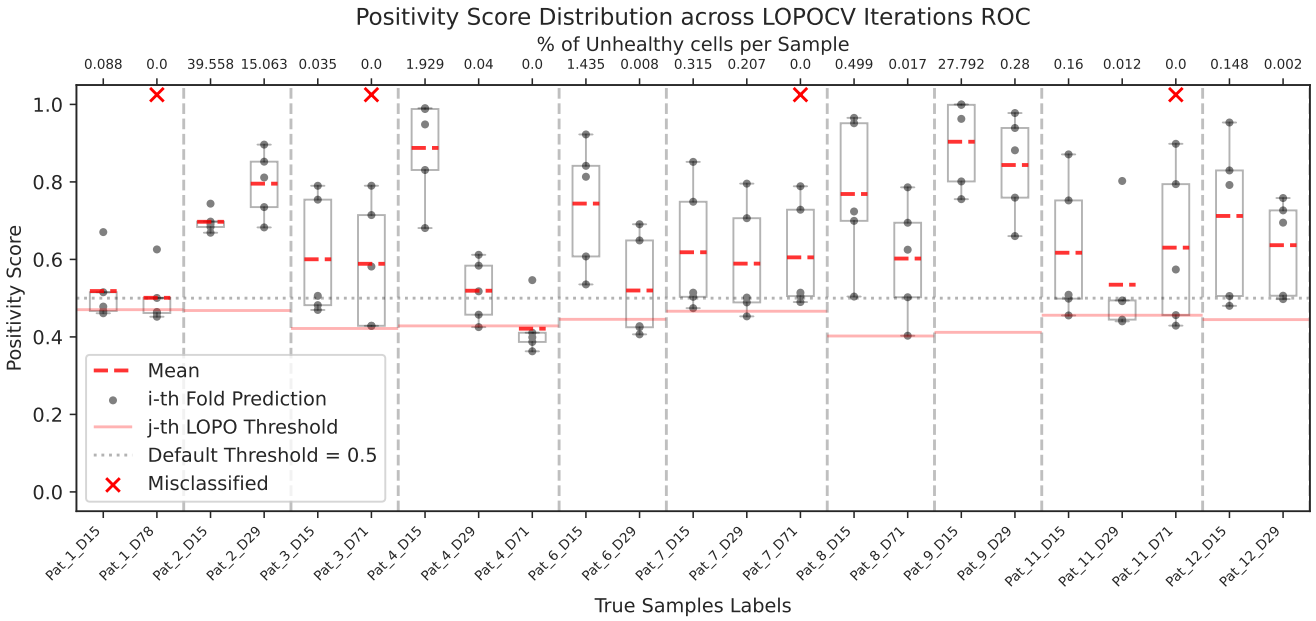


Figure 23: Positivity score distribution per sample across LOPOCV iterations under τ^*_{ROC} (Experiment 3). Each box summarises the $K = 5$ Fold-Level predictions for that sample. The orange horizontal lines indicate the tuned threshold τ^*_{ROC} at each LOPOCV iteration. The percentages of unhealthy cells are reported on the top x-axis, and the dotted horizontal line marks the default threshold $\tau = 0.5$. The bottom x-axis shows the samples ID. Misclassified samples are indicated by a red **x** marker at the top of the plot. Finally, the vertical dashed lines separates the LOPOCV iterations. LOPOCV is the abbreviation of Leave-One-Patient-Out Cross Validation.

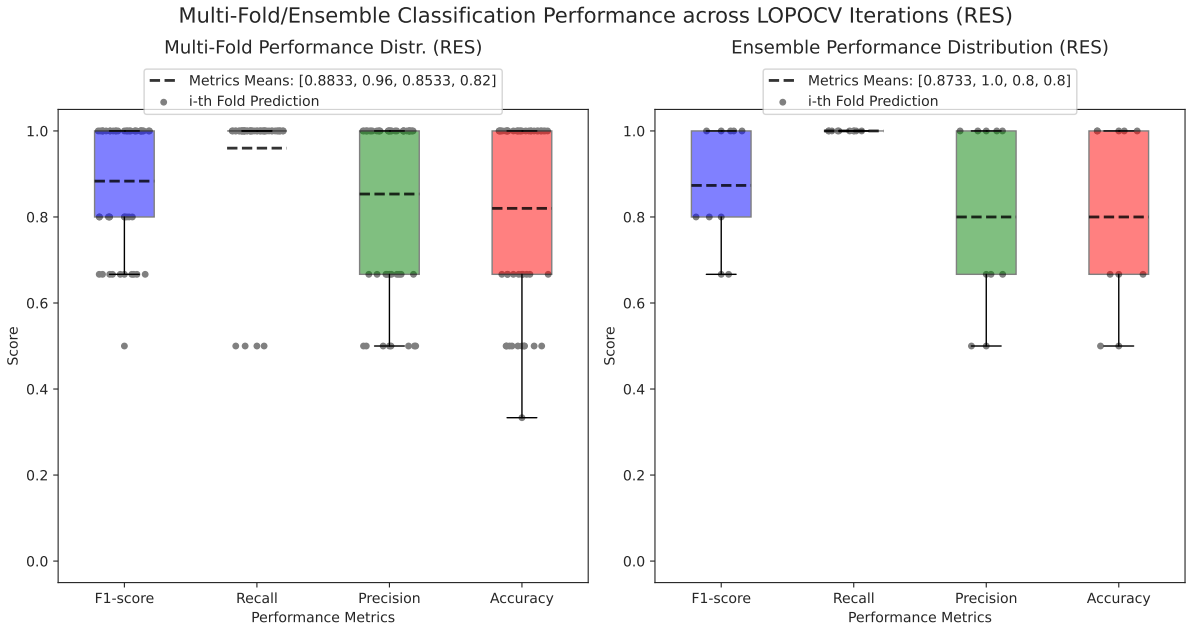


Figure 24: Performance distribution of the Resampling approach with τ^*_{RES} across LOPOCV iterations (Experiment 3). *Left*: performance distribution of the Fold-Level predictions. *Right*: performance distribution of the Ensemble procedure, which averages predictions across $K = 5$ folds. The single grey points in the *Left* plots represent the $K \times N_{LOPOCV} = 50$ Fold-Level predictions, while in the *Right* plots, each point represents one of the $N_{LOPOCV} = 10$ fold-averaged predictions. The black dashed lines indicate the mean across all iterations.

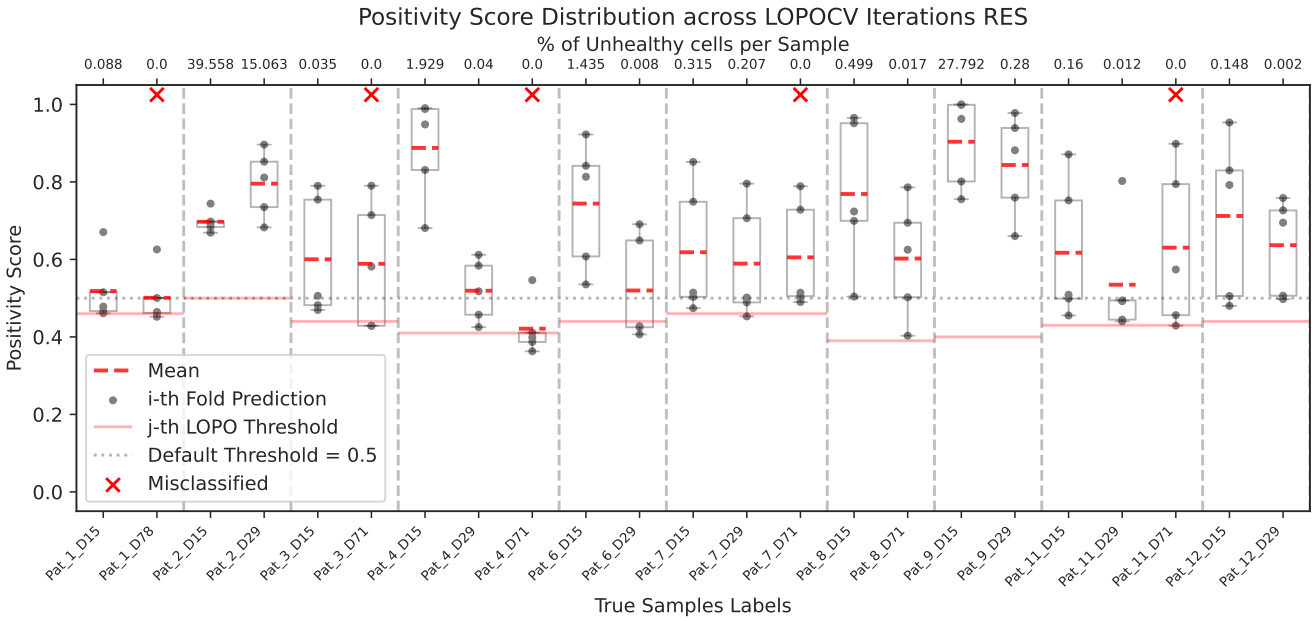


Figure 25: Positivity score distribution per sample across LOPOCV iterations under τ^*_{RES} (Experiment 3). Each box summarises the $K = 5$ Fold-Level predictions for that sample. The orange horizontal lines indicate the tuned threshold τ^*_{RES} at each LOPOCV iteration. The percentages of unhealthy cells are reported on the top x-axis, and the dotted horizontal line marks the default threshold $\tau = 0.5$. The bottom x-axis shows the samples ID. Misclassified samples are indicated by a red **x** marker at the top of the plot. Finally, the vertical dashed lines separates the LOPOCV iterations. LOPOCV is the abbreviation of Leave-One-Patient-Out Cross Validation.

4.3.4 Experiment 4: Training Procedure 2 without AS

In this Experiment the models are trained using patient samples and Training Procedure 2 (Section 3.8.3). The test samples are classified using the default threshold $\tau = 0.5$, and the two thresholds τ^*_{ROC} and τ^*_{RES}, tuned after the training.
The mean and standard deviation of F_1-score, Recall, Precision and Accuracy are computed across LOPOCV iterations and divided by Fold-Level and Training Procedure, as shown in the Table 29.

Experiment 4 — without Artificial Samples (Mean ± Std)

	CellCNN Approach		**Resampling Approach**			
	$\tau = 0.5$		τ^*_{ROC}		τ^*_{RES}	
Metric	Fold-Level	Ensemble	Fold-Level	Ensemble	Fold-Level	Ensemble
F_1	0.8813 ± 0.1375	0.84 ± 0.1481	0.8667 ± 0.1952	0.88 ± 0.1596	0.896 ± 0.1862	$\mathbf{0.9467} \pm 0.1167$
Recall	$\mathbf{0.96} \pm 0.137$	0.95 ± 0.1581	0.88 ± 0.2382	0.9 ± 0.2108	0.9 ± 0.2259	0.95 ± 0.1581
Precision	0.8533 ± 0.1953	0.8 ± 0.2194	0.9067 ± 0.208	0.9167 ± 0.18	0.93 ± 0.1847	$\mathbf{0.9667} \pm 0.1054$
Accuracy	0.8167 ± 0.2082	0.75 ± 0.2257	0.8167 ± 0.2457	0.8333 ± 0.2222	0.8567 ± 0.2309	$\mathbf{0.9167} \pm 0.18$

Table 29: Experiment 4 results across all classification approaches. *Fold-Level* summarises the $K \times N_{LOPOCV} = 5 \times 10 = 50$ individual Fold-Level predictions. *Ensemble* summarises the $N_{LOPOCV} = 10$ ensemble predictions, each obtained by averaging across $K = 5$ folds. The highest performance metrics score for each metric is highlighted in bold. "Std" represents the Standard Deviation.

Figures 26, 28, and 30 show the Performance distributions for the CellCNN approach ($\tau = 0.5$), Resampling ROC (τ^*_{ROC}), and Resampling RES (τ^*_{RES}), respectively, trained on patient samples using Training Procedure 2.

The CellCNN approach achieves a Fold-Level mean $F_1 = 0.8813$, which slightly decreases after ensemble aggregation to $F_1 = 0.84$. All other performance metrics decrease as well. This decrease is due to the correct classification achieved by some individual folds of samples that show high variability in Positivity score, as the sample f_2^8 (see Figure 31). A different behaviour is observed for the Resampling approaches, where the ensemble consistently improves performance: τ^*_{ROC} increases from $F_1 = 0.8667$ to $F_1 = 0.88$, and τ^*_{RES} from $F_1 = 0.896$ to $F_1 = 0.9467$, the highest F_1 value observed across all four Experiments.
The τ^*_{RES} ensemble also achieves the best overall Precision (0.9667) and Accuracy (0.9167) across all four Experiments.

Fold-Level variability is reduced relative to Experiments 1 and 3, with only one iteration producing a score of 0.0 across all approaches. This result is due to the misclassification of an unhealthy and mathematical formulation of F_1-score (see Section 3.5), which produced an F_1-score $= 0$ even if the other sample in the test set was correctly classified as unhealthy. As shown in Figures 27, 29 and 31, there is a relatively high variability across single fold predictions. Different is the case of the ensemble Positivity score, whose distribution is generally concentrated in the upper part, with very few outliers, indicating a relatively stable and consistent classification across LOPOCV iterations. Additionally, τ^*_ζ is generally higher than the 0.5 level, correctly classifying most of the samples.

Note that Figures 27, 29 and 31 show the same distribution of Positivity score for each sample, but the tuned thresholds differ, which are τ^*_{ROC} and τ^*_{RES} respectively. The two figures are maintained separated to better distinguish the thresholds and their classification across LOPOCV iterations.

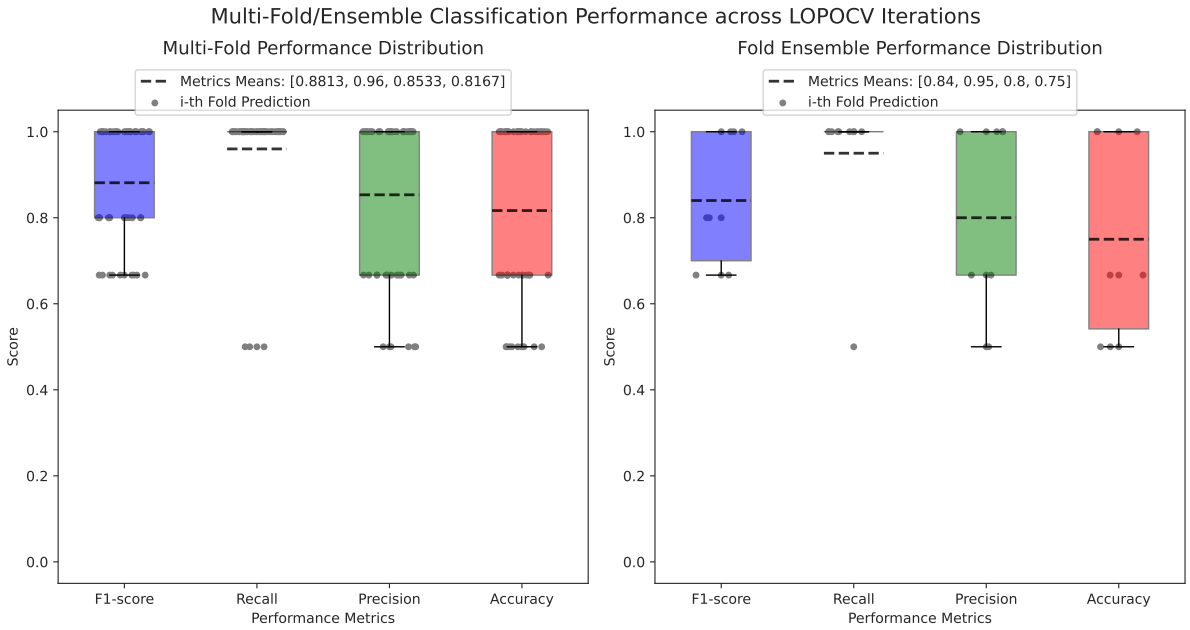


Figure 26: Performance distribution of the CellCNN approach ($\tau = 0.5$) across LOPOCV iterations (Experiment 4). *Left*: Fold-Level distribution ($K \times N_{LOPOCV} = 50$ values). *Right*: Ensemble distribution ($N_{LOPOCV} = 10$ values). The single grey points in the *Left* plots represent the $K \times N_{LOPOCV} = 50$ Fold-Level predictions, while in the *Right* plots, each point represents one of the $N_{LOPOCV} = 10$ fold-averaged predictions. The black dashed lines indicate the mean across all iterations.

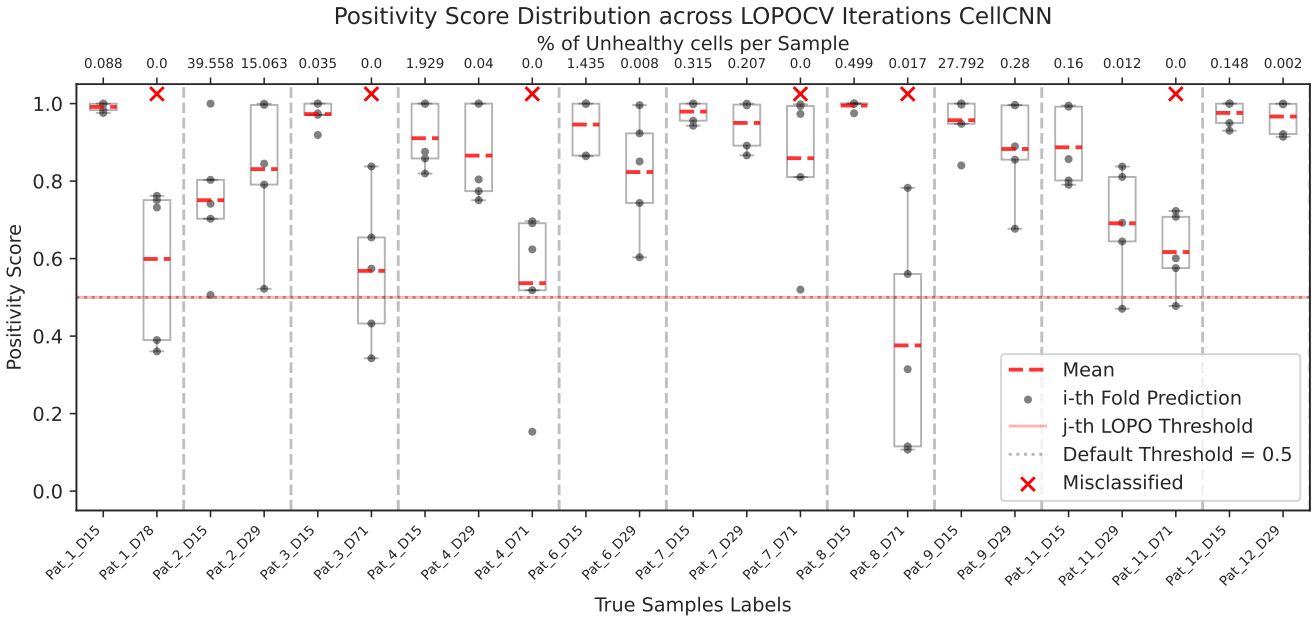


Figure 27: Positivity score distribution per sample across LOPOCV iterations under $\tau = 0.5$ (Experiment 4). Each box summarises the $K = 5$ Fold-Level predictions for that sample. The orange horizontal lines indicate the tuned threshold $\tau = 0.5$ at each LOPOCV iteration. The percentages of unhealthy cells are reported on the top x-axis, and the dotted horizontal line marks the default threshold $\tau = 0.5$. The bottom x-axis shows the samples ID. Misclassified samples are indicated by a red **x** marker at the top of the plot. Finally, the vertical dashed lines separates the LOPOCV iterations. LOPOCV is the abbreviation of Leave-One-Patient-Out Cross Validation.

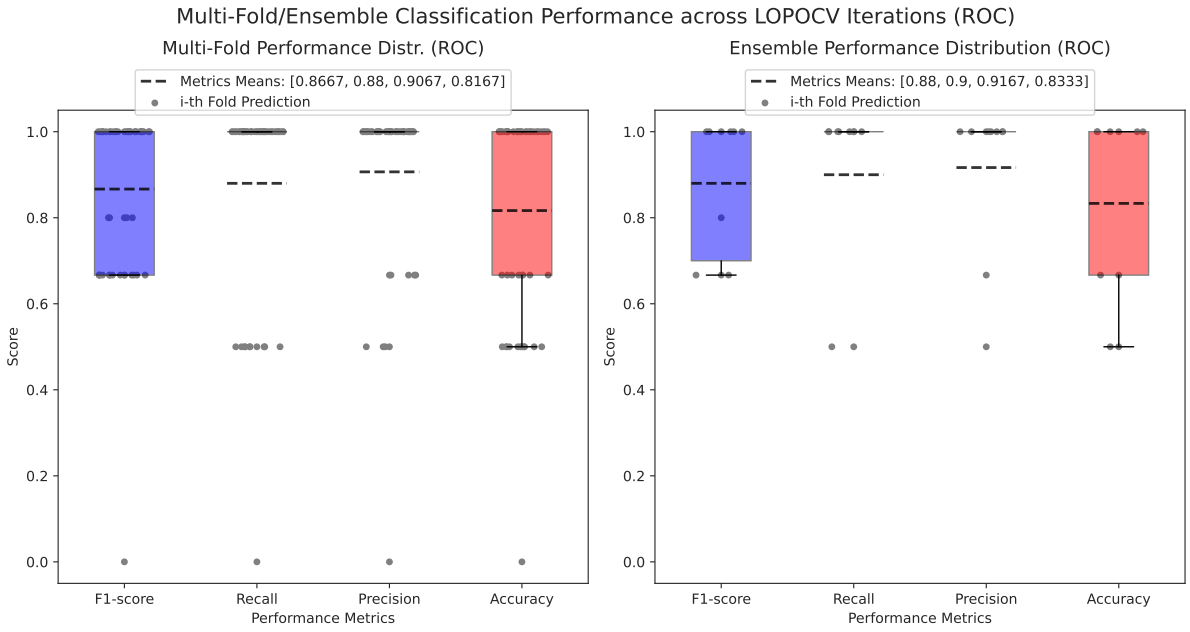


Figure 28: Performance distribution of the CellCNN approach ($\tau = 0.5$) across LOPOCV iterations (Experiment 4). *Left*: performance distribution of the Fold-Level predictions. *Right*: performance distribution of the Ensemble procedure, which averages predictions across $K = 5$ folds. The single grey points in the *Left* plots represent the $K \times N_{LOPOCV} = 50$ Fold-Level predictions, while in the *Right* plots, each point represents one of the $N_{LOPOCV} = 10$ fold-averaged predictions. The black dashed lines indicate the mean across all iterations.

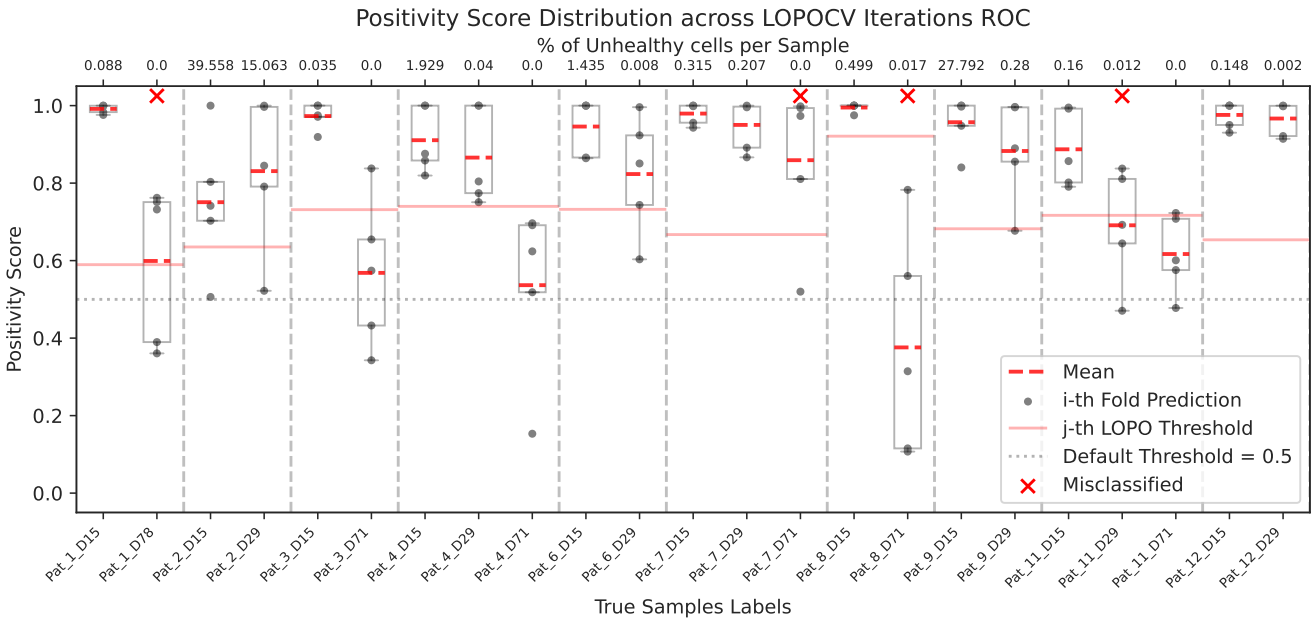


Figure 29: Positivity score distribution per sample across LOPOCV iterations under τ^*_{ROC} (Experiment 4). Each box summarises the $K = 5$ Fold-Level predictions for that sample. The orange horizontal lines indicate the tuned threshold τ^*_{RES} at each LOPOCV iteration. The percentages of unhealthy cells are reported on the top x-axis, and the dotted horizontal line marks the default threshold $\tau = 0.5$. The bottom x-axis shows the ground truth labels of each sample. Misclassified samples are indicated by a red **x** marker at the top of the plot. Finally, the vertical dashed lines separates the LOPOCV iterations. LOPOCV is the abbreviation of Leave-One-Patient-Out Cross Validation.

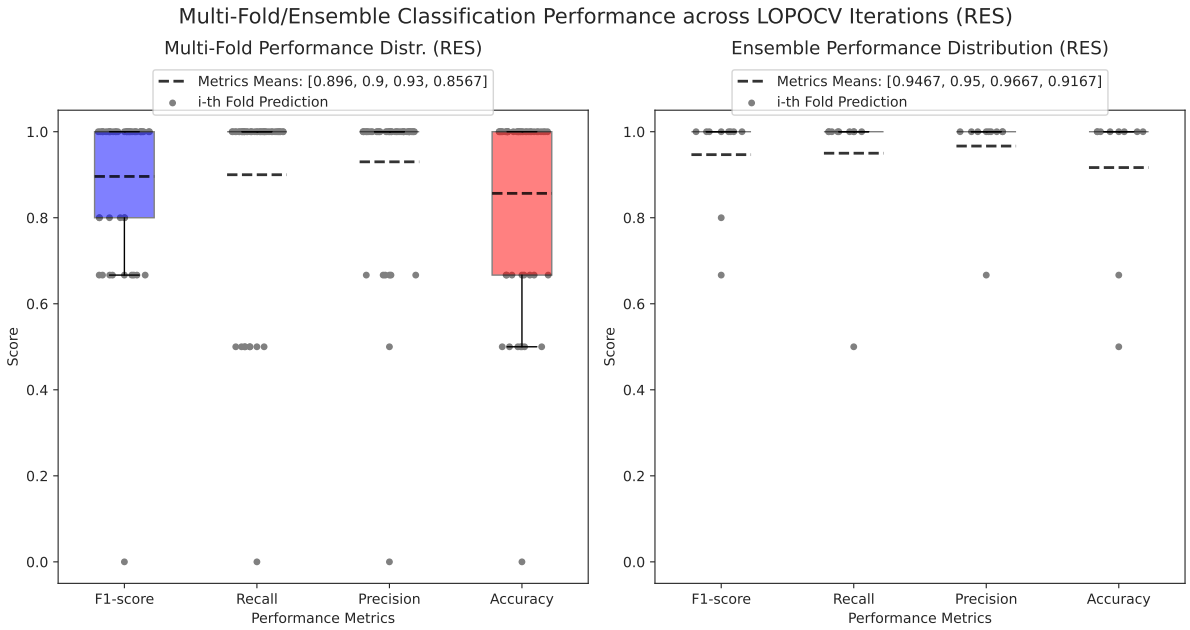


Figure 30: Performance distribution of the Resampling approach with τ^*_{RES} across LOPOCV iterations (Experiment 4). *Left*: performance distribution of the Fold-Level predictions. *Right*: performance distribution of the Ensemble procedure, which averages predictions across $K = 5$ folds. The single grey points in the *Left* plots represent the $K \times N_{LOPOCV} = 50$ Fold-Level predictions, while in the *Right* plots, each point represents one of the $N_{LOPOCV} = 10$ fold-averaged predictions. The black dashed lines indicate the mean across all iterations.

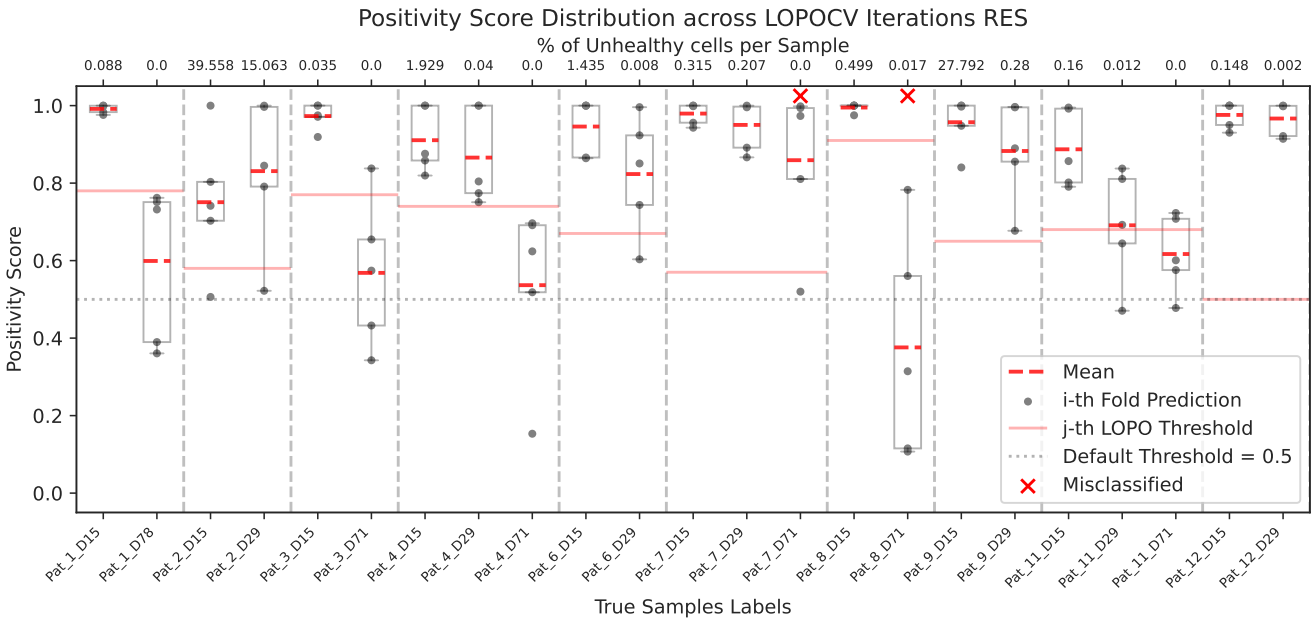


Figure 31: Positivity score distribution per sample across LOPOCV iterations under τ^*_{RES} (Experiment 4). Each box summarises the $K = 5$ Fold-Level predictions for that sample. The orange horizontal lines indicate the tuned threshold τ^*_{RES} at each LOPOCV iteration. The percentages of unhealthy cells are reported on the top x-axis, and the dotted horizontal line marks the default threshold $\tau = 0.5$. The bottom x-axis shows the ground truth labels of each sample. Misclassified samples are indicated by a red **x** marker at the top of the plot. Finally, the vertical dashed lines separates the LOPOCV iterations. LOPOCV is the abbreviation of Leave-One-Patient-Out Cross Validation.

4.3.5 Experiments 1-4: performance metrics comparison

To achieve a better comparison of the computed results, in Figures 32, 33, 34, 35, we plotted scores per performance metrics yielded by each classification approach in each experiment.

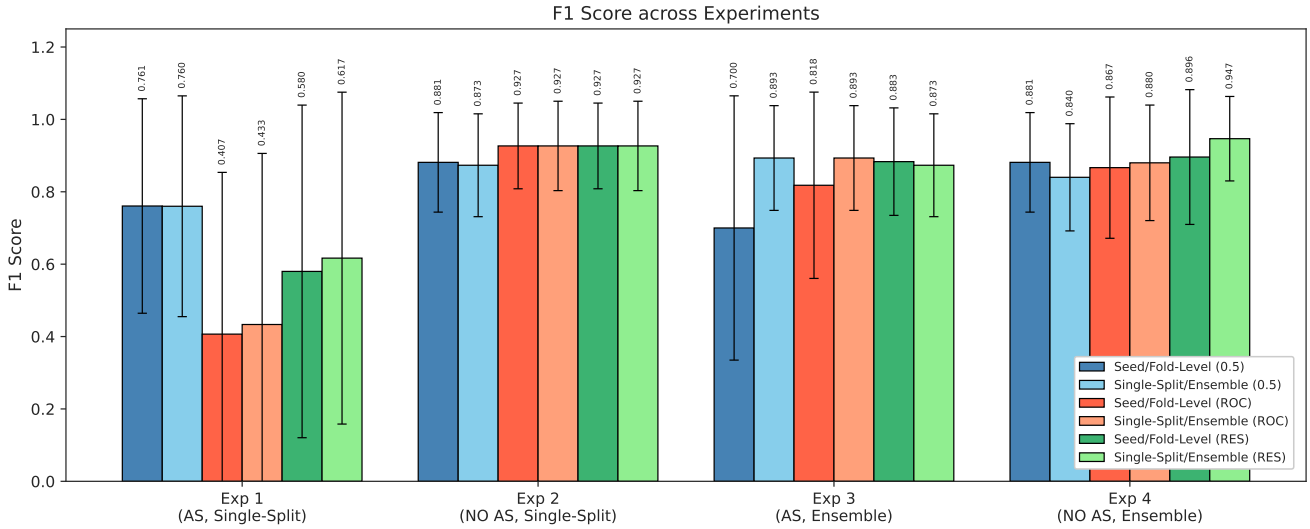


Figure 32: Comparison of F_1-score across the four Experiments. In all Experiments, 1st, 3rd, 5th bars represent aggregated predictions ($T \times N_{LOPOCV} = 50$ for Experiments 1 and 2, $K \times N_{LOPOCV} = 50$ for Experiments 3 and 4), and 2nd, 4th, 6th bars represent the average across LOPOCV iterations ($N_{LOPOCV} = 10$) evaluated under thresholds $\tau = 0.5$, $\tau^{*(BO)}_{ROC}$, $\tau^{*(BO)}_{RES}$ for Experiments 1 and 2 and thresholds $\tau = 0.5$, τ^{*}_{ROC}, τ^{*}_{RES} for Experiments 3 and 4. AS and NO-AS denote the scenarios with and without Artificial Samples, respectively.

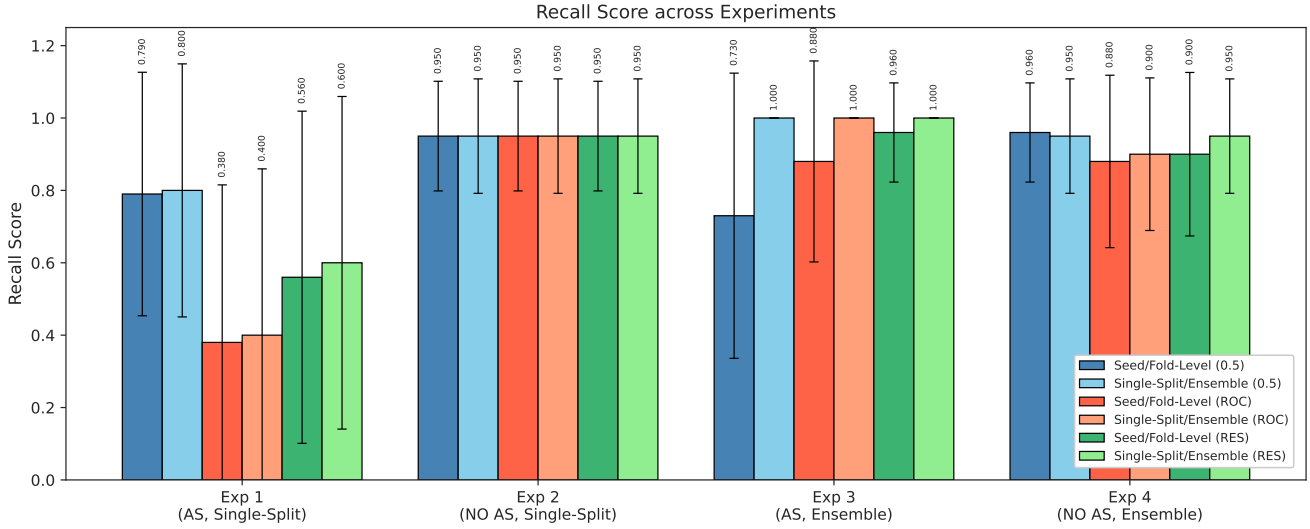


Figure 33: Comparison of Recall across the four Experiments. In all Experiments, 1st, 3rd, 5th bars represent aggregated predictions ($T \times N_{LOPOCV} = 50$ for Experiments 1 and 2, $K \times N_{LOPOCV} = 50$ for Experiments 3 and 4), and 2nd, 4th, 6th bars represent the average across LOPOCV iterations ($N_{LOPOCV} = 10$) evaluated under thresholds $\tau = 0.5$, $\tau^{*(BO)}_{ROC}$, $\tau^{*(BO)}_{RES}$ for Experiments 1 and 2 and thresholds $\tau = 0.5$, τ^{*}_{ROC}, τ^{*}_{RES} for Experiments 3 and 4. AS and NO-AS denote the scenarios with and without Artificial Samples, respectively.

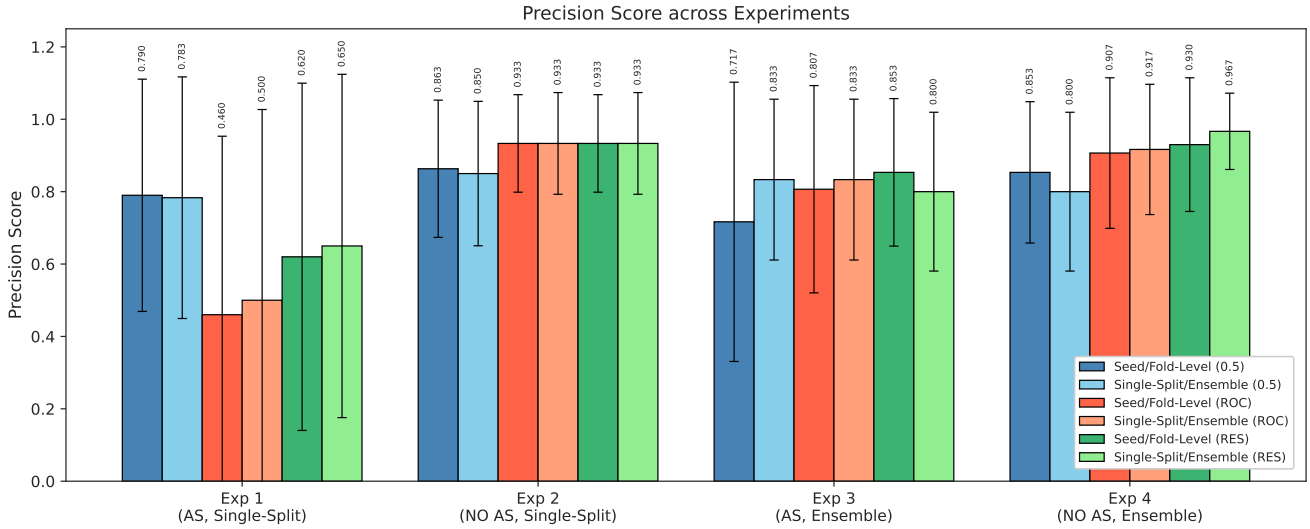


Figure 34: Comparison of Precision across the four Experiments. In all Experiments, 1st, 3rd, 5th bars represent aggregated predictions ($T \times N_{LOPOCV} = 50$ for Experiments 1 and 2, $K \times N_{LOPOCV} = 50$ for Experiments 3 and 4), and 2nd, 4th, 6th bars represent the average across LOPOCV iterations ($N_{LOPOCV} = 10$) evaluated under thresholds $\tau = 0.5$, $\tau^{*(BO)}_{ROC}$, $\tau^{*(BO)}_{RES}$ for Experiments 1 and 2 and thresholds $\tau = 0.5$, τ^{*}_{ROC}, τ^{*}_{RES} for Experiments 3 and 4. AS and NO-AS denote the scenarios with and without Artificial Samples, respectively.

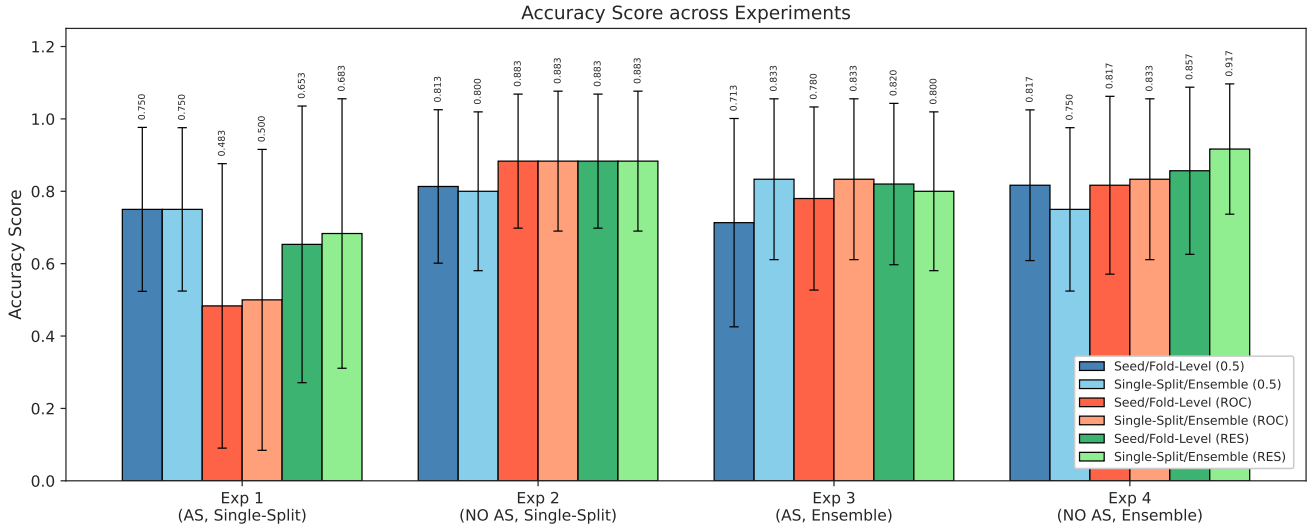


Figure 35: Comparison of Accuracy across the four Experiments. In all Experiments, 1st, 3rd, 5th bars represent aggregated predictions ($T \times N_{LOPOCV} = 50$ for Experiments 1 and 2, $K \times N_{LOPOCV} = 50$ for Experiments 3 and 4), and 2nd, 4th, 6th bars represent the average across LOPOCV iterations ($N_{LOPOCV} = 10$) evaluated under thresholds $\tau = 0.5$, $\tau^{*(BO)}_{ROC}$, $\tau^{*(BO)}_{RES}$ for Experiments 1 and 2 and thresholds $\tau = 0.5$, τ^{*}_{ROC}, τ^{*}_{RES} for Experiments 3 and 4. AS and NO-AS denote the scenarios with and without Artificial Samples, respectively.

4.3.6 Ensemble Misclassification Table

	Experiment 1			Experiment 2			Experiment 3			Experiment 4		
Sample ID (%)	0.5	ROC	RES	0.5	ROC	RES	0.5	ROC	RES	0.5	ROC	RES
Pat_1_D15 (0.088%)	x	x	x	·	·	·	·	·	·	·	·	·
Pat_1_D78 (0%)	·	·	·	·	·	·	x	x	x	x	x	·
Pat_2_D15 (39.558%)	·	·	·	·	·	·	·	·	·	·	·	·
Pat_2_D29 (15.036%)	·	·	·	·	·	·	·	·	·	·	·	·
Pat_3_D15 (0.035%)	·	·	·	·	·	·	·	·	·	·	·	·
Pat_3_D71 (0%)	x	·	·	x	·	·	x	x	x	x	·	·
Pat_4_D15 (1.929%)	·	·	·	·	·	·	·	·	·	·	·	·
Pat_4_D29 (0.04%)	x	x	x	·	·	·	·	·	·	·	·	·
Pat_4_D71 (0%)	·	·	·	x	·	·	·	·	·	x	·	·
Pat_6_D15 (1.435%)	·	·	·	·	·	·	·	·	·	·	·	·
Pat_6_D29 (0.008%)	·	x	·	·	·	·	·	·	·	·	·	·
Pat_7_D15 (0.315%)	·	x	x	·	·	·	·	·	·	·	·	·
Pat_7_D29 (0.207%)	·	x	x	·	·	·	·	·	·	·	·	·
Pat_7_D71 (0%)	x	·	·	x	x	x	x	x	x	x	x	x
Pat_8_D15 (0.499%)	·	x	x	·	·	·	·	·	·	·	·	·
Pat_8_D71 (0.017%)	x	x	x	x	x	x	·	·	·	x	x	x
Pat_9_D15 (27.792%)	·	·	·	·	·	·	·	·	·	·	·	·
Pat_9_D29 (0.28%)	·	·	·	·	·	·	·	·	·	·	·	·
Pat_11_D15 (0.16%)	·	x	·	·	·	·	·	·	·	·	·	·
Pat_11_D29 (0.012%)	·	x	x	·	·	·	·	·	·	·	x	·
Pat_11_D71 (0%)	x	x	x	x	x	x	x	x	x	x	·	·
Pat_12_D15 (0.148%)	·	x	·	·	·	·	·	·	·	·	·	·
Pat_12_D29 (0.002%)	·	x	·	·	·	·	·	·	·	·	·	·

Healthy Misclassified	3	1	1	4	2	2	4	4	4	5	2	1
Unhealthy Misclassified	2	11	7	1	1	1	0	0	0	1	2	1
TOT Misclassified	5	12	8	5	3	3	4	4	4	6	4	2

Table 30: Ensemble misclassification Table. For each Experiment, the three thresold approaches are reported (default $\tau = 0.5$, ROC-tuned and RES-tuned). "x"'s represent a misclassification and "·"'s represent a correct classification. Each element is obtained by taking the average of the $T = 5$ prediction (for Experiments 1 and 2) or $K = 5$ predictions (for Experiments 3 and 4) and classifying accordingly to the respective Leave-One-Patient-Out Cross Validation (LOPOCV) iteration tuned thresholds. The column "Sample ID" includes the percentage of unhealthy cells within the sample. Healthy samples are marked by percentage = 0%. Healthy/Unhealthy cells distribution and Unhealthy cells % distribution. The notation "Pat_Y_DX" denotes the sample collected on Day X from Patient Y.

4.3.7 Experiments Comparison

The results of Experiments 1-4 show multiple aspects of the proposed pipeline under different training data conditions and aggregation strategies. Experiments are discussed in the order 1, 3, 2, 4 to easily compare AS and no-AS settings under the same aggregation strategy.

Experiment 1 In Experiment 1, the higher performance of the CellCNN approach compared to the Resampling-based approaches suggests that the fixed default threshold performs better in settings where AS are employed. In this context, the default threshold $\tau = 0.5$ achieved better results by reducing the misclassification of unhealthy samples.
Generally, Experiment 1 performed worse than the others under both the CellCNN and Resampling Prediction Approaches and across all three thresholds (default, ROC-tuned and RES-tuned). This confirms that, even if the CellCNN approach outperformed the Resampling approach in Experiment 1, the model is not able to discriminate properly between the unhealthy and healthy classes in this setting.

Experiment 3 While Experiment 3 generally yielded better classifications with respect to Experiment 1, both Experiments performed worse than their no-AS counterparts, further confirming the inability of models trained with AS, at least in this settings, to discriminate correctly between the two classes.
Experiment 3 showed that the Fold-Ensemble strategy mitigated the instability observed in Experiment 1, at least for small dataset studies. While Experiment 1 showed negligible differences between *Seed-Level* and *Single-Split* aggregation, Experiment 3 showed that averaging predictions across different training sets produced performance gains, particularly for the CellCNN approach.

After ensemble, both CellCNN and Resampling approaches scored Recall= 1. This suggests that the ensemble mitigates the high variance introduced by individual fold partitions. However, analysing the other metrics in Figures 32, 34, 35 and thresholds values in Table 25, it is evident that the perfect Recall was achieved not through to an improved ability to discriminate between unhealthy and healthy classes, but because the tuned threshold values were overconservative, generally adopting values even lower to the default value $\tau = 0.5$. The result is then an inability to correctly detect healthy samples, as further supported in Table 30, which shows that the ensemble in this Experiment correctly predicted the unhealthy samples, but failed to classify any healthy sample correctly.

Experiment 2 The differences in metric scores between *Seed-Level* and *Single-Split* remained small ($< 2\%$) across aproaches, suggesting that weight initialization plays a marginal role in this context.
Experiment 2 showed general performance improvement across all approaches relative to Experiment 1. Since Experiment 2 used patient samples instead of AS, and the two Experiments differ only in the training data, the better performance can be attributed to patient samples. Indeed, while patient samples contain a number of cells ranging from 10^5 to 10^6, every AS generated contains exactly $q = 10^5$ cells and only 6 AS (3 unhealthy and 3 healthy) are generated per patient, effectively discarding thousands of unique cells (both healthy and unhealthy) that the model had never processed. However, a deeper analysis of AS performance limitations is provided in Section 5.2.

It is important to notice that in some performance metrics and Prediction Approaches, Experiment 2 performed better than Experiment 4. However, Experiment 2 may be particularly biased given that all its predictions were made on the same patient split. This bias is further suggested by the results of Experiment 4, where aggregated predictions generally reduced variability and increased classification performance. This makes a direct comparison between Training Procedures difficult, but it suggests that weights initialization has a more limited impact in performance compared to patient split.

Experiment 4 Experiment 4 provides a direct comparison with Experiment 3, as both employed the Fold-Ensemble strategy but differ in the training data. The decrease in ensemble F_1-score for the CellCNN approach (F_1-score: $0.8813 \rightarrow 0.84$) contrasted the improvements achieved by the other approaches and those observed in Experiment 3 (F_1-score: $0.70 \rightarrow 0.8933$).
This may suggest that the Fold-Ensemble performs better when training data exhibit low internal variability, as in the AS setting where the percentage of unhealthy cells are fixed by the parameter u. Due to the small dataset used, this hypothesis should be further investigated. When trained on original samples, the Fold-Level predictions already showed good performance, and ensemble aggregation provided a more modest effect, or even a slightly negative one in the case of the CellCNN approach.

By contrast, the Resampling approaches benefited more from aggregation, with τ^*_{RES} achieving the best overall results across Experiments 2 and 4 (F_1-score $= 0.9467$, Precision $= 0.9667$, Accuracy $= 0.9167$). Compared to Experiment 2, the Fold-Ensemble with τ^*_{RES} showed a clear improvement (F_1-score : $0.9267 \rightarrow 0.9467$), also supported by the Table 30, where Experiment 4, with RES-tuned thresholds, has achieved the smallest number of misclassified samples. This confirms that aggregating across different training subsets provided a benefit over aggregating across weight initializations, and that choosing a threshold balanced between Recall and Precision may provide a reliable alternative to Recall-biased thresholds, such as the ROC-tuned ones. Despite these results, it is important to note that one of the two misclassified samples is a unhealthy sample, meaning that a full minimization of the Type II error is not achieved, outcome reached only by Experiment 3 at the cost of misclassifying all healthy samples.

Sample-Level Analysis A note about Experiments 2 and 4 is that both correctly predicted, with low variability across inner ensemble CV folds, the sample f_2^{12} (Pat_12_D29), which shows an extremely low percentage of unhealthy cells (0.002%).
Notable is also the case of the sample f_2^8 (Pat_8_D71), an unhealthy sample that was misclassified as healthy in three out of four Experiments. Across all Experiments, it showed high variability in Positivity Score across folds and seeds. In Experiments employing AS, the variability of the Positivity Score is reduced compared to the non-AS Experiments suggesting that, while AS did not improve overall classification performance, they may stabilize predictions for samples with unusual or underrepresented cellular patterns.

Since the MRD percentage of sample f_2^8 is 0.017%, which falls within in the range of unhealthy cells in the generated AS, this behaviour might be attributed to the low MRD percentages of the AS. However, this explanation only is not sufficient. Indeed, in Experiments with AS, healthy samples and samples with lower MRD percentages were associated with higher Positivity Scores than f_2^8. On the other hand, in Experiments without AS, both healthy samples and samples with lower MRD percentages were correctly classified. These observations may suggest that the difficulty in classifying f_2^8 does not depend only on MRD percentage, but may reflect an unusual cellular pattern that is more difficult to detect regardless of the Training Procedure.

5 Discussion

5.1 Answers to the Research Questions

Does CellCNN benefit from Artificial Sample generation? A direct comparison between the Experiments is not straightforward because Experiments 1 and 4 differ both in the training data (AS vs. patients' samples) and in the aggregation strategy (single-split vs. fold ensemble). As a result, separating the specific effect of AS generation from that of the training strategy is difficult. When each training procedure is considered separately, however, the Experiments using AS generally showed lower performance than those trained on patient samples. This observation supports the hypothesis that models trained with AS have greater difficulty to separate the classes, possibly due to the mismatch between the AS size generated in this study and the size of patients' samples and the too low fixed percentages of unhealthy cells included in the AS.

Does an ensemble approach achieve better results in terms of both predictive performance and robustness compared to the Fold-Level and Seed-Level baselines? In both the Fold-Ensemble (Section 3.8.3) and Single-Split (Section 3.8.2) settings, which consists of averaging across K folds or T seeds predictions respectively, the aggregated predictions generally performed better than the corresponding Fold or Seed-Level baselines both in classification and variance reduction. While the proposed procedure analyses a different dataset and proposes a variant of K-Fold CV and Bagging, these findings are consistent with the ensemble learning literature [36]. This suggests that ensemble averaging reduces the variance inherent in the original CellCNN implementation [3].

Comparing Experiments 1 with 3 and Experiments 2 with 4, it can be seen that Experiments 3 and 4 exhibited the largest reduction in variance relative to the CellCNN baseline, with generally wider distributions of Positivity Scores across samples (Figures 9- 31). This suggests that the dominant source of variance is the training dataset composition. Indeed, the standard deviation decreases more from Fold-Level to Fold-Ensemble than from Seed-Level to Single-Split, indicating that different combinations of patients across training subsets have a higher impact on performance variability than weight initialization alone.

Do ROC-tuned or RES-tuned thresholds produce better classification compared to the classical $\tau = 0.5$ baseline? Across the Experiments, the tuned thresholds τ_ζ^* generally produced better classification performance than the fixed default threshold, with τ_{RES}^* consistently performing better than τ_{ROC}^*, particularly in the ensemble setting. The exception was the Experiment 1, where the fixed threshold $\tau = 0.5$ achieved higher performance, likely due to the poor prediction ability of the model trained on a single patient split with AS. In particular RES-tuned threshold in Fold Ensemble setting achieved the highest F_1-score, Precision and Accuracy across all Experiments.

General Observations The high variability observed across LOPOCV folds, including complete misclassification (Accuracy = 0.0) in multiple iterations, reflects the instability introduced by the limited dataset size. With only 2-3 samples available in the test set per LOPOCV fold, individual splits may generate non representative training-test partitions, making robust generalisation more difficult.
Finally, the BO [10] is a deterministic methodology for the choice of θ^*, which is a central parameter of the CellCNN model. Although a formal comparison with alternative strategies for θ^* selection was not performed, the results suggest that thresholds optimized from the BO validation predictions can improve with respect to the original fixed-threshold classification strategy, in particular in the patients' samples setting.

5.2 Limitations

Artificial Samples are generated using fixed and very low percentages of unhealthy cells (0.01%, 0.05%, 0.1%) at patient level, with a fixed size of $q = 10^5$ cells per sample. This design may not reflect the biological variability of true patient samples. In fact, patient samples contain between 10^5 and 10^6 cells, thus AS are considerably smaller, and only 6 AS (3 unhealthy, 3 healthy) are generated per patient, effectively discarding a large proportion of unique cells (both healthy or unhealthy). If the AS size is small relative to patients' samples, the model is exposed to a too low number of cells during training, which may reduce its discriminative ability.
Additionally, for patients presenting only low percentages of unhealthy cells across all samples, AS are generated by sampling with replacement, which may introduce duplicate cells in the AS, potentially further reducing the model's discriminative ability.
To mitigate this, an alternative might be the generation of larger AS reducing cell discarding, to increase the number of AS per patient, or to modify the unhealthy cell percentages to better reflect the range observed in clinical samples. The lower performance observed in AS-trained Experiments supports the need for alternative generation strategies.

The dataset used in all Experiments consists of only 23 samples divided aross 10 patients. The availability of such a small dataset motivated the use of LOPOCV, which results in only $N_{LOPOCV} = 10$ folds containing as few as 2-3 test samples each. In addition, the evaluation relied only on Leave-One-Patient-Out Cross-Validation without an independent held-out test set, impacting on the dimension of the training dataset. This constrains the statistical reliability of the reported metrics and limits the generalisability of the conclusions. With such a small number of samples in test set, individual metrics like F_1-score and Recall only may not correctly summarise the model performances. For instance, if the test set contains only a single healthy sample and it is correctly classified, the F_1-score is zero, despite the correct prediction.
Due to this limitation, it is not possible to compute and present derived metrics such as the Matthews Correlation Coefficient (MCC) [6, 25], which would provide a more comprehensive summary across all metrics outcomes. To partially mitigate this limitation, we reported multiple performance metrics (F_1-score, Recall, Precision and Accuracy), providing a more complete picture of classification outcomes. A more robust evaluation would require a larger test set, which remains a direction for future work.

Finally, an important limitation of this study is the computational cost. Indeed, the entire pipeline was time-intensive, particularly for the BO and Ensemble approaches. In BO, K models were trained for each configuration proposed by the optimizer, yielding $K \times n_\theta = 5 \times 15 = 75$ CellCNN pipelines per LOPOCV iteration. Considering also the internal hyperparameter tuning performed by CellCNN (random search in BO for 10 combinations, see Table 7), this resulted in a total of $10 \times 75 = 750$ models trained in BO alone per LOPOCV iteration.
Including the Ensemble approach, the total rises to $750 + 120 = 870$ models trained per LOPOCV iteration, where $120 = K \times 24$ and 24 is the number of hyperparameter combinations used in the internal CellCNN grid search during final ensemble training (see Table 7).

As described in section 3.9.1, all Experiments were run on CPU due to package version conflicts, and parameters were selected under hardware constraints. Increasing the number of epochs, or the values of θ^{max}, n_θ and CellCNN internal hyperparameters, would considerably increase the computational cost, even with powerful GPUs. These limitations motivate the future directions proposed in the next section.

5.3 Future Works

The limitations and observations of this study suggest several directions for future work.

First, the AS generation strategy could be refined to better reflect the biological variability of true patient samples. In the current implementation, AS are generated at the patient level with fixed unhealthy cell percentages. Alternatively, AS may be generated at sample level, building a higher number of AS without mixing samples from the same patient. The AS size may be increased to reduce the number of discarded cells (both healthy and unhealthy). Additionally, a training set that comprises both patient samples and AS may be generated, enriching it with samples containing fixed percentages of unhealthy cells. This is motivated by the observation that sample f_2^8, which exhibited high variability in Experiments trained on patient samples, was predicted more consistently in Experiments employing AS, indicating that this specific augmentation approach may help the model to generalise to unusual cellular patterns.

Second, in the current implementation, patient samples are used in the BO phase even for Experiments whose final model is trained on AS. To better generalise the impact of the AS, they might be employed even in the BO phase, tuning an optimal configuration θ^* better suited for settings that employ AS and derive more appropriate tuned thresholds $\tau_\zeta^{*(BO)}$.
Third, the default value of the training batch size is fixed at 200 subsets [2], where a subset contains n_{cells} cells, taking as CNN input $200 \times n_{cells}$. Depending on the value n_{cells}, the input taken to the CNN may vary considerably. A potential modification would be to align the batch size with the subset size used in the resampling prediction approach, so that the model is trained directly on a similar input size (in our implementation is of 100 000 cells), performing a variant of Batch Augmentation [16]. This may reduce the batch size mismatch between training and test conditions and improve the consistency of the learned representations.

Fourth, the CellCNN framework includes an internal mechanism for estimating the number of cells that belongs to a given class and to identify unhealthy cells. This is controlled by a cell-level classification threshold, which is currently set to zero by the original CellCNN implementation [2]. Similarly to the sample-level threshold tuning proposed in this study, tuning this threshold could improve the quality of the cell-level predictions.

Fifth, the current pipeline separates the BO phase from the final training phase, requiring the model to be retrained from scratch on the K CV folds (or using T different weights initializations) after θ^* has been selected. An alternative would be to use the models trained during BO directly for both threshold tuning and inference, without additional retraining. Under this approach, the selection of θ^* would also find the optimal internal hyperparameters of the model at the same time, requiring though a higher number of epochs. This would reduce computational cost, replacing the full retraining on K folds with a single BO phase, while still allowing a final high-epoch retraining on the selected configuration if computational resources permit.
Finally, future work should include an analysis of the CellCNN filter responses, which represent a key tool to intepret and understand sample misclassifications. In particular, inspecting the filters activated by regularly misclassified samples could reveal specific cell subpopulations that are difficult to characterize, providing insights into the sources of misclassification.

6 Bibliography

References

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, and Xiaoqiang Zhang. Tensorflow: A system for large-scale machine learning. 2016.

[2] Eirini Arvaniti and Manfred Claassen. Cellcnn. `https://github.com/eiriniar/CellCnn/tree/master/cellCnn`, 2017. Accessed: 2025-09-01.

[3] Eirini Arvaniti and Manfred Claassen. Sensitive detection of rare disease-associated cell subsets via representation learning. *Nature Communications*, page 14825, 2017.

[4] Sean Bendall, Erin Simonds, Peng Qiu, El-ad Amir, Peter Krutzik, Rachel Finck, Robert Bruggner, Rachel Melamed, Angelica Trejo, Olga Ornatsky, Robert Balderas, Sylvia Plevritis, Karen Sachs, Dana Pe'er, Scott Tanner, and Garry Nolan. Single-cell mass cytometry of differential immune and drug responses across a human hematopoietic continuum. *Science (New York, N.Y.)*, 332:687–96, 2011.

[5] Marie C. Béné. Immunophenotyping of acute leukaemias. *Immunology Letters*, pages 9–21, 2005.

[6] Davide Chicco, Matthijs Warrens, and Giuseppe Jurman. The matthews correlation coefficient (mcc) is more reliable than balanced accuracy, bookmaker informedness, and markedness in two-class confusion matrix evaluation. *IEEE Access*, page 1, 2021.

[7] D. Wackerly Dennis, Mendenhall III William, and Richard L. Scheaffer. *John E. Freund's Mathematical Statistics With Applications 7th Ed.* Duxbury Advanced Series, 2008.

[8] Joseph DiGiuseppe, Michelle Tadmor, and Dana Pe'er. Detection of minimal residual disease in b lymphoblastic leukemia using visne. *Cytometry. Part B, Clinical cytometry*, 88, 2015.

[9] Annelies Emmaneel, Katrien Quintelier, Dorine Sichien, Paulina Rybakowska, Concepción Marañón, Marta E. Alarcón-Riquelme, Gert Van Isterdael, Sofie Van Gassen, and Yvan Saeys. Peacoqc: Peak-based selection of high quality cytometry data. *Cytometry Part A*, 101:325–338, 2022.

[10] Peter I. Frazier. A tutorial on bayesian optimization, 2018.

[11] James Gareth, Witten Daniela, Hastie Trevor, and Tibshirani Robert. *An introduction to statistical learning: with applications in r*. Springer New York, NY, 2021.

[12] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[13] Google. Google colab. `https://colab.research.google.com`, 2025. Accessed: 2025-02-20.

[14] Charles Harris, K Millman, Stéfan Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten Kerkwijk, Matthew Brett, Allan Haldane, Jaime Río, Mark Wiebe, Pearu Peterson, and Travis Oliphant. Array programming with numpy. *Nature*, 585:357–362, 2020.

[15] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Second Edition (Springer Series in Statistics)*. 02 2009.

[16] Elad Hoffer, Tal Ben-Nun, Itay Hubara, Niv Giladi, Torsten Hoefler, and Daniel Soudry. *Augment your batch: Improving generalization through instance repetition.* 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020.

[17] Kenneth Kaushansky, Josef T. Prchal, Linda J. Burns, Marshall A. Lichtman, Marcel Levi, and David C. Linch. *Williams hematology, 10e*. McGraw-Hill Education, 2021.

[18] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations*, 12 2014.

[19] Sotiris Kotsiantis, D. Kanellopoulos, and P. Pintelas. Handling imbalanced datasets: A review. *GESTS International Transactions on Computer Science and Engineering*, 30:25–36, 11 2005.

[20] Aaron Kruse, Nour Abdel-Azim, Hye Na Kim, Yongsheng Ruan, Valerie Phan, Heather Ogana, William Wang, Rachel Lee, Eun Ji Gang, Sajad Khazal, and Yong-Mi Kim. Minimal residual disease detection in acute lymphoblastic leukemia. *International Journal of Molecular Sciences*, 2020.

[21] Weijie Li. *Leukemia*. Exon Publications, 2022.

[22] Hongyan Liao, Nenggang Jiang, Ying Yang, Xin Zhang, Jiao Chen, Hongli Lai, and Qin Zheng. Association of minimal residual disease by a single-tube 8-color flow cytometric analysis with clinical outcome in adult b-cell acute lymphoblastic leukemia. *Archives of pathology and laboratory medicine*, 147, 2022.

[23] Wes McKinney. Data structures for statistical computing in python. pages 56–61, 01 2010.

[24] Kenneth Murphy and Casey Weaver. Janeway's immunobiology. *New York, NY: Garland Science/Taylor and Francis Group, LLC*, page 4, 2017.

[25] David Núñez Nepomuceno, José A. Sáez, and Pedro Carmona-Saez. A survey of preprocessing techniques for flow cytometry data in classification tasks. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 22:3112–3130, 2025.

[26] David Parks, Mario Roederer, and Wayne Moore. A new "logicle" display method avoids deceptive effects of logarithmic scaling for low signals and compensated data. *Cytometry. Part A : the journal of the International Society for Analytical Cytology*, 69:541–51, 2006.

[27] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, Edouard Duchesnay, and Gilles Louppe. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12, 2012.

[28] Simon J.D. Prince. *Understanding Deep Learning*. The MIT Press, 2023.

[29] Katrien Quintelier, Artuur Couckuyt, Annelies Emmaneel, Joachim Aerts, Yvan Saeys, and Sofie Van Gassen. Analyzing high-dimensional cytometry data using flowsom. *Nature Protocols*, 16:1–27, 2021.

[30] Susan Rheingold, Deepa Bhojwani, Lingyun Ji, Xinxin Xu, Meenakshi Devidas, John Kairalla, Mary Shago, Nyla Heerema, Andrew Carroll, Heather Breidenbach, Michael Borowitz, Brent Wood, Anne Angiolillo, Barbara Asselin, W. Bowman, Patrick Brown,

Zoann Dreyer, Kimberly Dunsmore, Joanne Hilden, and Mignon Loh. Determinants of survival after first relapse of acute lymphoblastic leukemia: a children's oncology group study. *Leukemia*, 38:2382–2394, 2024.

[31] Yvan Saeys, Sofie Van Gassen, and Bart Lambrecht. Computational flow cytometry: Helping to make sense of high-dimensional immunology data. *Nature Reviews Immunology*, 16, 2016.

[32] Claude Sammut and Geoffrey Webb. Encyclopedia of machine learning, 2010.

[33] Martijn Verbeek, Chiara Buracchi, Anna Laqua, Stefan Nierkens, Lukasz Sedek, Juan Flores-Montero, Mattias Hofmans, Elaine Costa, Michaela Novakova, Ester Mejstrikova, Susana Barrena, Saskia Kohlscheen, Monika Szczepanowski, Jan Kulis, Elen Oliveira, Romana Jugooa, Anja Jong, Tomasz Szczepanski, Jan Philippé, and Vincent H J van der Velden. Flow cytometric minimal residual disease assessment in b-cell precursor acute lymphoblastic leukaemia patients treated with cd19-targeted therapies — a euroflow study. *British Journal of Haematology*, 197, 2021.

[34] Li W. Measurable residual disease testing in acute leukemia: Technology and clinical significance. *Li W. editor. Leukemia. Brisbane (AU): Exon Publications.*, Online first 21 Jul 202.

[35] W. J. Youden. Index for rating diagnostic tests. *Cancer*, 3:32–35, 1950.

[36] Zhi-Hua Zhou. *Ensemble Methods: Foundations and Algorithms*, volume 14. 2012.